

COMPUTER SCIENCE

Unit 1: Algorithm Analysis

*Comprehensive Notes with Detailed Complexity Analysis,
Visual Diagrams & Correctness Proofs*

Table of Contents

1. Insertion Sort

- 1.1 Algorithm Overview
- 1.2 Complexity Analysis
- 1.3 Properties
- 1.4 Visual Walkthrough
- 1.5 Loop Invariant & Correctness

2. Selection Sort

- 2.1 Algorithm Overview
- 2.2 Complexity Analysis
- 2.3 Properties
- 2.4 Visual Walkthrough
- 2.5 Loop Invariant & Correctness

3. Merge Sort

- 3.1 Algorithm Overview
- 3.2 Complexity Analysis
- 3.3 Properties
- 3.4 Visual Walkthrough
- 3.5 Loop Invariant & Correctness

4. Bubble Sort

- 4.1 Algorithm Overview
- 4.2 Complexity Analysis
- 4.3 Properties
- 4.4 Visual Walkthrough
- 4.5 Loop Invariant & Correctness

5. Heapsort

- 5.1 Algorithm Overview
- 5.2 Complexity Analysis
- 5.3 Properties

5.4 Visual Walkthrough

5.5 Loop Invariant & Correctness

6. Quicksort

6.1 Algorithm Overview

6.2 Complexity Analysis

6.3 Properties

6.4 Visual Walkthrough

6.5 Loop Invariant & Correctness

7. Randomized Quicksort

7.1 Algorithm Overview

7.2 Complexity Analysis

7.3 Properties

7.4 Visual Walkthrough

8. Counting Sort

8.1 Algorithm Overview

8.2 Complexity Analysis

8.3 Properties

8.4 Visual Walkthrough

9. Minimum Element

9.1 Algorithm Overview

9.2 Complexity Analysis

9.3 Visual Walkthrough

10. Simultaneous Min-Max

10.1 Algorithm Overview

10.2 Complexity Analysis

10.3 Visual Walkthrough

11. Strassen's Matrix Multiplication

11.1 Algorithm Overview

11.2 Complexity Analysis

11.3 Properties

11.4 Visual Walkthrough

12. Summary

1. Insertion Sort

1.1 Algorithm Overview

Type: Comparison-based, incremental (online) sorting algorithm.

How it works: Builds a sorted subarray one element at a time. Takes one element from the unsorted portion and inserts it into its correct position in the sorted portion by shifting larger elements to the right.

Best Cases to Use

- Small input sizes ($n \leq 20$)
- Nearly sorted arrays (adaptive property)
- Online sorting (data arriving one by one)
- When stability is required with simple implementation

1.2 Complexity Analysis

INSERTION-SORT(A, n)

```

1  for  $i = 2$  to  $n$   $\rightarrow c_1$  ( $n$  times)
2       $key = A[i]$ 
3      // Insert  $A[i]$  into the sorted subarray  $A[1 : i - 1]$   $\rightarrow c_2$  ( $n - 1$  times)
4       $j = i - 1$ 
5      while  $j > 0$  and  $A[j] > key$   $\rightarrow c_4$  ( $n - 1$  times)
6           $A[j + 1] = A[j]$   $\rightarrow c_5$  ( $\sum_{i=2}^n t_i$  times)
 $\rightarrow c_6$  ( $\sum_{i=2}^n (t_i - 1)$  times)
7           $j = j - 1$ 
8       $A[j + 1] = key$   $\rightarrow c_7$  ( $\sum_{i=2}^n (t_i - 1)$  times)
 $\rightarrow c_8$  ( $n - 1$  times)

```

Where t_i = number of times the **while** loop test executes for each i .

Worst Case (Reverse Sorted Array)

In the worst case, each element must move all the way to the front of the sorted subarray. For each i , the inner **while** loop executes i times, so $t_i = i$.

Step-by-Step Derivation:

$$T(n) = c_1n + c_2(n-1) + c_4(n-1) + c_5 \sum_{i=2}^n i + c_6 \sum_{i=2}^n (i-1) + c_7 \sum_{i=2}^n (i-1) + c$$

Step 1: Evaluate the summations using the arithmetic series formula $\sum_{i=1}^n i = \frac{n(n+1)}{2}$:

$$\sum_{i=2}^n i = \sum_{i=1}^n i - 1 = \frac{n(n+1)}{2} - 1$$

$$\sum_{i=2}^n (i-1) = \sum_{j=1}^{n-1} j = \frac{(n-1)n}{2}$$

Step 2: Substitute back:

$$T(n) = c_1n + c_2(n-1) + c_4(n-1) + c_5 \left[\frac{n(n+1)}{2} - 1 \right] + c_6 \left[\frac{n(n-1)}{2} \right] + c_7$$

Step 3: Expand and collect terms by powers of n :

$$T(n) = \left(\frac{c_5}{2} + \frac{c_6}{2} + \frac{c_7}{2} \right) n^2 + \left(c_1 + c_2 + c_4 + \frac{c_5}{2} - \frac{c_6}{2} - \frac{c_7}{2} + c_8 \right) n - (c_2 + c_4 +$$

Step 4: Let $a = \frac{c_5+c_6+c_7}{2}$, $b = c_1 + c_2 + c_4 + \frac{c_5-c_6-c_7}{2} + c_8$, $d = c_2 + c_4 + c_5 + c_8$:

$$T(n) = an^2 + bn - d$$

Conclusion: Since $a > 0$ (all $c_i > 0$), the dominant term is n^2 .

$$\boxed{T(n) = \Theta(n^2)}$$

Average Case

On average, each element moves halfway through the sorted subarray. So $t_i \approx i/2$.

The summations become:

$$\sum_{i=2}^n \frac{i}{2} = \frac{1}{2} \left(\frac{n(n+1)}{2} - 1 \right) = \Theta(n^2)$$

$$\sum_{i=2}^n \left(\frac{i}{2} - 1 \right) = \Theta(n^2)$$

Even with half the comparisons, the n^2 term still dominates:

$$T(n) = \Theta(n^2)$$

Best Case (Already Sorted)

When the array is already sorted, $t_i = 1$ for all i (the **while** condition fails immediately each time).

Step-by-Step Derivation:

$$T(n) = c_1n + c_2(n-1) + c_4(n-1) + c_5(n-1) + c_8(n-1)$$

$$T(n) = c_1n + (c_2 + c_4 + c_5 + c_8)(n-1)$$

$$T(n) = c_1n + (c_2 + c_4 + c_5 + c_8)n - (c_2 + c_4 + c_5 + c_8)$$

$$T(n) = (c_1 + c_2 + c_4 + c_5 + c_8)n - (c_2 + c_4 + c_5 + c_8) = \Theta(n)$$

1.3 Properties

Property	Value	Reasoning
Stable	YES	When $A[j] > key$ (strictly greater), we shift. Equal elements are not shifted past each other. The relative order of equal elements is preserved because we only move when strictly greater.
In-place	YES	Only uses $O(1)$ extra space (variables: key, i, j). All rearrangements happen within the original array. No auxiliary arrays needed.
Adaptive	YES	Running time depends on input order. For nearly sorted arrays, the inner while loop executes fewer times. Best case $\Theta(n)$ when array is already sorted.

1.4 Visual Walkthrough

Sorting $A = [5, 2, 4, 6, 1, 3]$ using Insertion Sort:

1. Insertion Sort

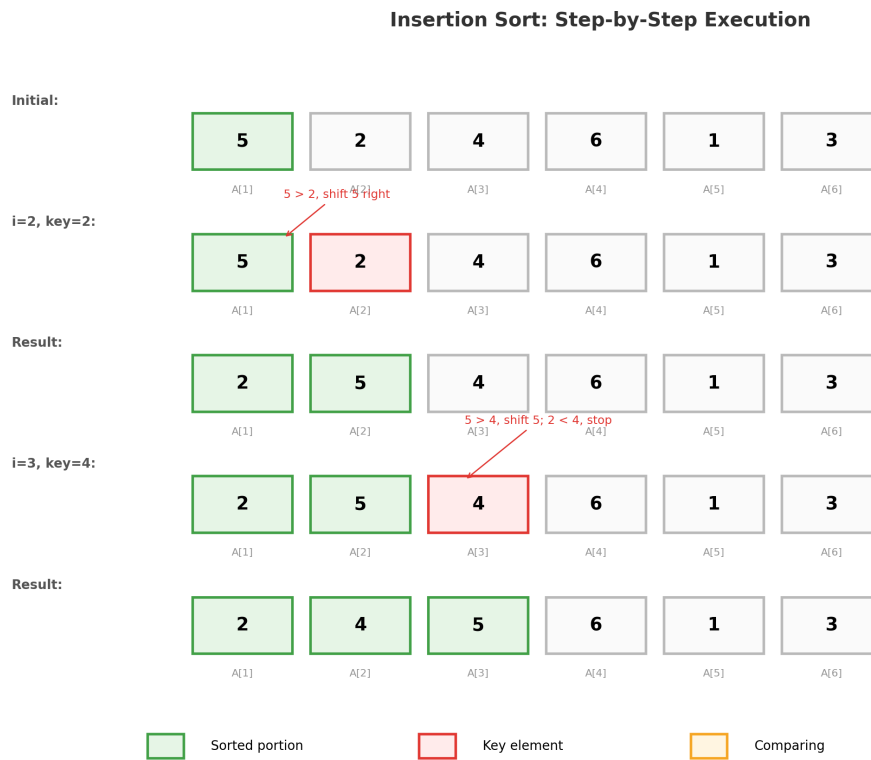


Figure 1: Insertion Sort: Array state at each iteration showing the sorted portion (green), key element being inserted (red), and comparison point (yellow).

1.5 Loop Invariant & Correctness Proof

Loop Invariant (outer for loop)

At the start of each iteration i (where $2 \leq i \leq n + 1$), the subarray $A[1 : i - 1]$ consists of the elements originally in $A[1 : i - 1]$ but in sorted order.

2. Selection Sort

2.1 Algorithm Overview

Type: Comparison-based, selection sorting algorithm.

How it works: Divides the array into sorted and unsorted regions. Repeatedly finds the minimum element from the unsorted region and swaps it with the first element of the unsorted region, expanding the sorted region by one.

Best Cases to Use

- When memory writes are expensive (only $O(n)$ swaps)
- Small datasets where simplicity matters
- When checking all elements is necessary anyway

2.2 Complexity Analysis

SELECTION-SORT(A, n)

1	for $i = 1$ to $n - 1$	$\rightarrow c_1$ (n times)
2	$min_index = i$	
3	for $j = i + 1$ to n	$\rightarrow c_2$ ($n - 1$ times)
4	if $A[j] < A[min_index]$	$\rightarrow c_3$ ($\sum_{i=1}^{n-1} (n - i + 1)$ times)
		$\rightarrow c_4$ ($\sum_{i=1}^{n-1} (n - i)$ times)
5	$min_index = j$	
6	exchange $A[i]$ with $A[min_index]$	$\rightarrow c_5$ ($\sum_{i=1}^{n-1} t_i$ times)
		$\rightarrow c_6$ ($n - 1$ times)

Where t_i = number of times min_index is updated in iteration i .

Worst, Average, and Best Cases

Selection Sort performs the same number of comparisons regardless of input order.

Step-by-Step Derivation:**Step 1:** Evaluate the summation for Line 3:

$$\sum_{i=1}^{n-1} (n - i + 1) = n + (n - 1) + (n - 2) + \dots + 2 = \sum_{k=2}^n k = \frac{n(n+1)}{2} - 1$$

Step 2: Evaluate the summation for Line 4:

$$\sum_{i=1}^{n-1} (n - i) = (n - 1) + (n - 2) + \dots + 1 = \frac{(n-1)n}{2}$$

Step 3: Combine all terms:

$$T(n) = c_1 n + c_2(n - 1) + c_3 \left[\frac{n(n+1)}{2} - 1 \right] + c_4 \left[\frac{n(n-1)}{2} \right] + c_5 \cdot O(n) + c_6$$

Step 4: Expand the n^2 terms:

$$T(n) = \frac{c_3 + c_4}{2} \cdot n^2 + \left(c_1 + c_2 + \frac{c_3}{2} - \frac{c_4}{2} + c_6 \right) n - (c_2 + c_3 + c_6) + c_5 \cdot O(n)$$

Conclusion: The dominant term is n^2 for all cases.

$T(n) = \Theta(n^2)$ for worst, average, and best cases

Note: Even if the array is already sorted, Selection Sort performs all $\frac{n(n-1)}{2}$ comparisons. The number of swaps is $O(n)$ in all cases.

2.3 Properties

Property	Value	Reasoning
Stable	NO	The swap operation can change the relative order of equal elements. Example: $[4_a, 4_b, 3]$ becomes $[3, 4_b, 4_a]$ after first swap. The two 4s have reversed their relative order.
In-place	YES	Uses only $O(1)$ extra space (variables: i, j, min_index). All operations are swaps within the original array.

Adaptive **NO** Running time is $\Theta(n^2)$ regardless of input order. Even if array is already sorted, the algorithm performs all $\frac{n(n-1)}{2}$ comparisons.

2.4 Visual Walkthrough

Sorting $A = [64, 25, 12, 22, 11]$ using Selection Sort:

Iteration	Array State	Action
Initial	[64, 25, 12, 22, 11]	—
$i = 1$	[11 , 25, 12, 22, 64]	Found min = 11 at index 5, swapped with A[1]
$i = 2$	[11, 12 , 25, 22, 64]	Found min = 12 at index 3, swapped with A[2]
$i = 3$	[11, 12, 22 , 25, 64]	Found min = 22 at index 4, swapped with A[3]
$i = 4$	[11, 12, 22, 25 , 64]	Found min = 25 at index 4, swap with itself (no change)
Final	[11, 12, 22, 25, 64]	Sorted

2.5 Loop Invariant & Correctness Proof

Loop Invariant (outer for loop)

At the start of each iteration i (where $1 \leq i \leq n$), the subarray $A[1 : i - 1]$ contains the $i - 1$ smallest elements of the original array in sorted order.

Proof of Correctness:

Initialization: Before first iteration ($i = 1$), $A[1 : 0]$ is empty. Empty subarray trivially contains 0 smallest elements in sorted order. Invariant holds.

Maintenance: Assume invariant holds at start of iteration i ($A[1 : i - 1]$ is sorted). The inner loop finds the minimum in $A[i : n]$. This minimum is the i -th smallest overall element. Swapping it to position i ensures $A[1 : i]$ contains i smallest elements sorted. Incrementing i preserves invariant.

Termination: Loop terminates when $i = n$. By invariant, $A[1 : n - 1]$ contains $n - 1$ smallest elements sorted. The remaining element at $A[n]$ must be the largest. Thus $A[1 : n]$ is sorted. **QED**

3. Merge Sort

3.1 Algorithm Overview

Type: Comparison-based, divide-and-conquer sorting algorithm.

How it works: Recursively divides the array into halves until each subarray has one element, then merges subarrays back together in sorted order.

Best Cases to Use

- Large datasets where $O(n \log n)$ guarantee is needed
- Linked lists (efficient merging without extra space for pointers)
- External sorting (data too large for memory)
- When stability is required with optimal worst-case performance

3.2 Complexity Analysis

MERGE(A, p, q, r)

```

1   $n_L = q - p + 1$   $\rightarrow c_1$ 
2   $n_R = r - q$   $\rightarrow c_2$ 
3  let  $L[0:n_L - 1]$  and  $R[0:n_R - 1]$  be new arrays  $\rightarrow c_3$ 
4  for  $i = 0$  to  $n_L - 1$ 
5       $L[i] = A[p + i]$   $\rightarrow c_4$  ( $n_L + 1$  times)
 $\rightarrow c_5$  ( $n_L$  times)
6  for  $j = 0$  to  $n_R - 1$ 
7       $R[j] = A[q + j + 1]$   $\rightarrow c_6$  ( $n_R + 1$  times)
 $\rightarrow c_7$  ( $n_R$  times)
8   $i = 0; j = 0; k = p$ 
12 while  $i < n_L$  and  $j < n_R$   $\rightarrow c_8, c_9, c_{10}$ 
13     if  $L[i] \leq R[j]$   $\rightarrow c_{11}$  (at most  $n_L + n_R + 1$ )
 $\rightarrow c_{12}$  (at most  $n_L + n_R$ )
14          $A[k] = L[i]; i = i + 1$   $\rightarrow c_{13}, c_{14}$ 
16     else  $A[k] = R[j]; j = j + 1$   $\rightarrow c_{15}, c_{16}$ 
18      $k = k + 1$   $\rightarrow c_{17}$ 
20-27 // Copy remaining elements  $\Theta(n_L + n_R) = \Theta(n)$ 

```

MERGE Time: $\Theta(n)$ where $n = r - p + 1$ (total elements merged).

MERGE-SORT(A, p, r)

```

1  if  $p < r$   $\rightarrow c_1$ 
2       $q = \lfloor (p + r) / 2 \rfloor$   $\rightarrow c_2$ 
4      MERGE-SORT(A, p, q)
5      MERGE-SORT(A, q+1, r)  $\rightarrow T(n/2)$ 
 $\rightarrow T(n/2)$ 
7      MERGE(A, p, q, r)  $\rightarrow \Theta(n)$ 

```

Recurrence Relation

$$T(n) = \begin{cases} \Theta(1) & \text{if } n = 1 \\ 2T(n/2) + \Theta(n) & \text{if } n > 1 \end{cases}$$

Solving by Recursion Tree Method

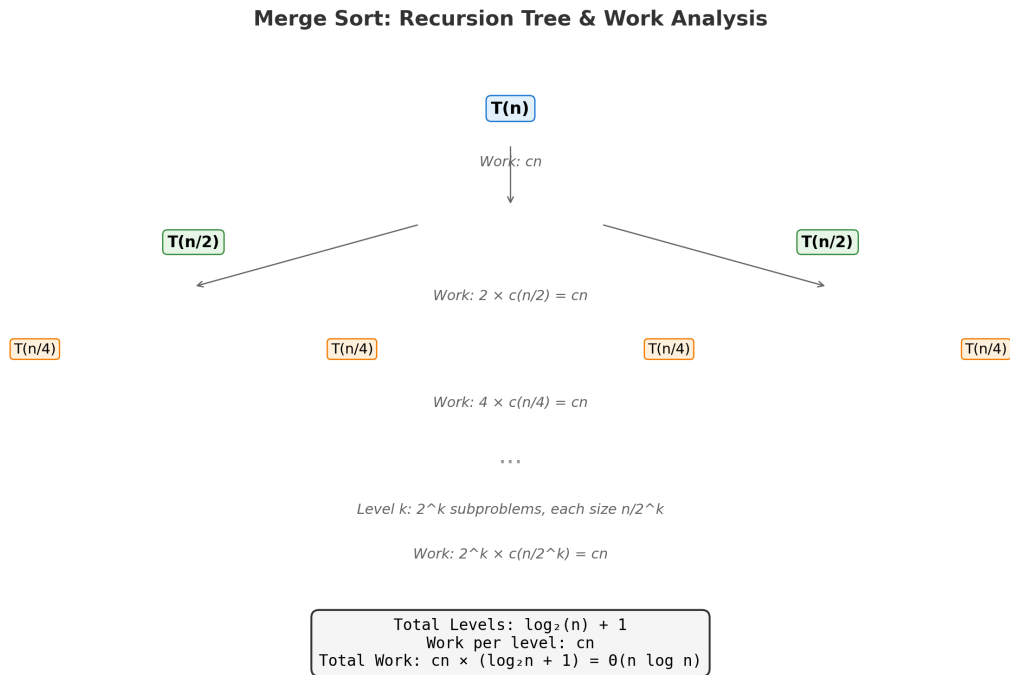


Figure 2: Merge Sort recursion tree showing work at each level. Total work = work per level $\times$ number of levels = $\Theta(n) \times \Theta(\log n) = \Theta(n \log n)$.

Detailed Recursion Tree Analysis:

Level 0 (Root): One problem of size n . Work = cn (the MERGE step).

Level 1: Two problems of size $n/2$. Work = $2 \times c(n/2) = cn$.

Level 2: Four problems of size $n/4$. Work = $4 \times c(n/4) = cn$.

Level k : 2^k problems of size $n/2^k$. Work = $2^k \times c(n/2^k) = cn$.

Number of levels: The recursion stops when $n/2^k = 1$, so $k = \log_2 n$. Total levels = $\log_2 n + 1$.

Total work:

$$T(n) = \sum_{k=0}^{\log_2 n} cn = cn(\log_2 n + 1) = cn \cdot \log_2 n + cn = \Theta(n \log n)$$

Solving by Master Theorem

$$n^{\log_b a} = n^{\log_2 2} = n^1 = n$$

Since $f(n) = \Theta(n) = \Theta(n^{\log_b a})$, **Case 2** of the Master Theorem applies:

$$T(n) = \Theta(n^{\log_b a} \cdot \log n) = \Theta(n \log n)$$

This holds for all cases (worst, average, best) since the division and merge structure is identical regardless of input.

3.3 Properties

Property	Value	Reasoning
Stable	YES	In MERGE, when $L[i] \leq R[j]$, we take from L first. Equal elements from the left subarray (which appeared first in the original array) are placed before elements from the right subarray. The $\leq$ ensures stability.
In-place	NO	MERGE requires auxiliary arrays L and R of total size $n_L + n_R = n$. Extra space requirement is $\Theta(n)$.
Adaptive	NO	Always divides into halves and merges regardless of input order. Running time is $\Theta(n \log n)$ even for sorted arrays.

3.4 Visual Walkthrough

Sorting $A = [38, 27, 43, 3, 9, 82, 10]$ using Merge Sort:

Phase	Operation	Result
Divide	Split $[38, 27, 43, 3, 9, 82, 10]$	$[38, 27, 43]$ and $[3, 9, 82, 10]$
Divide	Further split	$[38]$, $[27, 43]$, $[3, 9]$, $[82, 10]$
Divide	Final split	$[38]$, $[27]$, $[43]$, $[3]$, $[9]$, $[82]$, $[10]$
Merge	Merge $[27]$ and $[43]$	$[27, 43]$
Merge	Merge $[38]$ and $[27, 43]$	$[27, 38, 43]$
Merge	Merge $[3]$ and $[9]$	$[3, 9]$
Merge	Merge $[82]$ and $[10]$	$[10, 82]$

Merge	Merge [3,9] and [10,82]	[3, 9, 10, 82]
Merge	Final merge [27,38,43] and [3,9,10,82]	[3, 9, 10, 27, 38, 43, 82]

3.5 Loop Invariant & Correctness Proof

Loop Invariant for MERGE

At the start of each iteration of the main **while** loop, $A[p : k - 1]$ contains the $k - p$ smallest elements of $L[0 : n_L - 1]$ and $R[0 : n_R - 1]$ in sorted order. $L[i]$ and $R[j]$ are the smallest elements of their arrays not yet copied to A .

Proof of MERGE-SORT Correctness (by induction):

Base case: $n = 1$. Array with one element is trivially sorted.

Inductive step: Assume MERGE-SORT works for all sizes $< n$. For size n , it sorts two halves of size $n/2$ correctly (by Inductive Hypothesis), then MERGE correctly combines them. Since MERGE preserves sorted order when combining two sorted arrays, the result is sorted.

Conclusion: MERGE-SORT is correct. **QED**

4. Bubble Sort

4.1 Algorithm Overview

Type: Comparison-based, exchange sorting algorithm.

How it works: Repeatedly steps through the array, compares adjacent elements, and swaps them if they are in the wrong order. Each pass "bubbles" the largest unsorted element to its correct position at the end.

Best Cases to Use

- Educational purposes (simple to understand)
- Nearly sorted arrays with optimization (adaptive variant)
- Small datasets where code simplicity matters

4.2 Complexity Analysis

BUBBLESORT(A, n)

1	for $i = 1$ to $n - 1$	$\rightarrow c_1$ (n times)
2	for $j = n$ downto $i + 1$	
3	if $A[j] < A[j - 1]$	$\rightarrow c_2$ ($\sum_{i=1}^{n-1} (n - i + 1)$ times)
		$\rightarrow c_3$ ($\sum_{i=1}^{n-1} (n - i)$ times)
4	exchange $A[j]$ with $A[j - 1]$	
		$\rightarrow c_4$ ($\sum_{i=1}^{n-1} t_i$ times)

Where t_i = number of swaps in iteration i .

Worst Case (Reverse Sorted)

Every comparison results in a swap. $t_i = n - i$ for all i .

Step-by-Step Derivation:**Step 1:** Evaluate the summation for Line 2:

$$\sum_{i=1}^{n-1} (n - i + 1) = n + (n - 1) + \dots + 2 = \frac{n(n + 1)}{2} - 1$$

Step 2: Evaluate the summation for Line 3 (comparisons):

$$\sum_{i=1}^{n-1} (n - i) = (n - 1) + (n - 2) + \dots + 1 = \frac{n(n - 1)}{2}$$

Step 3: Evaluate the summation for Line 4 (swaps in worst case):

$$\sum_{i=1}^{n-1} (n - i) = \frac{n(n - 1)}{2}$$

Step 4: Combine all terms:

$$T(n) = c_1 \cdot n + c_2 \left[\frac{n(n + 1)}{2} - 1 \right] + c_3 \left[\frac{n(n - 1)}{2} \right] + c_4 \left[\frac{n(n - 1)}{2} \right]$$

Step 5: Expand and collect by powers of n :

$$T(n) = \frac{c_2 + c_3 + c_4}{2} \cdot n^2 + \left(c_1 + \frac{c_2}{2} - \frac{c_3}{2} - \frac{c_4}{2} \right) n - c_2$$

Conclusion: The dominant term is n^2 .

$$T(n) = \Theta(n^2)$$

Average CaseAbout half the comparisons result in swaps, but the number of comparisons remains $\Theta(n^2)$.

$$T(n) = \Theta(n^2)$$

Best Case (Already Sorted)

No swaps needed, but all comparisons are still performed (without optimization flag).

$$T(n) = c_1 \cdot n + c_2 \left[\frac{n(n+1)}{2} - 1 \right] + c_3 \left[\frac{n(n-1)}{2} \right] = \Theta(n^2)$$

Optimization

With a flag to detect if any swap occurred in a pass, Bubble Sort can achieve $\Theta(n)$ best case by terminating early when the array is already sorted.

4.3 Properties

Property	Value	Reasoning
Stable	YES	We only swap when $A[j] < A[j-1]$ (strictly less). When equal elements are adjacent, no swap occurs. The relative order of equal elements is preserved.
In-place	YES	Only uses $O(1)$ extra space (variables: i , j , temp for swap). All operations are swaps within the original array.
Adaptive	NO	Basic version performs all $\frac{n(n-1)}{2}$ comparisons regardless of input. However, with an optimization flag that stops early if no swaps occur in a pass, it becomes adaptive with best case $\Theta(n)$ for sorted arrays.

4.4 Visual Walkthrough

Sorting $A = [5, 1, 4, 2, 8]$ using Bubble Sort:

Pass	Comparisons	Result After Pass
$i = 1$	(5,1)→swap, (5,4)→swap, (5,2)→swap, (5,8)→no swap	[1, 4, 2, 5, 8]
$i = 2$	(1,4)→no swap, (4,2)→swap, (4,5)→no swap	[1, 2, 4, 5, 8]
$i = 3$	(1,2)→no swap, (2,4)→no swap	[1, 2, 4 , 5, 8]
$i = 4$	(1,2)→no swap	[1, 2, 4, 5, 8] (sorted)

4.5 Loop Invariant & Correctness Proof

Loop Invariant (outer for loop)

At the start of each iteration i (where $1 \leq i \leq n$), the subarray $A[n - i + 2 : n]$ contains the $i - 1$ largest elements of the array in sorted order.

Proof of Correctness:

Initialization: Before first iteration ($i = 1$), $A[n + 1 : n]$ is empty. Empty subarray trivially satisfies the invariant.

Maintenance: During iteration i , the inner loop bubbles the largest element among $A[1 : n - i + 1]$ to position $n - i + 1$. This element is at least as large as all elements in $A[n - i + 2 : n]$. Thus $A[n - i + 1 : n]$ contains i largest elements sorted.

Termination: When $i = n$, $A[2 : n]$ contains $n - 1$ largest elements sorted. $A[1]$ must be the smallest element. Thus entire array is sorted. **QED**

5. Heapsort

5.1 Algorithm Overview

Type: Comparison-based, selection sorting using heap data structure.

How it works: First builds a max-heap from the array. Repeatedly extracts the maximum element (root) and places it at the end of the array, then restores the heap property on the remaining elements.

Best Cases to Use

- When guaranteed $O(n \log n)$ worst-case is needed without extra space
- Priority queue implementations
- Systems with limited memory (in-place)
- When finding k largest/smallest elements

5.2 Complexity Analysis

HEAPSORT(A, n)

1	BUILD-MAX-HEAP(A, n)	
		$\rightarrow \Theta(n)$
2	for $i = n$ downto 2	
3	exchange $A[1]$ with $A[i]$	$\rightarrow c_1 (n - 1 \text{ times})$
		$\rightarrow c_2 (n - 1 \text{ times})$
4	$A.heap\text{-}size = A.heap\text{-}size - 1$	
5	MAX-HEAPIFY(A, 1)	$\rightarrow c_3 (n - 1 \text{ times})$
		$\rightarrow O(\log n) \text{ each, } (n - 1) \text{ times}$

BUILD-MAX-HEAP Analysis

BUILD-MAX-HEAP calls MAX-HEAPIFY on $\lfloor n/2 \rfloor$ nodes, starting from the last non-leaf node and moving upward.

Why BUILD-MAX-HEAP is $\Theta(n)$, not $\Theta(n \log n)$:

At first glance, $n/2$ calls to MAX-HEAPIFY each taking $O(\log n)$ seems like $O(n \log n)$. However, most nodes are at the bottom of the tree where MAX-HEAPIFY is very fast.

Key insight: A node at height h takes $O(h)$ time. The number of nodes at height h is at most $\lceil n/2^{h+1} \rceil$.

$$T(n) = \sum_{h=0}^{\lfloor \log n \rfloor} \left\lceil \frac{n}{2^{h+1}} \right\rceil \cdot O(h) = O\left(n \sum_{h=0}^{\infty} \frac{h}{2^h}\right)$$

Using the identity $\sum_{h=0}^{\infty} \frac{h}{2^h} = 2$:

$$T(n) = O(n \cdot 2) = \Theta(n)$$

MAX-HEAPIFY Analysis**MAX-HEAPIFY(A, i)**

Compare $A[i]$ with left and right children
 If violation, swap with larger child and recurse

Time: $O(h)$ where h is the height of the subtree. Since heap height is $\lfloor \log_2 n \rfloor$:

$$T_{\text{heapify}}(n) = O(\log n)$$

Recurrence for MAX-HEAPIFY

In the worst case, the affected subtree can be at most $2n/3$ of the parent (when the last level is exactly half full):

$$T(n) \leq T(2n/3) + \Theta(1)$$

By the Master Theorem: $a = 1$, $b = 3/2$, $f(n) = \Theta(1)$, so $n^{\log_{3/2} 1} = n^0 = 1$. Since $f(n) = \Theta(1) = \Theta(n^{\log_{3/2} 1})$, Case 2 applies:

$$T(n) = \Theta(\log n)$$

Total HEAPSORT Time**Step-by-Step Derivation:**

$$T(n) = T_{\text{build}}(n) + T_{\text{extract}}(n)$$

$$T(n) = \Theta(n) + \sum_{i=2}^n O(\log i)$$

Since $\sum_{i=2}^n \log i = \log(n!) = \Theta(n \log n)$ by Stirling's approximation:

$$T(n) = \Theta(n) + \Theta(n \log n) = \boxed{\Theta(n \log n)}$$

This holds for all cases (worst, average, best) because the heap structure and extraction process are identical regardless of input order.

5.3 Properties

Property	Value	Reasoning
Stable	NO	The swap of $A[1]$ with $A[i]$ can change relative order of equal elements. When the root (max element) is swapped to the end, it may jump over elements equal to it.
In-place	YES	Uses only $O(1)$ extra space (variables for swapping and heap operations). The heap is built within the original array.
Adaptive	NO	Always builds a max-heap and performs $n - 1$ extractions. Running time is $\Theta(n \log n)$ regardless of input order.

5.4 Visual Walkthrough

Sorting $A = [4, 10, 3, 5, 1]$ using Heapsort:

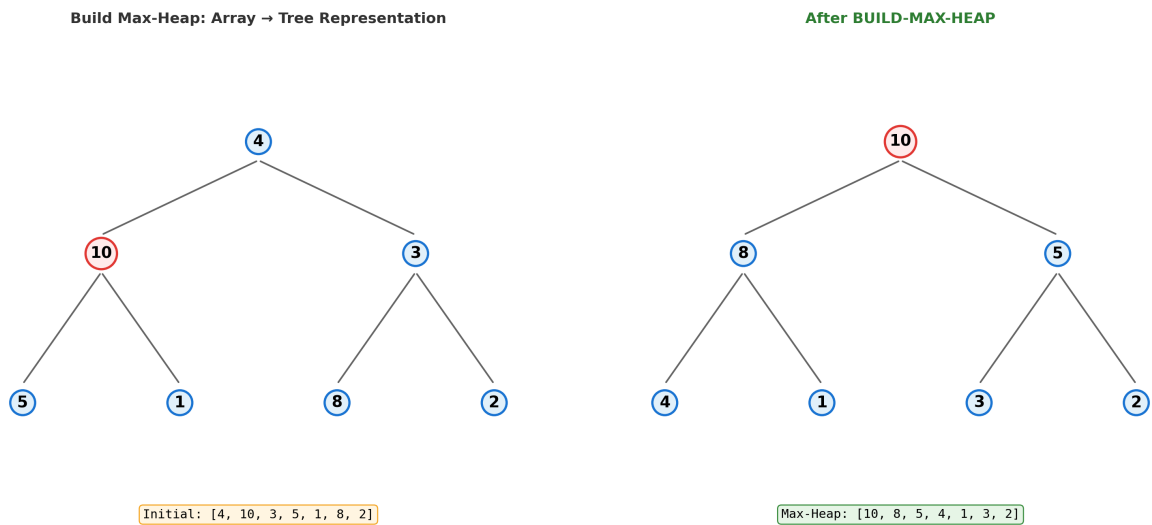


Figure 3: Heapsort: Transformation of array into max-heap. Left: Initial array as complete binary tree. Right: After BUILD-MAX-HEAP, the maximum element (10) is at the root.

Step	Heap Array	Action	Sorted Portion
After Build	[10, 8, 5, 4, 1, 3, 2]	Max-heap built	[]
$i = 7$	[<u>2</u> , 8, 5, 4, 1, 3, 10]	Swap root(10) with $A[7]$	[10]
Heapify	[8, 4, 5, 2, 1, 3]	Restore heap on 6 elements	[10]

$i = 6$	[<u>3</u> , 4, 5, 2, 1, 8]	Swap root(8) with A[6]	[8, 10]
Heapify	[5, 4, 3, 2, 1]	Restore heap on 5 elements	[8, 10]
$i = 5$	[<u>1</u> , 4, 3, 2, 5]	Swap root(5) with A[5]	[5, 8, 10]
...continues until sorted: [1, 2, 3, 4, 5, 8, 10]			

5.5 Loop Invariant & Correctness Proof

Loop Invariant (for HEAPSORT for loop)

At the start of each iteration i (where $2 \leq i \leq n + 1$), the subarray $A[1 : i - 1]$ is a max-heap containing the $i - 1$ smallest elements, and $A[i : n]$ contains the $n - i + 1$ largest elements of the original array in sorted order.

Proof of Correctness:

Initialization: Before first iteration ($i = n$), BUILD-MAX-HEAP ensures $A[1 : n]$ is a max-heap. $A[n + 1 : n]$ is empty. Invariant holds.

Maintenance: At start of iteration i , $A[1 : i]$ is a max-heap. $A[1]$ is the maximum of $A[1 : i]$. Swapping it to $A[i]$ places the $(n - i + 1)$ -th largest element in its final position. Reducing heap-size and calling MAX-HEAPIFY restores the heap property for $A[1 : i - 1]$.

Termination: When $i = 1$, $A[2 : n]$ contains $n - 1$ largest elements sorted. $A[1]$ is the smallest element. Thus $A[1 : n]$ is sorted. **QED**

6. Quicksort

6.1 Algorithm Overview

Type: Comparison-based, divide-and-conquer sorting algorithm.

How it works: Selects a "pivot" element and partitions the array such that elements smaller than pivot are on the left and larger elements are on the right. Recursively sorts the subarrays.

Best Cases to Use

- General-purpose sorting (fastest in practice for most inputs)
- When average-case performance matters more than worst-case
- Cache-friendly applications (good locality of reference)
- When in-place sorting with $O(\log n)$ stack space is acceptable

6.2 Complexity Analysis**PARTITION(A, p, r)**

```

1   $x = A[r]$   $\rightarrow c_1$ 
2   $i = p - 1$   $\rightarrow c_2$ 
3  for  $j = p$  to  $r - 1$ 
4      if  $A[j] \leq x$   $\rightarrow c_3$  ( $n$  times,  $n = r - p + 1$ )
                         $\rightarrow c_4$  ( $n - 1$  times)
5           $i = i + 1$ 
6          exchange  $A[i]$  with  $A[j]$   $\rightarrow c_5$  ( $t$  times)
7  exchange  $A[i + 1]$  with  $A[r]$   $\rightarrow c_6$  ( $t$  times)
                         $\rightarrow c_7$ 
8  return  $i + 1$   $\rightarrow c_8$ 

```

PARTITION Time: $\Theta(n)$ where $n = r - p + 1$.

QUICKSORT(A, p, r)

```

1  if  $p < r$   $\rightarrow c_1$ 
3       $q = \text{PARTITION}(A, p, r)$ 
4       $\text{QUICKSORT}(A, p, q-1)$   $\rightarrow \Theta(n)$ 
                         $\rightarrow T(k)$ 
5       $\text{QUICKSORT}(A, q+1, r)$   $\rightarrow T(n-k-1)$ 

```

Where $k = q - p$ (number of elements in left partition).

Worst Case (Unbalanced Partitions)

Occurs when the pivot is always the smallest or largest element, creating partitions of size $n - 1$ and 0 .

Step-by-Step Derivation:

Step 1: Set up the recurrence:

$$T(n) = T(n - 1) + T(0) + \Theta(n) = T(n - 1) + \Theta(n)$$

Step 2: Unfold the recurrence:

$$T(n) = T(n - 1) + cn$$

$$T(n - 1) = T(n - 2) + c(n - 1)$$

$$T(n - 2) = T(n - 3) + c(n - 2)$$

$$\vdots$$

$$T(2) = T(1) + c(2)$$

Step 3: Substitute back (telescoping):

$$T(n) = T(1) + c(2 + 3 + 4 + \dots + n)$$

$$T(n) = T(1) + c \sum_{i=2}^n i = T(1) + c \left(\frac{n(n+1)}{2} - 1 \right)$$

Step 4: Since $T(1) = \Theta(1)$:

$$T(n) = \Theta(1) + c \left(\frac{n^2 + n - 2}{2} \right) = \Theta(n^2)$$

$$\boxed{T(n) = \Theta(n^2)}$$

Best Case (Balanced Partitions)

Partitions are roughly equal: $k \approx n/2$.

Step-by-Step Derivation:**Step 1:** Set up the recurrence:

$$T(n) = 2T(n/2) + \Theta(n)$$

Step 2: This is the same recurrence as Merge Sort!**Step 3:** Using the Master Theorem: $a = 2, b = 2, f(n) = \Theta(n)$:

$$n^{\log_2 2} = n = f(n) \Rightarrow \text{Case 2 applies}$$

$$T(n) = \Theta(n \log n)$$

Average CaseAssume all splits are equally likely (any partition size k from 0 to $n - 1$ is equally probable):**Expected Time Analysis:**

$$E[T(n)] = \frac{1}{n} \sum_{k=0}^{n-1} (E[T(k)] + E[T(n - k - 1)]) + \Theta(n)$$

By symmetry, $E[T(k)] = E[T(n - k - 1)]$:

$$E[T(n)] = \frac{2}{n} \sum_{k=0}^{n-1} E[T(k)] + \Theta(n)$$

Solving via the substitution method (assume $E[T(n)] \leq an \log n + bn$):

$$E[T(n)] \leq \frac{2}{n} \sum_{k=0}^{n-1} (ak \log k + bk) + \Theta(n)$$

Using $\sum_{k=1}^{n-1} k \log k \leq \frac{n^2 \log n}{2} - \frac{n^2}{4}$ for $n \geq 2$:

$$E[T(n)] \leq \frac{2a}{n} \left(\frac{n^2 \log n}{2} - \frac{n^2}{4} \right) + O(n) = an \log n - \frac{an}{2} + O(n)$$

For sufficiently large a , the negative term dominates the $O(n)$:

$$E[T(n)] = O(n \log n)$$

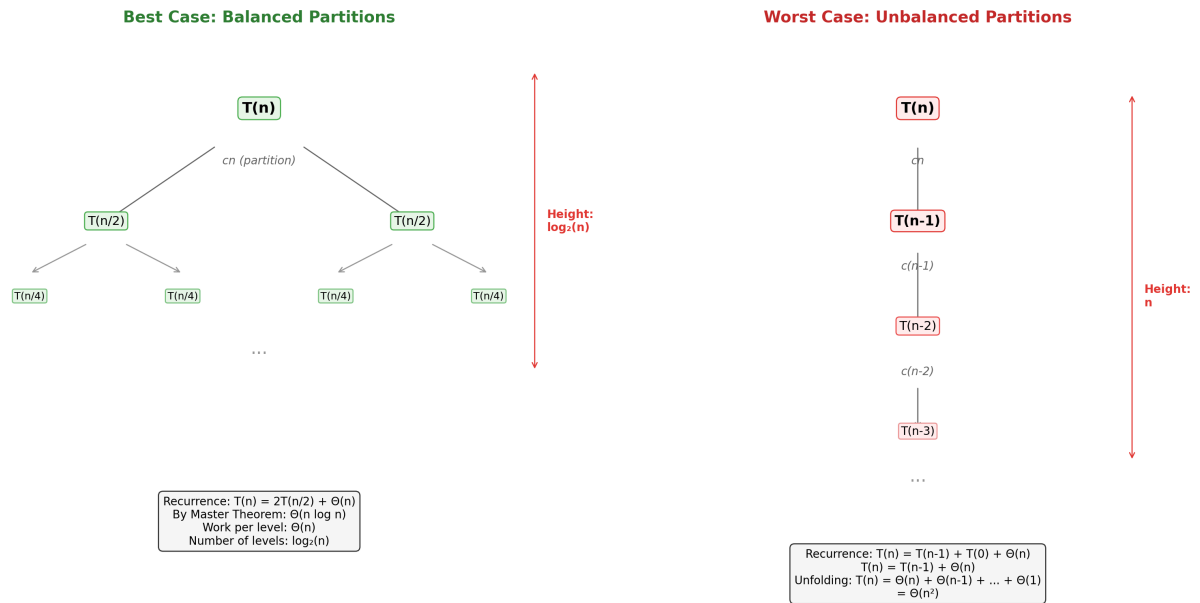


Figure 4: Quicksort recursion trees: Best case (balanced, left) produces a shallow tree with $\Theta(n \log n)$ height. Worst case (unbalanced, right) produces a degenerate tree with $\Theta(n^2)$ height.

6.3 Properties

Property	Value	Reasoning
Stable	NO	The exchange operations in PARTITION can change relative order of equal elements. When $A[j] \leq x$, we swap elements that may jump over equal elements.
In-place	YES	PARTITION rearranges elements within the array using only $O(1)$ extra space. Recursion uses $O(\log n)$ stack space in best/average case, $O(n)$ in worst case.
Adaptive	NO	Running time depends on pivot selection and partition balance, not on input order. Even sorted arrays can cause worst-case behavior with poor pivot choice (e.g., always choosing last element).

6.4 Visual Walkthrough

Sorting $A = [10, 80, 30, 90, 40, 50, 70]$ using Quicksort (Lomuto partition, pivot = last element):

Step	Subarray	Pivot	Partition Result
Initial	[10, 80, 30, 90, 40, 50, 70]	70	[10, 30, 40, 50, 70 , 80, 90]

Left	[10, 30, 40, 50]	50	[10, 30, 40, 50]
Left-Left	[10, 30, 40]	40	[10, 30, 40]
Right	[80, 90]	90	[80, 90]
Final	[10, 30, 40, 50, 70, 80, 90]		

6.5 Loop Invariant & Correctness Proof

Loop Invariant for PARTITION (for loop)

At the start of each iteration j (where $p \leq j \leq r$):

1. $A[p : i]$ contains elements $\leq x$ (pivot)
2. $A[i + 1 : j - 1]$ contains elements $> x$
3. $A[j : r - 1]$ contains elements not yet processed
4. $A[r] = x$ (pivot)

Proof of QUICKSORT Correctness (by induction on subarray size):

Base case: Subarray of 0 or 1 element is trivially sorted.

Inductive step: Assume QUICKSORT works for all subarrays of size $< n$. For a subarray of size n , PARTITION correctly places the pivot at position q such that all elements in $A[p : q - 1]$ are $\leq A[q]$ and all elements in $A[q + 1 : r]$ are $> A[q]$. By the Inductive Hypothesis, the recursive calls correctly sort both partitions. Since pivot is between two sorted partitions with all left elements $\leq$ pivot and all right elements $>$ pivot, the entire subarray is sorted.

Conclusion: QUICKSORT is correct. **QED**

7. Randomized Quicksort

7.1 Algorithm Overview

Type: Comparison-based, randomized divide-and-conquer sorting algorithm.

How it works: Same as Quicksort, but before partitioning, randomly selects an element and swaps it with the last element to use as pivot. This randomization prevents worst-case behavior on specific input patterns (e.g., already sorted arrays).

Best Cases to Use

- When input may be adversarial or nearly sorted
- When worst-case $O(n^2)$ must be avoided with high probability
- General-purpose sorting where average-case guarantee is preferred

7.2 Complexity Analysis

RANDOMIZED-PARTITION(A, p, r)

```

1  i = RANDOM(p, r)
2  exchange A[r] with A[i]
3  return PARTITION(A, p, r)
```

$\rightarrow c_1$
 $\rightarrow c_2$
 $\rightarrow \Theta(n)$

RANDOMIZED-QUICKSORT(A, p, r)

```

1  if p < r
2      q = RANDOMIZED-PARTITION(A, p, r)
3      RANDOMIZED-QUICKSORT(A, p, q-1)
4      RANDOMIZED-QUICKSORT(A, q+1, r)
```

$\rightarrow c_1$
 $\rightarrow \Theta(n)$
 $\rightarrow T(k)$
 $\rightarrow T(n - k - 1)$

Expected Time Analysis

Step-by-Step Derivation of Expected Running Time:

Step 1: The expected running time $E[T(n)]$ satisfies:

$$E[T(n)] = \frac{1}{n} \sum_{k=0}^{n-1} (E[T(k)] + E[T(n - k - 1)]) + \Theta(n)$$

Each possible split k (where the pivot ends up at position $k + 1$) occurs with probability $1/n$.

Step 2: By symmetry ($E[T(k)] = E[T(n - k - 1)]$):

$$E[T(n)] = \frac{2}{n} \sum_{k=0}^{n-1} E[T(k)] + \Theta(n)$$

Step 3: Multiply both sides by n :

$$n \cdot E[T(n)] = 2 \sum_{k=0}^{n-1} E[T(k)] + \Theta(n^2)$$

Step 4: Write the same equation for $n - 1$:

$$(n - 1) \cdot E[T(n - 1)] = 2 \sum_{k=0}^{n-2} E[T(k)] + \Theta((n - 1)^2)$$

Step 5: Subtract the two equations:

$$n \cdot E[T(n)] - (n - 1) \cdot E[T(n - 1)] = 2E[T(n - 1)] + \Theta(n)$$

$$n \cdot E[T(n)] = (n + 1) \cdot E[T(n - 1)] + \Theta(n)$$

Step 6: Divide by $n(n + 1)$:

$$\frac{E[T(n)]}{n + 1} = \frac{E[T(n - 1)]}{n} + \frac{\Theta(n)}{n(n + 1)} = \frac{E[T(n - 1)]}{n} + \Theta\left(\frac{1}{n}\right)$$

Step 7: Let $S(n) = \frac{E[T(n)]}{n+1}$. Then $S(n) = S(n - 1) + \Theta(1/n)$:

$$S(n) = \sum_{i=1}^n \Theta\left(\frac{1}{i}\right) = \Theta(H_n) = \Theta(\log n)$$

where H_n is the n -th harmonic number.

Step 8: Substitute back:

$$\frac{E[T(n)]}{n + 1} = \Theta(\log n)$$

$$\boxed{E[T(n)] = \Theta(n \log n)}$$

Worst Case: Still $\Theta(n^2)$, but occurs with very low probability. The probability of consistently picking the worst pivot is $1/n!$ for any specific adversarial input.

7.3 Properties

Property	Value	Reasoning
Stable	NO	Same as Quicksort. The random swap and subsequent PARTITION exchanges can change relative order of equal elements.
In-place	YES	Same as Quicksort. Only $O(1)$ extra space for swapping plus $O(\log n)$ expected stack space for recursion.
Adaptive	NO	Randomization makes running time independent of input order in expectation. No performance improvement for partially sorted input.

7.4 Visual Walkthrough

Sorting $A = [3, 6, 2, 8, 1, 4, 7]$ using Randomized Quicksort:

Call	Subarray	Random Index	Pivot	Partition Result
RQ(A, 1, 7)	[3, 6, 2, 8, 1, 4, 7]	3	2	[2, 6, 3, 8, 1, 4, 7] $\rightarrow$ q=2
RQ(A, 1, 1)	[2]	—	—	Base case
RQ(A, 3, 7)	[6, 3, 8, 1, 4, 7]	5	1	[1, 3, 8, 6, 4, 7] $\rightarrow$ q=3
RQ(A, 4, 7)	[3, 8, 6, 4, 7]	6	7	[3, 6, 4, 7, 8] $\rightarrow$ q=7
...continues recursively until sorted: [1, 2, 3, 4, 6, 7, 8]				

Correctness of Randomized Quicksort:

RANDOMIZED-PARTITION produces a random pivot, but once chosen, PARTITION behaves deterministically. The random choice does not affect the correctness of the partitioning. By the same inductive argument as Quicksort, RANDOMIZED-QUICKSORT correctly sorts any input. The randomization only affects the probability distribution of running times, not correctness. **QED**

8. Counting Sort

8.1 Algorithm Overview

Type: Non-comparison-based, linear time sorting algorithm.

How it works: Counts the number of occurrences of each distinct element, computes cumulative counts to determine positions, then places elements in output array based on these positions.

Best Cases to Use

- When range of input $k = O(n)$
- Integer sorting with small range
- As subroutine in Radix Sort
- When stability is required with linear time

8.2 Complexity Analysis

COUNTING-SORT(A, n, k)

1	let $B[1 : n]$ and $C[0 : k]$ be new arrays	
2	for $i = 0$ to k	$\rightarrow c_1$
		$\rightarrow c_2$ ($k + 1$ times)
3	$C[i] = 0$	
		$\rightarrow c_3$ ($k + 1$ times)
4	for $j = 1$ to n	
		$\rightarrow c_4$ ($n + 1$ times)
5	$C[A[j]] = C[A[j]] + 1$	
		$\rightarrow c_5$ (n times)
7	for $i = 1$ to k	
		$\rightarrow c_6$ ($k + 1$ times)
8	$C[i] = C[i] + C[i - 1]$	
		$\rightarrow c_7$ (k times)
11	for $j = n$ down to 1	
12	$B[C[A[j]]] = A[j]$	$\rightarrow c_8$ ($n + 1$ times)
		$\rightarrow c_9$ (n times)
13	$C[A[j]] = C[A[j]] - 1$	
		$\rightarrow c_{10}$ (n times)
14	return B	
		$\rightarrow c_{11}$

Time Complexity Derivation

Step-by-Step Derivation:

Step 1: Line 2-3 (initialize count array): $\Theta(k)$

Step 2: Line 4-5 (count occurrences): $\Theta(n)$

Step 3: Line 7-8 (cumulative counts): $\Theta(k)$

Step 4: Line 11-13 (place elements): $\Theta(n)$

Step 5: Total:

$$T(n, k) = \Theta(k) + \Theta(n) + \Theta(k) + \Theta(n) = \Theta(n + k)$$

Step 6: When $k = O(n)$:

$$T(n) = \Theta(n)$$

Space Complexity

Requires two auxiliary arrays: B (output, size n) and C (counts, size $k + 1$):

$$S(n, k) = \Theta(n + k)$$

8.3 Properties

Property	Value	Reasoning
Stable	YES	The algorithm processes A from right to left (line 11: $j = n$ downto 1) and places elements in B using cumulative counts. Elements with equal values appear in B in the same relative order as in A because later occurrences are placed first (due to decrementing $C[A[j]]$).
In-place	NO	Requires two auxiliary arrays: B (output, size n) and C (counts, size $k + 1$). Extra space is $\Theta(n + k)$.
Adaptive	NO	Always performs the same operations regardless of input order. The counting, cumulative sum, and placement steps are executed identically for any input of size n with range k .

8.4 Visual Walkthrough

Sorting $A = [4, 2, 2, 8, 3, 3, 1]$ where $n = 7, k = 8$:

Step	Array C	Description
Initialize	[0, 0, 0, 0, 0, 0, 0, 0]	Indices 0 to 8, all zeros
After counting	[0, 1, 2, 2, 1, 0, 0, 0, 1]	$C[i]$ = count of element i
Cumulative	[0, 1, 3, 5, 6, 6, 6, 6, 7]	$C[i]$ = elements $\leq i$

j	$A[j]$	$C[A[j]]$	Place at $B[.]$	After Decrement	B Array
7	1	1	$B[1] = 1$	$C[1] = 0$	[1, _, _, _, _, _, _]
6	3	5	$B[5] = 3$	$C[3] = 4$	[1, _, _, 3, _, _]
5	3	4	$B[4] = 3$	$C[3] = 3$	[1, _, 3, 3, _, _]
4	8	7	$B[7] = 8$	$C[8] = 6$	[1, _, 3, 3, _, 8]
3	2	3	$B[3] = 2$	$C[2] = 2$	[1, _, 2, 3, 3, _, 8]
2	2	2	$B[2] = 2$	$C[2] = 1$	[1, 2, 2, 3, 3, _, 8]
1	4	6	$B[6] = 4$	$C[4] = 5$	[1, 2, 2, 3, 3, 4, 8]

Loop Invariant for COUNTING-SORT (final for loop)

At the start of each iteration j , B contains the correctly positioned elements from $A[j + 1 : n]$, and C contains the correct remaining counts for placing elements from $A[1 : j]$.

Proof of Correctness:

Initialization: Before first iteration ($j = n$), B is empty. C contains cumulative counts of all elements. Invariant holds vacuously.

Maintenance: At iteration j , $C[A[j]]$ contains the correct final position for $A[j]$ (number of elements $\leq A[j]$ that haven't been placed yet). Placing $A[j]$ at $B[C[A[j]]]$ and decrementing $C[A[j]]$ ensures the next occurrence of the same value goes to the position immediately before. This maintains stability and correct positioning.

Termination: When $j = 0$, all elements from $A[1 : n]$ have been placed in B . By the invariant, all are in correct sorted positions. **QED**

9. Minimum Element

9.1 Algorithm Overview

Type: Selection algorithm, linear search.

How it works: Initializes `min` with the first element, then scans through remaining elements updating `min` whenever a smaller element is found.

Best Cases to Use

- Finding minimum in an unsorted array
- When only minimum is needed (not full sort)
- Any scenario requiring a single linear scan

9.2 Complexity Analysis

MINIMUM(A, n)

1	<i>min</i> = A[1]	→ c_1
2	for <i>i</i> = 2 to <i>n</i>	
		→ c_2 (n times)
3	if <i>min</i> > A[<i>i</i>]	
		→ c_3 ($n - 1$ times)
4	<i>min</i> = A[<i>i</i>]	
		→ c_4 (t times)
5	return <i>min</i>	
		→ c_5

Where t = number of times a new minimum is found ($0 \leq t \leq n - 1$).

Worst Case (Decreasing Order)

New minimum found at every comparison: $t = n - 1$.

Step-by-Step Derivation:

$$T(n) = c_1 + c_2 \cdot n + c_3(n - 1) + c_4(n - 1) + c_5$$

$$T(n) = c_1 + c_2n + c_3n - c_3 + c_4n - c_4 + c_5$$

$$T(n) = (c_2 + c_3 + c_4) \cdot n + (c_1 - c_3 - c_4 + c_5)$$

$$\boxed{T(n) = \Theta(n)}$$

Average Case

Expected number of updates is the $(n - 1)$ -th harmonic number $H_{n-1} \approx \ln(n)$:

$$T(n) = c_1 + c_2n + c_3(n - 1) + c_4 \cdot H_{n-1} + c_5 = \Theta(n)$$

Best Case (Increasing Order)

No updates needed: $t = 0$.

$$T(n) = c_1 + c_2n + c_3(n - 1) + c_5 = \Theta(n)$$

9.3 Visual Walkthrough

Finding minimum in $A = [45, 12, 78, 23, 67, 8, 34]$:

i	$A[i]$	Comparison	min	Updated?
1	45	—	45	Initial
2	12	$45 > 12$	12	Yes
3	78	$12 > 78$	12	No
4	23	$12 > 23$	12	No
5	67	$12 > 67$	12	No
6	8	$12 > 8$	8	Yes
7	34	$8 > 34$	8	No
Return min = 8				

Loop Invariant

At the start of each iteration i (where $2 \leq i \leq n + 1$), min contains the minimum value among $A[1 : i - 1]$.

Proof of Correctness:

Initialization: Before first iteration ($i = 2$), $min = A[1]$. The subarray $A[1 : 1]$ contains one element, which is trivially the minimum.

Maintenance: Assume min is the minimum of $A[1 : i - 1]$. At iteration i , compare min with $A[i]$. If $A[i] < min$, then $A[i]$ is the new minimum of $A[1 : i]$. Otherwise, min remains the minimum of $A[1 : i]$.

Termination: When $i = n + 1$, min is the minimum of $A[1 : n]$, the entire array. **QED**

10. Simultaneous Min-Max

10.1 Algorithm Overview

Type: Selection algorithm, optimized pairwise comparison.

How it works: Processes elements in pairs, comparing each pair first, then comparing the smaller with current `min` and the larger with current `max`. Reduces total comparisons from $2n - 2$ to approximately $3n/2$.

Best Cases to Use

- When both minimum and maximum are needed simultaneously
- Tournament structures
- Range queries on unsorted data

10.2 Complexity Analysis

SIMULTANEOUS-MIN-MAX(A, n)

1-12	Initialize min, max, and index i	$\Theta(1)$
13	while $i \leq n - 1$	$\rightarrow c_{11} \text{ (} n/2 \text{ times)}$
14	if $A[i] < A[i + 1]$	$\rightarrow c_{12} \text{ (} n/2 - 1 \text{ times)}$
15-18	Compare smaller with min, larger with max	$\rightarrow 2 \text{ comparisons}$
19-23	else (reverse comparison)	$\rightarrow 2 \text{ comparisons}$
24	$i = i + 2$	$\rightarrow c_{21}$

Comparison Count Analysis

Step-by-Step Derivation:

Case 1: n is odd

- Initial setup: 0 comparisons (set $\min = \max = A[1]$)
- Number of pairs in loop: $(n - 1)/2$
- Each pair: 1 comparison (pair vs pair) + up to 2 comparisons (vs min and max) = 3 comparisons
- Total: $3(n - 1)/2 = \lceil 3n/2 \rceil - 2$

Case 2: n is even

- Initial setup: 1 comparison ($A[1]$ vs $A[2]$)
- Number of pairs in loop: $(n - 2)/2$
- Total: $1 + 3(n - 2)/2 = 3n/2 - 2$

In both cases:

$$\text{Total comparisons} = \left\lceil \frac{3n}{2} \right\rceil - 2$$

Since all other operations are $\Theta(1)$ per iteration:

$$T(n) = \Theta(n)$$

Note: Naive approach uses $2n - 2$ comparisons. This saves $n/2 - 2$ comparisons.

Properties

Property	Value	Reasoning
Stable	N/A	Not a sorting algorithm. Stability is not applicable to selection algorithms.
In-place	YES	Uses only $O(1)$ extra space (variables: min, max, i). No auxiliary arrays needed.
Adaptive	NO	Always processes elements in pairs regardless of input order. The number of comparisons is fixed at approximately $3n/2$.

10.3 Visual Walkthrough

Finding min and max in $A = [3, 8, 2, 9, 5, 1, 7, 4]$ ($n = 8$, even):

Step	Action	min	max	Comparisons
Init	$A[1] = 3 < A[2] = 8$	3	8	1
Pair 1	$A[3] = 2 < A[4] = 9 \rightarrow 2 < \text{min? Yes, } 9 > \text{max? Yes}$	2	9	3
Pair 2	$A[5] = 5 > A[6] = 1 \rightarrow 1 < \text{min? Yes, } 5 > \text{max? No}$	1	9	3
Pair 3	$A[7] = 7 > A[8] = 4 \rightarrow 4 < \text{min? No, } 7 > \text{max? No}$	1	9	3
Result: (min, max) = (1, 9) Total comparisons: $10 = 3(8)/2 - 2$				

Loop Invariant

At the start of each while loop iteration, min contains the minimum of $A[1 : i - 1]$ and max contains the maximum of $A[1 : i - 1]$.

Proof of Correctness:

Initialization: Before first while iteration, if n is even, we compare $A[1]$ and $A[2]$, setting min and max correctly for $A[1 : 2]$. If n is odd, $\text{min} = \text{max} = A[1]$ for $A[1 : 1]$.

Maintenance: At each iteration, we compare $A[i]$ and $A[i + 1]$. The smaller is compared with min and the larger with max. After processing the pair, min is the minimum of $A[1 : i + 1]$ and max is the maximum of $A[1 : i + 1]$.

Termination: When $i > n - 1$, all elements have been processed. If n is even, all n elements are in pairs. If n is odd, the last element $A[n]$ was included in the initialization. **QED**

11. Strassen's Matrix Multiplication

11.1 Algorithm Overview

Type: Divide-and-conquer matrix multiplication algorithm.

How it works: Recursively divides $n \times n$ matrices into $n/2 \times n/2$ submatrices, then uses 7 clever multiplications (instead of 8 in the standard divide-and-conquer approach) to compute the product, reducing complexity from $O(n^3)$ to $O(n^{2.81})$.

Best Cases to Use

- Large dense matrix multiplication
- When matrix dimensions are powers of 2
- Theoretical applications where lower exponent matters
- When standard $O(n^3)$ is too slow for large matrices

11.2 Complexity Analysis

MATRIX-ADD(A, B) and MATRIX-SUB(A, B)

For $n \times n$ matrices: nested loops over n^2 elements

Time: $\Theta(n^2)$ each

STRASSEN(A, B)

1-5 Base case ($n = 1$): single multiplication

$\rightarrow \Theta(1)$

6-7 Partition A and B into $n/2 \times n/2$ submatrices

$\Theta(1)$ (indexing)

8-17 10 matrix additions/subtractions (S1-S10)

$\rightarrow 10 \times \Theta(n^2/4) = \Theta(n^2)$

18-24 7 recursive multiplications (P1-P7)

$\rightarrow 7 \times T(n/2)$

25-28 8 matrix additions/subtractions for C11-C22

$\rightarrow \Theta(n^2)$

Recurrence Relation

$$T(n) = \begin{cases} \Theta(1) & \text{if } n = 1 \\ 7T(n/2) + \Theta(n^2) & \text{if } n > 1 \end{cases}$$

Explanation:

- 7 recursive calls on $n/2 \times n/2$ submatrices (P1 through P7)
- $\Theta(n^2)$ work for additions/subtractions: 10 additions/subtractions of $n/2 \times n/2$ matrices for S-matrices, plus 8 more for combining C submatrices, totaling $\Theta(n^2)$

Solving by Master Theorem

Step-by-Step Derivation:

Given: $T(n) = 7T(n/2) + \Theta(n^2)$

Identify parameters: $a = 7, b = 2, f(n) = \Theta(n^2)$

Step 1: Compute $n^{\log_b a}$:

$$n^{\log_2 7} \approx n^{2.807}$$

Step 2: Compare $f(n)$ with $n^{\log_b a}$:

$$f(n) = \Theta(n^2) = O(n^{\log_2 7 - \epsilon}) \text{ for } \epsilon \approx 0.81$$

Since n^2 is polynomially smaller than $n^{2.807}$, **Case 1** of the Master Theorem applies.

Step 3: Apply Case 1 result:

$$T(n) = \Theta(n^{\log_b a}) = \Theta(n^{\log_2 7})$$

$$\boxed{T(n) = \Theta(n^{2.807})}$$

Note: $\log_2(7) \approx 2.807$, which is less than 3 (standard matrix multiplication).

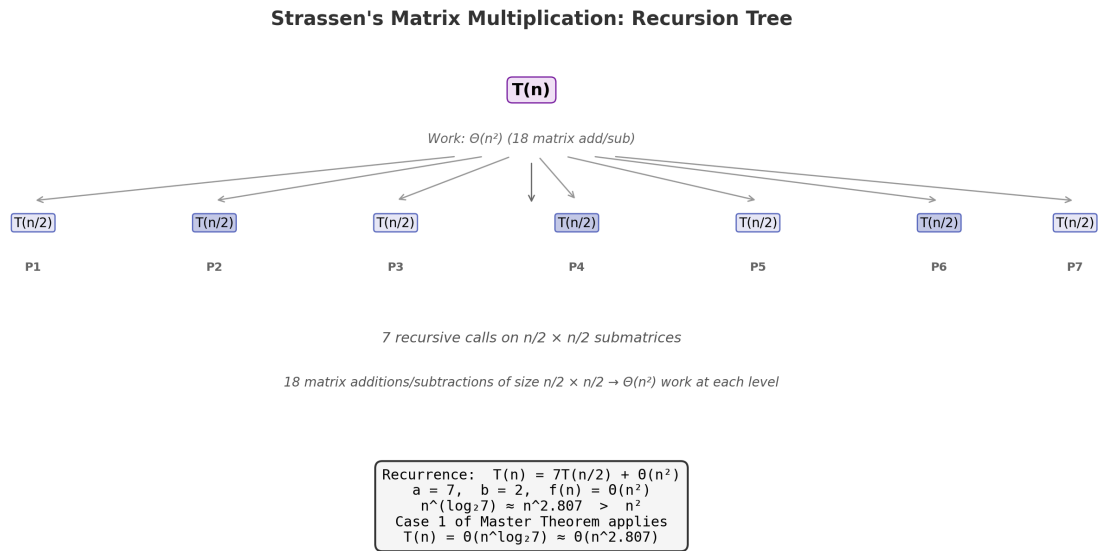


Figure 5: Strassen's algorithm recursion tree: 7 recursive calls on half-sized submatrices, with $\Theta(n^2)$ work at each level for matrix additions/subtractions. Since $7 > 2^2 = 4$, the recursive cost dominates, giving $T(n) = \Theta(n^{\log_2 7}) \approx \Theta(n^{2.807})$.

11.3 Properties

Property	Value	Reasoning
Stable	N/A	Not a sorting algorithm. Stability is not applicable to matrix operations.
In-place	NO	Requires multiple auxiliary matrices (S1-S10, P1-P7, and submatrices). Extra space is $\Theta(n^2)$ for the recursive stack and temporary matrices.
Adaptive	NO	Always performs the same 7 recursive multiplications and $\Theta(n^2)$ additions regardless of input matrix values.

11.4 Visual Walkthrough

Computing $C = A \times B$ where $A = \begin{bmatrix} 1 & 2 \\ 3 & 4 \end{bmatrix}, B = \begin{bmatrix} 5 & 6 \\ 7 & 8 \end{bmatrix}$ ($n = 2$):

Step 1: Partition into submatrices

$$A_{11} = [1], A_{12} = [2], A_{21} = [3], A_{22} = [4]$$

$$B_{11} = [5], B_{12} = [6], B_{21} = [7], B_{22} = [8]$$

Step 2: Compute S matrices (10 additions/subtractions)

S-Matrix	Formula	Value
S_1	$B_{12} - B_{22}$	$[6] - [8] = [-2]$
S_2	$A_{11} + A_{12}$	$[1] + [2] = [3]$
S_3	$A_{21} + A_{22}$	$[3] + [4] = [7]$
S_4	$B_{21} - B_{11}$	$[7] - [5] = [2]$
S_5	$A_{11} + A_{22}$	$[1] + [4] = [5]$
S_6	$B_{11} + B_{22}$	$[5] + [8] = [13]$
S_7	$A_{12} - A_{22}$	$[2] - [4] = [-2]$
S_8	$B_{21} + B_{22}$	$[7] + [8] = [15]$
S_9	$A_{11} - A_{21}$	$[1] - [3] = [-2]$
S_{10}	$B_{11} + B_{12}$	$[5] + [6] = [11]$

Step 3: Compute P matrices (7 recursive multiplications)

P-Matrix	Formula	Value
P_1	$A_{11} \times S_1$	$[1] \times [-2] = [-2]$
P_2	$S_2 \times B_{22}$	$[3] \times [8] = [24]$
P_3	$S_3 \times B_{11}$	$[7] \times [5] = [35]$
P_4	$A_{22} \times S_4$	$[4] \times [2] = [8]$
P_5	$S_5 \times S_6$	$[5] \times [13] = [65]$
P_6	$S_7 \times S_8$	$[-2] \times [15] = [-30]$
P_7	$S_9 \times S_{10}$	$[-2] \times [11] = [-22]$

Step 4: Combine to get C

$$C_{11} = P_5 + P_4 - P_2 + P_6 = [65] + [8] - [24] + [-30] = [19]$$

$$C_{12} = P_1 + P_2 = [-2] + [24] = [22]$$

$$C_{21} = P_3 + P_4 = [35] + [8] = [43]$$

$$C_{22} = P_5 + P_1 - P_3 - P_7 = [65] + [-2] - [35] - [-22] = [50]$$

Verification

$$C = \begin{bmatrix} 19 & 22 \\ 43 & 50 \end{bmatrix} \quad \text{and standard multiplication gives:}$$

$$C_{11} = 1(5) + 2(7) = 19\checkmark \quad C_{12} = 1(6) + 2(8) = 22\checkmark$$

$$C_{21} = 3(5) + 4(7) = 43\checkmark \quad C_{22} = 3(6) + 4(8) = 50\checkmark$$

Correctness Proof (Algebraic Verification)

We show that Strassen's formulas for C_{11} , C_{12} , C_{21} , C_{22} are algebraically equivalent to standard matrix multiplication.

For C_{11} : Standard formula: $C_{11} = A_{11}B_{11} + A_{12}B_{21}$

$$\begin{aligned} C_{11} &= P_5 + P_4 - P_2 + P_6 \\ &= (A_{11} + A_{22})(B_{11} + B_{22}) + A_{22}(B_{21} - B_{11}) - (A_{11} + A_{12})B_{22} + (A_{12} - A_{22})(B_{21} - B_{11}) \end{aligned}$$

Expanding and canceling terms (all cross-terms cancel):

$$\begin{aligned} &= A_{11}B_{11} + A_{11}B_{22} + A_{22}B_{11} + A_{22}B_{22} + A_{22}B_{21} - A_{22}B_{11} - A_{11}B_{22} - A_{12}B_{22} \\ &\quad - A_{12}B_{21} + A_{12}B_{11} + A_{22}B_{21} - A_{22}B_{11} \\ &= A_{11}B_{11} + A_{12}B_{21}\checkmark \end{aligned}$$

Similar algebraic verification holds for C_{12} , C_{21} , and C_{22} . By induction on matrix size (base case $n = 1$ is direct multiplication), STRASSEN is correct. **QED**

12. Summary

Master Theorem Reference

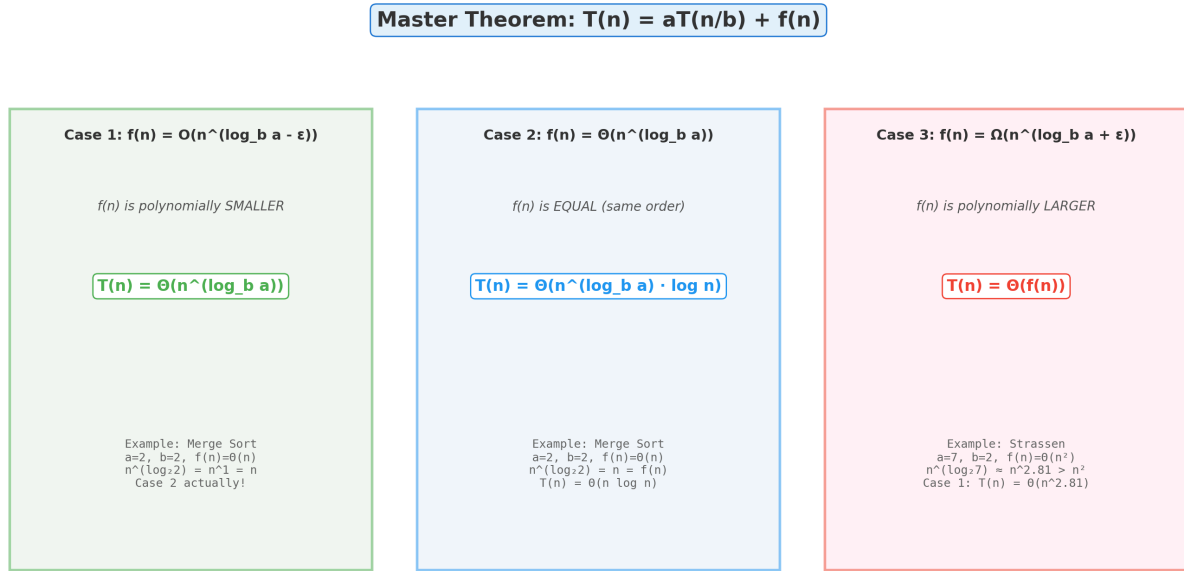


Figure 6: Master Theorem summary: Three cases based on comparing $f(n)$ with $n^{\log_b a}$. Case 1: recursive work dominates. Case 2: balanced. Case 3: non-recursive work dominates.

For recurrences of the form $T(n) = aT(n/b) + f(n)$ where $a \geq 1, b > 1$:

Case	Condition	Result	Examples
Case 1	$f(n) = O(n^{\log_b a - \epsilon})$ for $\epsilon > 0$	$T(n) = \Theta(n^{\log_b a})$	Strassen: $a = 7, b = 2, f(n) = n^2$
Case 2	$f(n) = \Theta(n^{\log_b a})$	$T(n) = \Theta(n^{\log_b a} \log n)$	Merge Sort: $a = 2, b = 2, f(n) = n$
Case 3	$f(n) = \Omega(n^{\log_b a + \epsilon})$ and $af(n/b) \leq cf(n)$	$T(n) = \Theta(f(n))$	Heapify: $a = 1, b = 3/2, f(n) = 1$

Complete Algorithm Comparison

Sorting Algorithms: Complexity & Properties Comparison

Algorithm	Worst Case	Average Case	Best Case	Stable	In-Place
Insertion Sort	$O(n^2)$	$O(n^2)$	$O(n)$	Yes	Yes
Selection Sort	$O(n^2)$	$O(n^2)$	$O(n^2)$	No	Yes
Merge Sort	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$	Yes	No
Bubble Sort	$O(n^2)$	$O(n^2)$	$O(n^2)$	Yes	Yes
Heap Sort	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$	No	Yes
Quick Sort	$O(n^2)$	$O(n \log n)$	$O(n \log n)$	No	Yes
Randomized QuickSort	$O(n^2)^*$	$O(n \log n)$	$O(n \log n)$	No	Yes
Counting Sort	$O(n+k)$	$O(n+k)$	$O(n+k)$	Yes	No

* Randomized Quicksort: Worst case occurs with very low probability $(1/n!)$ | Bubble Sort best case $O(n)$ with early termination optimization

Figure 7: Comprehensive comparison of all algorithms covered in Unit 1, including their time complexities and key properties.

Key Recurrences and Their Solutions

Algorithm	Recurrence	Solution	Method Used
Merge Sort	$T(n) = 2T(n/2) + \Theta(n)$	$\Theta(n \log n)$	Master Theorem (Case 2)
Quicksort (worst)	$T(n) = T(n-1) + \Theta(n)$	$\Theta(n^2)$	Unfolding / Telescoping
Quicksort (best)	$T(n) = 2T(n/2) + \Theta(n)$	$\Theta(n \log n)$	Master Theorem (Case 2)
Quicksort (avg)	$E[T(n)] = \frac{2}{n} \sum_{k=0}^{n-1} E[T(k)] + \Theta(n)$	$\Theta(n \log n)$	Substitution Method
Heapify	$T(n) \leq T(2n/3) + \Theta(1)$	$\Theta(\log n)$	Master Theorem (Case 2)
Strassen	$T(n) = 7T(n/2) + \Theta(n^2)$	$\Theta(n^{2.807})$	Master Theorem (Case 1)

Insertion Sort	$O(n^2)$	$O(n^2)$	$O(n)$	Yes	Yes	Yes
Selection Sort	$O(n^2)$	$O(n^2)$	$O(n^2)$	No	Yes	No
Merge Sort	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$	Yes	No	No
Bubble Sort	$O(n^2)$	$O(n^2)$	$O(n^2)^*$	Yes	Yes	No*
Heap Sort	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$	No	Yes	No
Quick Sort	$O(n^2)$	$O(n \log n)$	$O(n \log n)$	No	Yes	No
Randomized QS	$O(n^2)^{**}$	$O(n \log n)$	$O(n \log n)$	No	Yes	No
Counting Sort	$O(n + k)$	$O(n + k)$	$O(n + k)$	Yes	No	No
Minimum	$O(n)$	$O(n)$	$O(n)$	N/A	Yes	No
Sim. Min-Max	$O(n)$	$O(n)$	$O(n)$	N/A	Yes	No
Strassen	$O(n^{2.807})$	$O(n^{2.807})$	$O(n^{2.807})$	N/A	No	No

* Bubble Sort best case $O(n)$ with early termination optimization. Adaptive with optimization flag.

** Randomized Quicksort worst case occurs with very low probability ($1/n!$ for adversarial inputs).

Key Takeaways

When to Use Which Algorithm

- **Small n ($n \leq 20$):** Insertion Sort (low constant factors, adaptive)
- **General purpose, stable:** Merge Sort (guaranteed $O(n \log n)$, stable)
- **General purpose, in-place:** Quicksort or Heapsort
- **Guaranteed $O(n \log n)$ in-place:** Heapsort
- **Integer data, small range $k = O(n)$:** Counting Sort (linear time!)
- **Both min and max needed:** Simultaneous Min-Max ($3n/2$ comparisons vs $2n - 2$)
- **Matrix multiplication:** Strassen for large matrices ($O(n^{2.807})$ vs $O(n^3)$)

Unit 2: Graph Algorithms

Comprehensive Study Notes

Algorithms & Data Structures

Exam-Level Reference Material

Complete with Proofs, Visual Walkthroughs & Complexity Analysis

Table of Contents

Chapter 0: Introduction to Graphs

- 0.1 Fundamental Definitions
- 0.2 Examples and Applications
- 0.3 Paths, Connectivity, and Trees
- 0.4 Key Theorems on Trees

Chapter 1: Graph Representations

- 1.1 Adjacency Matrix
- 1.2 Adjacency List
- 1.3 Degree and the Handshaking Lemma

Chapter 2: Breadth-First Search (BFS)

- 2.1 Intuition and Layers
- 2.2 Algorithm and Working
- 2.3 Visual Walkthrough
- 2.4 Full Complexity Derivation
- 2.5 Proof of Correctness
- 2.6 The BFS Tree and Nontree Edges

Chapter 3: Depth-First Search (DFS)

- 3.1 Intuition and the Maze Analogy
- 3.2 Algorithm and Working
- 3.3 Visual Walkthrough
- 3.4 Full Complexity Derivation
- 3.5 Structural Theorems on DFS Trees

Chapter 4: Connected Components

- 4.1 The Generic Component-Growing Algorithm

4.2 Finding All Connected Components

4.3 The Disjointness Property

Chapter 5: Testing Bipartiteness

5.1 Definition and 2-Coloring

5.2 BFS-Based Algorithm

5.3 Visual Walkthrough

5.3 Theorems on Bipartiteness

Chapter 6: Directed Graphs

6.1 Representing Directed Graphs

6.2 BFS and DFS in Directed Graphs

6.3 Strong Connectivity

6.4 Strong Components

Chapter 7: DAGs and Topological Ordering

7.1 Directed Acyclic Graphs

7.2 Topological Ordering

7.3 Algorithm and Visual Walkthrough

Chapter 8: Dijkstra's Algorithm

8.1 Overview and Invariant

8.2 Algorithm and Working

8.3 Visual Walkthrough

8.4 Full Complexity Derivation

8.5 Proof of Correctness

Chapter 9: Minimum Spanning Trees

9.1 The MST Problem

9.2 The Cut Property

9.3 The Cycle Property

9.4 Prim's Algorithm

9.5 Reverse-Delete Algorithm

Summary: Key Facts and Complexity Table

Chapter 0: Introduction to Graphs

0.1 Fundamental Definitions

Definition: Graph (Undirected)

A **graph** G is a mathematical structure encoding **pairwise relationships** among a set of objects. Formally, $G = (V, E)$ where:

- V is a set of **nodes** (also called **vertices** — the terms are synonymous).
- E is a set of **edges**, where each edge $e \in E$ is a two-element subset $\{u, v\}$ for some $u, v \in V$.

The nodes u and v are called the **ends** of edge e . Edges encode a **symmetric relationship**: if $\{u, v\} \in E$, the relationship is mutual.

Notation warning: Although an undirected edge is technically a *set* $\{u, v\}$, it is commonly written as an ordered pair (u, v) . For undirected graphs, (u, v) and (v, u) refer to the exact same edge.

Definition: Directed Graph (Digraph)

A **directed graph** $G' = (V, E')$ models **asymmetric** relationships. Each directed edge $e' \in E'$ is an **ordered pair** (u, v) where:

- u is the **tail** of the edge (the edge **leaves** u).
- v is the **head** of the edge (the edge **enters** v).

Because the pair is ordered, (u, v) and (v, u) are **distinct** edges.

Default convention: When the term "graph" is used without qualification, it refers to an **undirected graph**.

Paths and Connectivity

Definition: Path

A **path** in an undirected graph is a sequence of nodes $P = v_1, v_2, \dots, v_k$ where each consecutive pair (v_i, v_{i+1}) is joined by an edge in E .

Definition: Simple Path

A path is **simple** if all its vertices are distinct (no node appears more than once).

Definition: Cycle

A **cycle** is a path $v_1, v_2, \dots, v_{k-1}, v_k$ satisfying:

1. $k > 2$ (at least three edges, preventing degenerate back-and-forth).
2. The first $k - 1$ nodes are all distinct.
3. $v_1 = v_k$ (the sequence cycles back to its start).

Definition: Connected Graph

An undirected graph is **connected** if for **every pair of nodes** u and v , there exists a path from u to v .

Definition: Strongly Connected (Directed)

A directed graph is **strongly connected** if for **every two nodes** u and v , there is a directed path from u to v **and** from v to u .

Definition: Distance

The **distance** between two nodes u and v is the **minimum number of edges** in a u - v path. If $u = v$, the distance is 0; if no path exists, the distance is ∞ .

0.2 Examples and Applications

Graphs model an enormous variety of real-world systems. For each application, understand **what nodes represent**, **what edges represent**, and **whether the graph is directed or undirected**.

Table 1 Major Categories of Graph Applications

Category	Nodes	Edges	Directed?
Transportation (Airlines)	Airports	Nonstop flights	Often undirected (bidirectional)
Communication (Internet)	Computers / ISPs	Physical links / Peering	Both
Information (WWW)	Web pages	Hyperlinks	Directed (crucial)
Social Networks	People	Friendship / Advice- seeking	Both
Dependency (Courses)	Courses	Prerequisites	Directed
Ecological Food Webs	Species	Consumption	Directed

0.3 Trees

Definition: Tree

An undirected graph is a **tree** if it satisfies two conditions:

1. It is **connected**.
2. It does **not contain a cycle**.

Key property: Deleting **any** edge from a tree disconnects it. Trees have no "redundant" edges — there is exactly one simple path between any two nodes.

Definition: Rooted Tree

A **rooted tree** is obtained by selecting one node r as the **root** and orienting each edge away from r . For every other node $v \neq r$:

- The **parent** of v is the unique node u that directly precedes v on the path from r to v .
- w is a **child** of v if v is the parent of w .
- w is a **descendant** of v (and v is an **ancestor** of w) if v lies on the unique path from r to w .
- A **leaf** is a node with no children.

0.4 Key Theorems on Trees

Theorem 3.1: Tree Edge Count

Every n -node tree has exactly $n - 1$ edges.

Proof (by Induction on n)

Base Case ($n = 1$): A tree with one node has 0 edges. Since $1 - 1 = 0$, the theorem holds.

Inductive Step: Assume the theorem holds for all trees with k nodes where $k < n$. Consider a tree T with n nodes.

Since T is a tree, it is connected and acyclic. Removing any edge from T disconnects it into exactly **two components** (there are no cycles, so every edge is a bridge). Let these components be T_1 and T_2 with n_1 and n_2 nodes respectively, where $n_1 + n_2 = n$ and $n_1, n_2 \geq 1$.

Each component is itself a tree (connected and acyclic). By the induction hypothesis:

$$|E(T_1)| = n_1 - 1 \quad \text{and} \quad |E(T_2)| = n_2 - 1$$

The total number of edges in T is:

$$|E(T)| = (n_1 - 1) + (n_2 - 1) + 1 = n_1 + n_2 - 1 = n - 1$$

The "+1" accounts for the edge we removed. Thus T has exactly $n - 1$ edges. ■

Theorem 3.2: Equivalence of Tree Properties

Let G be an undirected graph on n nodes. Any two of the following statements implies the third:

- (i) G is connected.
- (ii) G does not contain a cycle.
- (iii) G has $n - 1$ edges.

Proof**Case 1: (i) $\wedge$ (ii) $\Rightarrow$ (iii)**

If G is connected and acyclic, then by definition G is a tree. By Theorem 3.1, $|E(G)| = n - 1$. ■

Case 2: (i) $\wedge$ (iii) $\Rightarrow$ (ii)

Assume G is connected and has $n - 1$ edges. Suppose for contradiction that G contains a cycle. Remove one edge from this cycle. The graph remains connected (removing an edge from a cycle does not disconnect). Continue removing edges from cycles until no cycles remain. We obtain a connected, acyclic graph T' with n nodes. By Theorem 3.1, T' has $n - 1$ edges. But we removed at least one edge, so T' has at most $n - 2$ edges. **Contradiction.** Therefore G contains no cycle. ■

Case 3: (ii) $\wedge$ (iii) $\Rightarrow$ (i)

Assume G is acyclic and has $n - 1$ edges. Suppose for contradiction that G is disconnected. Then G has $k \geq 2$ connected components $C_1, C_2, \dots, C_k$ with $n_1, n_2, \dots, n_k$ nodes respectively, where $\sum n_i = n$.

Each component C_i is connected and acyclic (since G is acyclic), so each C_i is a tree. By Theorem 3.1, each C_i has $n_i - 1$ edges. The total number of edges is:

$$|E(G)| = \sum_{i=1}^k (n_i - 1) = \sum_{i=1}^k n_i - k = n - k$$

Since $k \geq 2$, we have $|E(G)| \leq n - 2$, contradicting the assumption that G has $n - 1$ edges. **Contradiction.** Therefore G must be connected. ■

Chapter 1: Graph Representations

Before implementing graph algorithms, we must decide how to represent a graph in memory. We use two fundamental parameters:

- $n = |V|$: number of nodes.
- $m = |E|$: number of edges.

For a simple undirected graph, the maximum number of edges is $\binom{n}{2} = \frac{n(n-1)}{2} = O(n^2)$. If the graph is connected, $m \geq n - 1$.

1.1 Adjacency Matrix Representation

Definition: Adjacency Matrix

Assume nodes are numbered $V = \{1, 2, \dots, n\}$. The **adjacency matrix** is an $n \times n$ matrix A where:

$$A[u, v] = \begin{cases} 1 & \text{if edge } (u, v) \in E \\ 0 & \text{otherwise} \end{cases}$$

For undirected graphs, A is **symmetric**: $A[u, v] = A[v, u]$.

Table 2 Adjacency Matrix: Trade-offs

Aspect	Performance	Analysis
Edge lookup	$O(1)$	Direct array access $A[u, v]$
Space	$\Theta(n^2)$	One entry per ordered pair of nodes
Neighbor enumeration	$\Theta(n)$	Must scan all n possible neighbors of u

Disadvantage: For sparse graphs where $m \ll n^2$, the adjacency matrix wastes space and time scanning mostly-zero rows.

1.2 Adjacency List Representation

Definition: Adjacency List

In the **adjacency list** representation, we maintain an array Adj of length n , where $Adj[v]$ is a list containing all nodes adjacent to v .

Table 3 Adjacency List: Trade-offs

Aspect	Performance	Analysis
Space	$O(m + n)$	n array pointers + $2m$ total list entries (undirected)
Edge lookup	$O(\deg(u))$	Scan $Adj[u]$ for target v
Neighbor enumeration	$O(1)$ per neighbor	Direct list iteration

Graph Representations: Adjacency Matrix vs Adjacency List

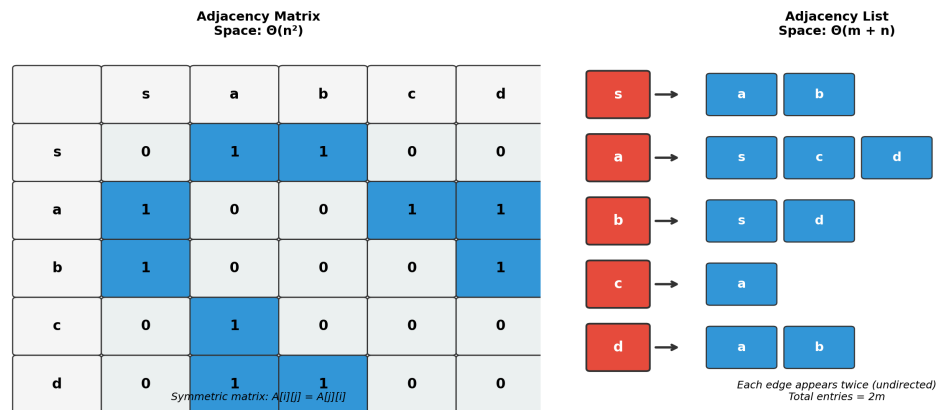


Figure 1: Visual comparison of adjacency matrix and adjacency list representations for the same graph. The matrix uses n^2 entries regardless of edge count, while the list uses only $2m$ entries for undirected graphs.

Theorem 3.10: Space Complexity of Representations

The adjacency matrix requires $O(n^2)$ space; the adjacency list requires $O(m + n)$ space.

Proof

Adjacency Matrix: The matrix has n rows and n columns, giving n^2 entries. Each entry is $O(1)$ space. Total: $O(n^2)$.

Adjacency List: We need an array of n pointers: $O(n)$. Each undirected edge $\{u, v\}$ appears in exactly two lists: $v \in Adj[u]$ and $u \in Adj[v]$. Thus total list entries = $2m = O(m)$. Total: $O(m + n)$. ■

1.3 Degree and the Handshaking Lemma

Definition: Degree

The **degree** of a node v , denoted n_v or $\deg(v)$, is the number of incident edges it has. In the adjacency list representation, $\deg(v) = |Adj[v]|$.

Theorem 3.9: The Handshaking Lemma

$$\sum_{v \in V} \deg(v) = 2m$$

Proof

Consider the sum of degrees over all nodes. Each edge $e = \{v, w\}$ contributes exactly 1 to $\deg(v)$ and exactly 1 to $\deg(w)$. Therefore, each edge contributes exactly 2 to the total sum of degrees.

Summing over all m edges:

$$\sum_{v \in V} \deg(v) = 2m$$

This identity is fundamental for proving linear-time bounds on graph algorithms. ■

Chapter 2: Breadth-First Search (BFS)

2.1 Intuition and Layers

Breadth-First Search (BFS) is the simplest algorithm for determining **s-t connectivity**: given a graph G and two nodes s (source) and t (target), is there a path from s to t ?

Physical analogy: Imagine the graph as a **maze** where nodes are rooms and edges are hallways. BFS corresponds to "flooding" the maze with an expanding wave starting at s . The wave grows outward uniformly, visiting all reachable nodes layer by layer.

Definition: BFS Layers

BFS constructs successive layers $L_0, L_1, L_2, \dots$:

- **Layer L_0 :** $\{s\}$ (just the starting node).
- **Layer L_1 :** All neighbors of s .
- **Layer L_{j+1} :** All nodes that (a) do not belong to any earlier layer, and (b) have an edge to at least one node in L_j .

Crucial connection to distance: Layer L_j is precisely the set of all nodes at **distance exactly j** from s . BFS simultaneously computes shortest paths (in terms of edge count) from s to every reachable node.

2.2 Algorithm and Working

Algorithm: BFS(s)

```

BFS(s):
    Set Discovered[s] = true and Discovered[v] = false for all other v
    Initialize L[0] to consist of the single element s
    Set the layer counter i = 0
    Set the current BFS tree T = ∅
    While L[i] is not empty
        Initialize an empty list L[i + 1]
        For each node u ∈ L[i]
            Consider each edge (u, v) incident to u
            If Discovered[v] = false then
                Set Discovered[v] = true
                Add edge (u, v) to the tree T
                Add v to the list L[i + 1]
            Endif
        Endfor
        Increment the layer counter i by one
    Endwhile

```

Initialization: The source node s is marked as discovered. All other nodes are undiscovered. Layer $L[0]$ contains only s . The BFS tree T starts empty.

Main Loop: The algorithm processes layers iteratively:

1. For each node u in the current layer $L[i]$, examine all edges (u, v) incident to u .
2. If v has not been discovered, mark it discovered, add edge (u, v) to the BFS tree T , and place v in the next layer $L[i + 1]$.
3. Once all nodes in $L[i]$ are processed, increment i and continue until a layer is empty.

Key Insight: Nodes are discovered in order of their distance from s . The BFS tree T contains exactly one path from s to each discovered node, and this path is a **shortest path** in the unweighted graph.

2.3 Visual Walkthrough

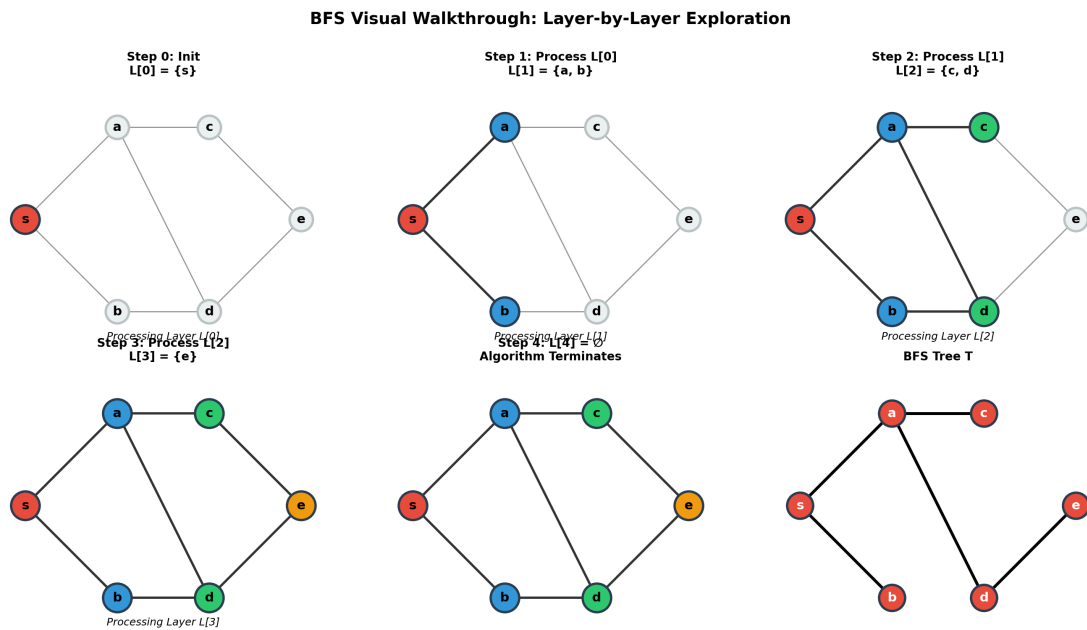


Figure 2: BFS layer-by-layer exploration starting from node s . Each step shows the newly discovered nodes (colored by layer) and the final BFS tree (rightmost panel). Red = Layer 0, Blue = Layer 1, Green = Layer 2, Orange = Layer 3.

Execution trace on the example graph:

Table 4 BFS Execution Trace

Step	Action	$L[0]$	$L[1]$	$L[2]$	$L[3]$	Discovered
Init	Set Discovered[s] = true	{ s }	$\emptyset$	$\emptyset$	$\emptyset$	{ s }
$i=0$	Process s : neighbors a, b discovered	{ s }	{ a, b }	$\emptyset$	$\emptyset$	{ s, a, b }
$i=1$	Process a : discovers c, d . Process b : d already discovered	{ s }	{ a, b }	{ c, d }	$\emptyset$	{ s, a, b, c, d }
$i=2$	Process c : neighbor a already known. Process d : discovers e	{ s }	{ a, b }	{ c, d }	{ e }	{ s, a, b, c, d, e }
$i=3$	Process e : neighbors c, d already known. $L[4] = \emptyset$	{ s }	{ a, b }	{ c, d }	{ e }	All nodes
$i=4$	$L[4]$ is empty	Algorithm terminates				

2.4 Full Complexity Derivation

Theorem 3.11: BFS Time Complexity

The above implementation of BFS runs in time $O(m + n)$ if the graph is given by adjacency list representation.

Step-by-Step Derivation

Step 1: Initialization

Setting `Discovered` for all n nodes takes $O(n)$ time. Initializing $L[0]$ and T takes $O(1)$.

Step 2: Main Loop — Node Processing

Each node u is placed in exactly one layer $L[i]$ and is processed exactly once (after being marked `Discovered`, it is never processed again).

Step 3: Edge Examination

When processing node u , the algorithm examines all edges incident to u by iterating through $Adj[u]$. The number of edges incident to u is $\deg(u)$.

The total time spent examining all edges is:

$$T_{\text{edges}} = \sum_{u \in V} \deg(u) = 2m \quad (\text{by Theorem 3.9, the Handshaking Lemma})$$

Step 4: Constant-Time Operations

Each discovery event involves: marking `Discovered[v] = true` ($O(1)$), adding edge to T ($O(1)$), adding v to layer list ($O(1)$). Since each node is discovered at most once, total: $O(n)$.

Step 5: Total Time

$$T_{\text{total}} = T_{\text{init}} + T_{\text{edges}} + T_{\text{ops}} = O(n) + O(2m) + O(n) = O(m + n)$$

For connected graphs, $m \geq n - 1$, so $O(m + n)$ is equivalent to $O(m)$. This is called **linear time** because it is linear in the input size ($\Theta(m + n)$ bits to represent the adjacency list). ■

2.5 Proof of Correctness

Theorem 3.3: BFS Layer Correctness

For each $j \geq 1$, layer L_j produced by BFS consists of all nodes at distance exactly j from s . There is a path from s to t if and only if t appears in some layer.

Proof (by Induction on j)

Part 1: L_j contains exactly nodes at distance j from s .

Base Case ($j = 0$): $L[0] = \{s\}$. The distance from s to itself is 0. No other node is at distance 0. Correct.

Inductive Step: Assume $L[i]$ contains exactly all nodes at distance i from s . We show $L[i + 1]$ contains exactly all nodes at distance $i + 1$ from s .

($\subseteq$) Let $v \in L[i + 1]$. Then v was discovered from some $u \in L[i]$ where $(u, v) \in E$ and v was undiscovered. By induction, $\text{dist}(s, u) = i$. Therefore there exists a path of length i from s to u , and adding edge (u, v) gives a path of length $i + 1$ from s to v . So $\text{dist}(s, v) \leq i + 1$.

Could $\text{dist}(s, v) < i + 1$? If $\text{dist}(s, v) \leq i$, then v would have been discovered in some layer $L[k]$ where $k \leq i$ (by the inductive hypothesis and the fact that BFS discovers nodes in non-decreasing order of distance). But v was undiscovered when we processed $L[i]$. **Contradiction.** Thus $\text{dist}(s, v) = i + 1$.

($\supseteq$) Let v be a node with $\text{dist}(s, v) = i + 1$. Then there exists a shortest path $s \rightsquigarrow \dots \rightsquigarrow u \rightarrow v$ where $\text{dist}(s, u) = i$. By the induction hypothesis, $u \in L[i]$. When processing u , the algorithm examines edge (u, v) . If v were already discovered, it would be in some layer $L[k]$ with $k \leq i$, implying $\text{dist}(s, v) = k \leq i$. **Contradiction.** Therefore v is undiscovered, and the algorithm adds v to $L[i + 1]$.

Part 2: Path existence $\Leftrightarrow$ appearance in some layer.

($\Rightarrow$) If there is a path from s to t , let $d = \text{dist}(s, t)$. By Part 1, $t \in L[d]$.

($\Leftarrow$) If $t \in L[j]$ for some j , then t was discovered from some node in $L[j - 1]$, which was discovered from $L[j - 2]$, and so on back to $s \in L[0]$. The edges used in discovery form a path from s to t in T . ■

2.6 The BFS Tree and Nontree Edges

Definition: BFS Tree

As BFS executes, it produces a **tree** T rooted at s containing exactly the nodes reachable from s . Node u becomes the **parent** of v if v is discovered while examining u .

The tree T contains exactly one path from s to each reachable node v , and because of layer-by-layer construction, this path is a **shortest path** in G .

Theorem 3.4: BFS Nontree Edge Property

Let T be a BFS tree, let x and y be nodes in T belonging to layers L_i and L_j respectively, and let (x, y) be an edge of G . Then $|i - j| \leq 1$.

Proof

Without loss of generality, assume x is processed before y (or simultaneously if in the same layer). When x is processed, all its neighbors are examined. If y is undiscovered, it is placed in $L[i + 1]$, so $j = i + 1$.

If y is already discovered, it must be in some layer $L[k]$ where $k \leq i + 1$ (since BFS processes layers in order). Also, since (x, y) is an edge and $x \in L_i$:

$$\text{dist}(s, y) \leq \text{dist}(s, x) + 1 = i + 1 \Rightarrow k \leq i + 1$$

Similarly, since (x, y) is an edge:

$$\text{dist}(s, x) \leq \text{dist}(s, y) + 1 = j + 1 \Rightarrow i \leq j + 1 \Rightarrow j \geq i - 1$$

Combining: $i - 1 \leq j \leq i + 1$, so $|i - j| \leq 1$. ■

Interpretation: Nontree edges can only connect nodes within the **same layer** or in **adjacent layers**. They **cannot** connect nodes whose layer numbers differ by 2 or more. If such an edge existed, the deeper node would have been discovered earlier via that edge.

Chapter 3: Depth-First Search (DFS)

3.1 Intuition and the Maze Analogy

Depth-First Search (DFS) corresponds to the strategy you would use if physically walking through a maze:

1. Start at s .
2. Pick the first edge leading out of s to some node v .
3. From v , pick the first edge leading to an unexplored node.
4. Continue going **deeper and deeper** until you reach a "**dead end**" — a node whose all neighbors are explored.
5. When you hit a dead end, **backtrack** to the most recent node that still has an unexplored neighbor, and resume from there.

DFS explores the graph by going as **deeply** as possible along each branch before retreating — hence the name "depth-first."

3.2 Algorithm and Working

Algorithm: DFS(s) — Iterative Stack-Based

```
DFS( $s$ ):
    Initialize  $S$  to be a stack with one element  $s$ 
    While  $S$  is not empty
        Take a node  $u$  from  $S$ 
        If Explored[ $u$ ] = false then
            Set Explored[ $u$ ] = true
            For each edge  $(u, v)$  incident to  $u$ 
                Add  $v$  to the stack  $S$ 
            Endfor
        Endif
    Endwhile
```

Initialization: Push the source node s onto stack S . All nodes are initially unexplored.

Main Loop: While the stack is not empty:

1. **Pop** a node u from the stack.
2. If u has not been explored, mark it as `Explored[u] = true`.
3. Push **all neighbors** of u onto the stack.
4. The next iteration pops one of these neighbors (the **last** one pushed), creating a depth-first pattern.

Key Insight: Because we use a stack (LIFO), DFS always extends the **most recently discovered** path. This creates a "deep" exploration pattern rather than the "broad" pattern of BFS.

Definition: DFS Tree

Like BFS, DFS yields a **rooted tree** T on the connected component containing s :

- The root of T is s .
- Node u becomes the **parent** of v if $\text{DFS}(v)$ is invoked directly during the execution of $\text{DFS}(u)$, or equivalently, if v is pushed onto the stack while processing u and v is later marked Explored from that push.

3.3 Visual Walkthrough

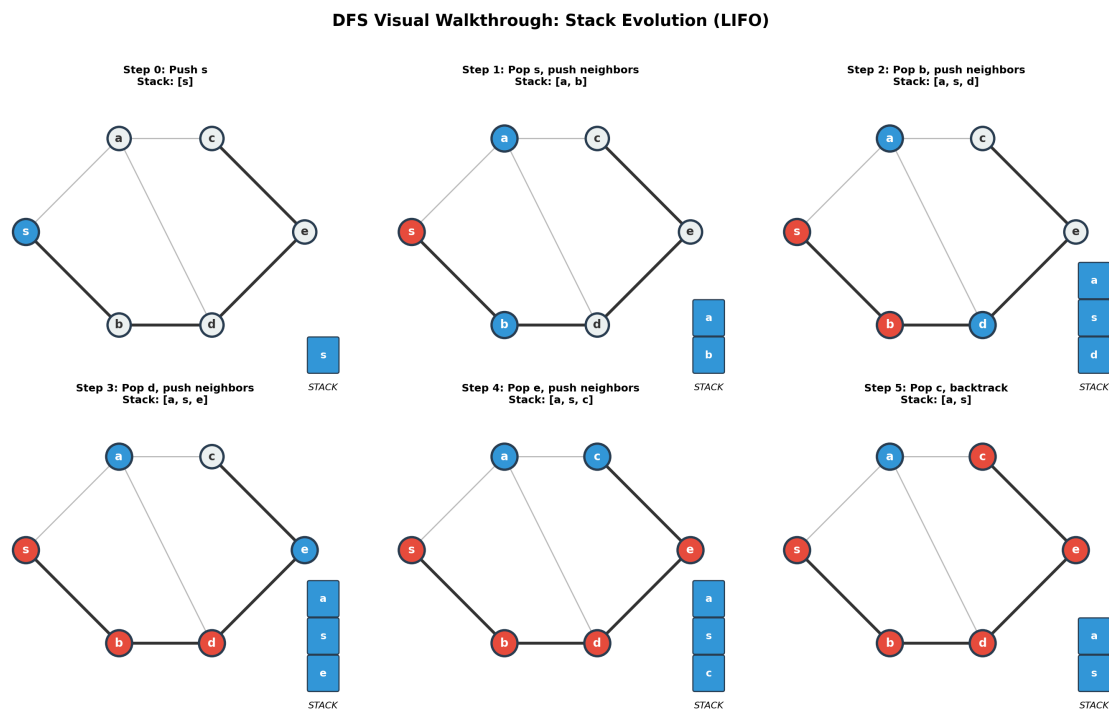


Figure 3: DFS stack evolution starting from node *s*. Red nodes are Explored; blue nodes are on the stack. The stack visualization shows LIFO behavior: the last neighbor pushed is processed first, causing deep exploration.

Execution trace (processing adjacency lists in alphabetical order):

Table 5 DFS Stack Trace

Step	Stack S (top at right)	u popped	Explored	Action
1	[s]	s	$\emptyset$	Push a, b $\rightarrow$ S = [a, b]
2	[a, b]	b	{s}	Push s, d $\rightarrow$ S = [a, s, d]
3	[a, s, d]	d	{s, b}	Push b, e $\rightarrow$ S = [a, s, b, e]
4	[a, s, b, e]	e	{s, b, d}	Push c, d $\rightarrow$ S = [a, s, b, c, d]
5	[a, s, b, c, d]	d	{s, b, d, e}	Already explored, skip
6	[a, s, b, c]	c	{s, b, d, e}	Push a, e $\rightarrow$ S = [a, s, b, a, e]
7	[a, s, b, a, e]	e	{s, b, d, e, c}	Already explored, skip
8	[a, s, b, a]	a	{s, b, d, e, c}	Push s, c $\rightarrow$ S = [a, s, b, s, c]
9–14	Remaining pops all hit already-explored nodes; stack empties. Algorithm terminates.			

DFS Tree edges: $(s, b), (b, d), (d, e), (e, c), (c, a)$. The DFS path goes deep: $s \rightarrow b \rightarrow d \rightarrow e \rightarrow c \rightarrow a$.

3.4 Full Complexity Derivation

Theorem 3.13: DFS Time Complexity

The above implementation of DFS runs in time $O(m + n)$ if the graph is given by adjacency list representation.

Step-by-Step Derivation

Step 1: Node Marking

Each node is marked `Explored` at most once. The check `If Explored[u] = false` is executed once per stack pop. There are at most $O(m)$ pops (see Step 3), so checking costs $O(m)$.

Step 2: When is a node processed?

A node u is first popped and found unexplored exactly once. At that moment, its entire adjacency list $Adj[u]$ is scanned, taking $O(\deg(u))$ time. After being marked `Explored`, subsequent pops of u are skipped in $O(1)$.

Step 3: Stack Push Operations

Node v is pushed onto the stack every time one of its $\deg(v)$ adjacent nodes is explored. Therefore, the total number of pushes is at most:

$$\sum_{u \in V} \deg(u) = 2m = O(m)$$

Since each push corresponds to one eventual pop, the total number of stack operations is $O(m)$.

Step 4: Adjacency List Scanning (the dominant cost)

The total time spent scanning adjacency lists (when nodes are first found unexplored) is:

$$T_{\text{scan}} = \sum_{u \in V} \deg(u) = 2m = O(m)$$

Step 5: Initialization

Initializing `Explored` array and stack: $O(n)$.

Step 6: Total Time

$$T_{\text{total}} = O(n) + O(m) + O(m) = O(m + n)$$

Each of n nodes and m edges is processed a constant number of times. ■

3.5 Structural Theorems on DFS Trees

Theorem 3.6: DFS Descendant Property

For a given recursive call $\text{DFS}(u)$, all nodes that are marked "Explored" between the invocation and end of this recursive call are descendants of u in T .

Proof

In the recursive formulation, when $\text{DFS}(u)$ is called, it marks u as Explored and then recursively calls $\text{DFS}(v)$ for each unexplored neighbor v .

Each such recursive call $\text{DFS}(v)$ marks v and its descendants before returning. Since v is added as a child of u in T , all nodes marked during $\text{DFS}(v)$ are descendants of v , and hence descendants of u .

No nodes outside the subtree of u are marked during $\text{DFS}(u)$ because the algorithm only explores neighbors of nodes in the current call stack. ■

Theorem 3.7: DFS Nontree Edge Property

Let T be a DFS tree, let x and y be nodes in T , and let (x, y) be an edge of G that is not an edge of T . Then one of x or y is an ancestor of the other in T .

Proof

Consider the edge $(x, y) \notin T$. Without loss of generality, assume x was marked Explored **before** y during the DFS execution.

When $\text{DFS}(x)$ was executed, it examined all neighbors of x . Since y was not marked Explored at that time (otherwise (x, y) would be in T), y must have been discovered **later** during the execution of $\text{DFS}(x)$ or during a recursive call initiated from $\text{DFS}(x)$.

If y was discovered during $\text{DFS}(x)$ directly, then y is a **child** of x . If y was discovered during $\text{DFS}(z)$ where z is a descendant of x , then y is a descendant of z , and hence a **descendant** of x . In either case, y is in the subtree rooted at x , meaning x is an **ancestor** of y in T . ■

Theorem 3.12: Iterative DFS Equivalence

The stack-based algorithm implements DFS, visiting nodes in exactly the same order as the recursive DFS procedure (except that each adjacency list is processed in reverse order).

Proof

Both the recursive and iterative versions use a stack. In the recursive version, the call stack implicitly stores the path. When $\text{DFS}(u)$ calls $\text{DFS}(v)$, v is processed before returning to u .

In the iterative version, pushing neighbors onto the stack ensures the **last neighbor pushed** is processed first (LIFO), mimicking the deepest recursive call. The only difference is that the iterative version pushes all neighbors at once, so if adjacency lists are processed in the same order, the iterative version processes them in **reverse** due to stack semantics. Reversing the adjacency list order makes the two identical. ■

Chapter 4: Connected Components

4.1 The Generic Component-Growing Algorithm

BFS and DFS are both instances of a more general algorithm for "exploring" a graph starting from s :

Algorithm: Generic Component Growing

```
R will consist of nodes to which  $s$  has a path
Initially  $R = \{s\}$ 
While there is an edge  $(u, v)$  where  $u \in R$  and  $v \notin R$ 
    Add  $v$  to  $R$ 
Endwhile
```

Working: The algorithm maintains a set R of reachable nodes. Initially $R = \{s\}$. At each step, if there exists an edge from a node in R to a node outside R , that outside node is added to R . The algorithm terminates when no such edge exists, meaning all edges from R stay within R .

Why is this safe? If there is a path P from s to u (which there must be, since $u \in R$), then there is a path from s to v obtained by first following P and then taking the edge (u, v) . Therefore, v is indeed reachable from s .

Theorem 3.5: Correctness of Component Growing

The set R produced at the end of the algorithm is precisely the connected component of G containing s .

Proof

Let $C(s) = \{v \in V : \text{there is a path from } s \text{ to } v \text{ in } G\}$ be the connected component containing s .

$(R \subseteq C(s))$: By induction on the number of iterations.

- *Base case*: $s \in R$ and $s \in C(s)$.
- *Inductive step*: If v is added to R because (u, v) is an edge with $u \in R$, and $u \in C(s)$ by induction, then there is a path from s to u , and adding edge (u, v) gives a path from s to v . Thus $v \in C(s)$.

$(C(s) \subseteq R)$: Suppose for contradiction that there exists $v \in C(s)$ but $v \notin R$. Since $v \in C(s)$, there is a path $s = v_0, v_1, \dots, v_k = v$ from s to v .

Let v_i be the **first** node on this path that is not in R . Then $i \geq 1$ (since $s \in R$), and $v_{i-1} \in R$. The edge (v_{i-1}, v_i) has one endpoint in R and one outside R .

But the algorithm only terminates when **no such edge exists**. **Contradiction**. Therefore $C(s) \subseteq R$.

Since $R \subseteq C(s)$ and $C(s) \subseteq R$, we have $R = C(s)$. ■

4.2 Finding All Connected Components

To find **all** connected components (not just the one containing s):

Algorithm: All Connected Components

```
Find-All-Components(G):
    Initialize all nodes as unvisited
    While there exists an unvisited node s
        Run BFS or DFS from s to generate its component
        Mark all reached nodes as visited
    Return the set of all components found
```

Why components are disjoint: By definition of connected components, the component found from v will be disjoint from the component of s (and all previously found

components). There are no edges between different connected components.

Running Time: Although we may run BFS/DFS multiple times (once per component), each individual run only spends work on edges and nodes **within that component**. No edge or node is ever processed by more than one invocation.

Complexity Analysis

Each node is visited exactly once across all searches. Each edge is examined exactly once across all searches (twice for undirected graphs, once from each endpoint). Therefore, the total work remains proportional to the total number of nodes plus edges:

$$T_{\text{total}} = O(m + n)$$

4.3 The Disjointness Property

Theorem 3.8: Connected Components are Disjoint

For any two nodes s and t in a graph, their connected components are either identical or disjoint.

Proof

Let $C(s)$ and $C(t)$ be the connected components containing s and t .

Case 1: If there is no path between s and t , then $C(s)$ and $C(t)$ share no common nodes. If they shared a node u , then there would be paths $s \rightsquigarrow u$ and $u \rightsquigarrow t$, implying a path $s \rightsquigarrow t$ by transitivity. **Contradiction.** Thus $C(s) \cap C(t) = \emptyset$.

Case 2: If there is a path between s and t , we show $C(s) = C(t)$.

($C(s) \subseteq C(t)$): Let $v \in C(s)$. Then there is a path from s to v . Since there is a path from s to t , there is a path from t to s (undirected graph). Concatenating path $t \rightsquigarrow s$ with path $s \rightsquigarrow v$ gives a path $t \rightsquigarrow v$. Thus $v \in C(t)$.

($C(t) \subseteq C(s)$): By symmetric argument. Therefore $C(s) = C(t)$. ■

Chapter 5: Testing Bipartiteness

5.1 Definition and 2-Coloring

Definition: Bipartite Graph

A graph $G = (V, E)$ is **bipartite** if its node set V can be partitioned into two disjoint sets X and Y such that **every edge** in E has one end in X and the other end in Y . No edge has both ends in X or both ends in Y .

Coloring analogy: Color all nodes in X **red** and all nodes in Y **blue**. A graph is bipartite if and only if it can be 2-colored so that every edge has one red end and one blue end. This 2-coloring formulation is equivalent to the partition definition.

5.2 BFS-Based Algorithm

Algorithm: Test-Bipartite(s)

```

Test-Bipartite(s):
    Initialize L[0] to be a list with one element s
    Set Color[s] = red
    Set Explored[s] = true
    Set i = 0
    While L[i] is not empty
        Initialize a new empty list L[i + 1]
        For each node u in L[i]
            For each edge (u, v) incident to u
                If Explored[v] = false then
                    Set Explored[v] = true
                    If (i + 1) is even then
                        Set Color[v] = red
                    Else
                        Set Color[v] = blue
                    Endif
                Add v to the list L[i + 1]
            Endif
        Endfor
        Set i = i + 1
    Endwhile
    For each edge (u, v) in the graph
        If Color[u] = Color[v] then
            Return "Not Bipartite"
        Endif
    Endfor
    Return "Bipartite"

```

Working: The algorithm uses BFS to 2-color the graph. Nodes in even layers are colored red; nodes in odd layers are colored blue. After BFS completes, it checks every edge in the graph. If any edge connects two nodes of the same color, the graph is not bipartite.

Key Insight: In a bipartite graph, all edges must go between the two partitions. In BFS terms, edges can only connect nodes in adjacent layers (different parity). An edge within the same layer (same parity) indicates an odd cycle.

5.3 Visual Walkthrough

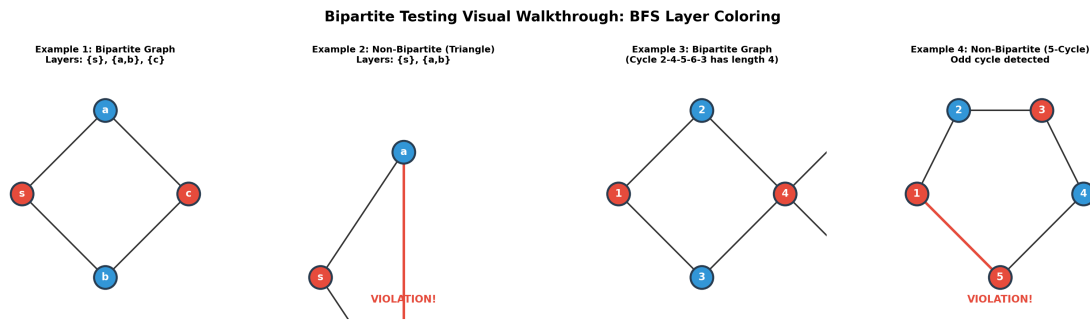


Figure 4: Bipartite testing visual walkthrough. Example 1 (left) shows a valid bipartite graph with proper 2-coloring. Example 2 shows a triangle where the edge (a, b) connects two blue nodes — a violation proving non-bipartiteness. Examples 3 and 4 contrast even and odd cycles.

5.4 Theorems on Bipartiteness

Theorem 3.14: Bipartite Graphs Have No Odd Cycles

If a graph G is bipartite, then it cannot contain an odd cycle.

Proof

Suppose G is bipartite with partition (X, Y) . Consider any cycle $v_1, v_2, \dots, v_k, v_1$.

Without loss of generality, let $v_1 \in X$. Since G is bipartite:

- $v_2 \in Y$ (adjacent to $v_1 \in X$)
- $v_3 \in X$ (adjacent to $v_2 \in Y$)
- $v_4 \in Y$ (adjacent to $v_3 \in X$)

Continuing this alternating pattern: all **odd-indexed** nodes are in X , all **even-indexed** nodes are in Y .

For the cycle to close, v_k must be in Y (to connect back to $v_1 \in X$), so k must be **even**. Therefore every cycle has even length. ■

Theorem 3.15: BFS-Based Bipartite Test is Correct

Let G be a connected graph, and let $L_1, L_2, \dots$ be the layers produced by BFS starting at node s . Then exactly one of the following holds:

- (i) There is no edge joining two nodes of the same layer. In this case G is bipartite.
- (ii) There is an edge joining two nodes of the same layer. In this case G contains an odd-length cycle and cannot be bipartite.

Proof

Mutual Exclusivity: If there is no edge within the same layer, then (ii) is false. If there is an edge within the same layer, then (i) is false. So exactly one holds.

Case (i): Suppose no edge joins two nodes in the same layer. By Theorem 3.4, edges only connect nodes in adjacent layers (or same layer, but we assumed none exist). Color even layers red and odd layers blue. Every edge connects adjacent layers, hence connects red to blue. Therefore G is bipartite. ■

Case (ii): Suppose edge (x, y) joins two nodes in the same layer L_i . Let z be the least common ancestor of x and y in the BFS tree T . Suppose $z \in L_j$ where $j < i$.

In T , there is a path from z to x of length $i - j$, and a path from z to y of length $i - j$. The edge (x, y) closes a cycle: $z \rightsquigarrow x \rightarrow y \rightsquigarrow z$. The length of this cycle is:

$$(i - j) + 1 + (i - j) = 2(i - j) + 1$$

which is **odd**. By Theorem 3.14, G cannot be bipartite. ■

Conclusion: The BFS-based algorithm correctly tests bipartiteness in $O(m + n)$ time because it performs one BFS ($O(m + n)$) plus one scan of all edges ($O(m)$).

Chapter 6: Directed Graphs

6.1 Representing Directed Graphs

The adjacency list representation is adapted for directed graphs as follows:

Directed Adjacency List

Each node u has **two lists** associated with it:

1. **Outgoing edges:** A list of nodes v such that $(u, v) \in E$ (nodes reachable by going one step forward).
2. **Incoming edges:** A list of nodes w such that $(w, u) \in E$ (nodes from which u can be reached by going one step backward).

This dual-list structure allows an algorithm examining node u to read off both the forward-reachable and backward-reachable nodes.

6.2 BFS and DFS in Directed Graphs

BFS and DFS extend naturally to directed graphs by **following edge directions only**:

Table 6 Directed Graph Search Algorithms

Algorithm	Behavior	Computes	Running Time
BFS from s	Follows outgoing edges	Nodes reachable from s	$O(m + n)$
DFS from s	Follows outgoing edges recursively	Nodes reachable from s	$O(m + n)$
BFS from s in G^{rev}	Follows reversed edges	Nodes that can reach to s	$O(m + n)$

The Reversal Trick

Define G^{rev} by reversing every edge: if $(u, v) \in E$, then $(v, u) \in E^{rev}$. Then:

v has a path from s in $G^{rev} \Leftrightarrow v$ has a path to s in G .

6.3 Strong Connectivity

Definition: Mutual Reachability

Two nodes u and v in a directed graph are **mutually reachable** if there is a path from u to v **and** a path from v to u .

Definition: Strongly Connected Graph

A directed graph is **strongly connected** if **every pair** of nodes is mutually reachable.

Theorem 3.16: Transitivity of Mutual Reachability

If u and v are mutually reachable, and v and w are mutually reachable, then u and w are mutually reachable.

Proof

Since u and v are mutually reachable, there exists a path P_1 from u to v and a path P_2 from v to u .

Since v and w are mutually reachable, there exists a path P_3 from v to w and a path P_4 from w to v .

Concatenating P_1 followed by P_3 gives a path from u to w .

Concatenating P_4 followed by P_2 gives a path from w to u .

Therefore u and w are mutually reachable. ■

Corollary: Mutual reachability is an **equivalence relation** (reflexive, symmetric, transitive). Thus it partitions V into equivalence classes.

Algorithm: Testing Strong Connectivity

```

Test-Strongly-Connected(G):
    Pick any node s
    Run BFS (or DFS) in G starting from s
    Run BFS (or DFS) in G_rev starting from s
    If both searches reach every node then
        Return "Strongly Connected"
    Else
        Return "Not Strongly Connected"

```

Proof of Correctness**If both searches reach all nodes:**

- BFS in G from s reaching all nodes means: for every v , there is a path $s \rightsquigarrow v$ in G .
- BFS in G^{rev} from s reaching all nodes means: for every v , there is a path $s \rightsquigarrow v$ in G^{rev} , which means there is a path $v \rightsquigarrow s$ in G .

Therefore, for every v , s and v are mutually reachable.

Now consider any two arbitrary nodes u and w :

- s and u are mutually reachable.
- s and w are mutually reachable.
- By transitivity (Theorem 3.16), u and w are mutually reachable.

Since u and w were arbitrary, **every pair** is mutually reachable. Hence G is strongly connected.

Conversely: If either search misses some node, then there exists a node not reachable from s (or from which s is not reachable), so s is not mutually reachable with that node, and G cannot be strongly connected. ■

Running Time: Two BFS/DFS executions, each $O(m + n)$. Total: $O(m + n)$.

6.4 Strong Components

Definition: Strong Component

The **strong component** containing a node s is the set of all nodes v such that s and v are mutually reachable. It is the maximal set of nodes where every node can reach every other via directed paths.

Computing the strong component of s :

- Run BFS from s in $G \rightarrow$ obtain R_{forward} (nodes reachable **from** s).
- Run BFS from s in $G^{\text{rev}} \rightarrow$ obtain R_{backward} (nodes that can reach **to** s).
- The **intersection** $R_{\text{forward}} \cap R_{\text{backward}}$ is exactly the strong component of s .

Theorem 3.17: Strong Components are Disjoint

For any two nodes s and t in a directed graph, their strong components are either identical or disjoint.

Proof

Let $SCC(s)$ and $SCC(t)$ be the strong components containing s and t .

Case 1: If s and t are mutually reachable, then by Theorem 3.16, any node $u \in SCC(s)$ is mutually reachable with s , and s is mutually reachable with t , so u is mutually reachable with t . Thus $u \in SCC(t)$. Similarly $SCC(t) \subseteq SCC(s)$. Therefore $SCC(s) = SCC(t)$.

Case 2: If s and t are not mutually reachable, then there is no node that is mutually reachable with both (otherwise by transitivity, s and t would be mutually reachable). Thus $SCC(s) \cap SCC(t) = \emptyset$. ■

Chapter 7: DAGs and Topological Ordering

7.1 Directed Acyclic Graphs

Definition: DAG

A **directed acyclic graph** (DAG) is a directed graph that contains **no directed cycles**.

Contrast with undirected acyclic graphs: In undirected graphs, acyclicity implies simplicity: each component is a tree with $n - c$ edges. Directed graphs behave very differently. A DAG can be dense: the graph on nodes $\{1, 2, \dots, n\}$ with edges (i, j) for all $i < j$ has $\binom{n}{2} = \Theta(n^2)$ edges and **no directed cycles** (every edge goes from smaller to larger index).

Why DAGs are ubiquitous: DAGs naturally encode **precedence relations**. If task i must be done before task j , add edge (i, j) . If the graph contained a cycle, the precedence relation would be **internally contradictory** — no task in the cycle could ever be done first.

7.2 Topological Ordering

Definition: Topological Ordering

A **topological ordering** of a directed graph G with n nodes is an ordering $v_1, v_2, \dots, v_n$ such that for **every directed edge** $(v_i, v_j) \in E$, we have $i < j$. Every edge points "forward" in the ordering.

Scheduling interpretation: A topological ordering provides a valid linear sequence for performing all tasks. When we reach task v_j , every prerequisite task v_i appears earlier and has already been completed.

Theorem 3.18: Topological Ordering Implies DAG

If G has a topological ordering, then G is a DAG.

Proof

Suppose for contradiction that G has a topological ordering and contains a directed cycle $C = (v_{i_1}, v_{i_2}, \dots, v_{i_k}, v_{i_1})$.

By the topological ordering property:

- $(v_{i_1}, v_{i_2}) \in E \Rightarrow i_1 < i_2$
- $(v_{i_2}, v_{i_3}) \in E \Rightarrow i_2 < i_3$
- $\dots$
- $(v_{i_k}, v_{i_1}) \in E \Rightarrow i_k < i_1$

Chaining these: $i_1 < i_2 < \dots < i_k < i_1$. **Contradiction.** Therefore G contains no directed cycles. ■

Theorem 3.19: Every DAG Has a Source Node

In every DAG G , there is at least one node with no incoming edges (in-degree 0).

Proof

Suppose for contradiction that every node has at least one incoming edge. Start from any node v_1 and follow an incoming edge to v_2 , then from v_2 follow an incoming edge to v_3 , and so on.

Since there are only n nodes, by the **pigeonhole principle**, some node must repeat within $n + 1$ steps, creating a directed cycle. This contradicts G being a DAG. ■

Theorem 3.20: Every DAG Has a Topological Ordering

If G is a DAG, then G has a topological ordering.

Proof (by Induction on n)

Base Case ($n = 1$): The single node is trivially a topological ordering.

Inductive Step: Assume every DAG with $k < n$ nodes has a topological ordering. Let G be a DAG with n nodes.

By Theorem 3.19, G has a node v with no incoming edges. Remove v to obtain $G' = G - \{v\}$. G' is also a DAG (removing nodes cannot create cycles). G' has $n - 1$ nodes, so by the induction hypothesis, G' has a topological ordering $v_1, v_2, \dots, v_{n-1}$.

Place v first, followed by the topological ordering of G' : $v, v_1, v_2, \dots, v_{n-1}$.

Since v has no incoming edges, all edges involving v are outgoing, so they go from v to nodes in G' , satisfying $i < j$. All other edges are within G' and respect its topological ordering. Therefore this is a topological ordering of G . ■

7.3 Algorithm and Visual Walkthrough

Algorithm: Topological Sort

```

Topological-Sort(G):
    Compute in-degree for all nodes
    Initialize queue Q with all nodes of in-degree 0
    Initialize empty list Order
    While Q is not empty
        Remove node v from Q
        Append v to Order
        For each edge (v, w) in G
            Decrement in-degree of w
            If in-degree of w becomes 0 then
                Add w to Q
            Endif
        Endfor
    Endwhile
    If Order contains all n nodes then
        Return Order
    Else
        Return "Graph contains a cycle"
  
```

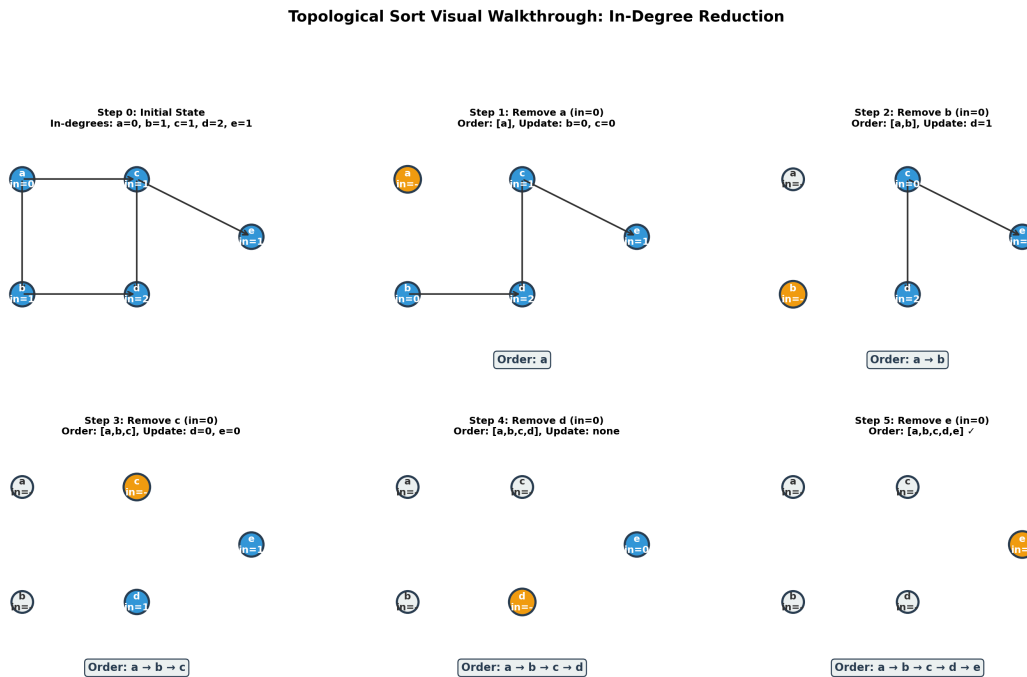


Figure 5: Topological sort using in-degree reduction. At each step, a node with in-degree 0 is removed (highlighted in orange), its outgoing edges are deleted, and in-degrees are updated. The final order $a \rightarrow b \rightarrow c \rightarrow d \rightarrow e$ is a valid topological ordering.

Complexity Analysis

Naive implementation: Scanning all nodes to find in-degree 0 at each step takes $O(n^2)$ time.

Efficient implementation (using a queue):

- Each node is removed from the queue exactly once: $O(n)$ queue operations.
- Each edge is examined exactly once (when its tail is removed): $O(m)$ work.
- In-degree updates are $O(1)$ per edge.

Total time: $O(m + n)$.

Chapter 8: Dijkstra's Algorithm

8.1 Overview and Invariant

Dijkstra's algorithm finds shortest paths from a source node s to all other nodes in a graph with **non-negative edge lengths**. It maintains a set S of **explored** nodes and a distance estimate $d(u)$ for each $u \in S$, which is the **true** shortest path distance.

Invariant

At each step, S contains nodes whose shortest path distance from s is known exactly. For each $u \in S$, $d(u)$ is the length of the shortest s - u path.

8.2 Algorithm and Working

Algorithm: Dijkstra's Algorithm

```
Dijkstra(G, l):
  Let S be the set of explored nodes
  For each  $u \in S$ , store distance  $d(u)$ 
  Initially  $S = \{s\}$  and  $d(s) = 0$ 
  While  $S \neq V$ 
    Select a node  $v \notin S$  with at least one edge from  $S$  for which
       $d'(v) = \min_{e=(u,v): u \in S} (d(u) + l_e)$ 
    is as small as possible
    Add  $v$  to  $S$  and define  $d(v) = d'(v)$ 
  EndWhile
```

Working:

1. **Initialization:** $S = \{s\}$, $d(s) = 0$. This is correct since the shortest path from s to itself has length 0.
2. **Main Loop:** At each iteration:
 - For each node $v \notin S$ with at least one edge from S , compute $d'(v) = \min_{(u,v): u \in S} [d(u) + l(u, v)]$. This represents the shortest path from s to v that stays

entirely within S except for the last edge.

- Select the node v with minimum $d'(v)$ among all such nodes.
- Add v to S and set $d(v) = d'(v)$.

Key Insight: Because all edge lengths are non-negative, the shortest path to v cannot pass through any unexplored node (such a path would be at least as long as $d'(v)$). Therefore $d'(v)$ is indeed the true shortest path distance.

8.3 Visual Walkthrough

Dijkstra's Algorithm Visual Walkthrough: Priority Queue Evolution

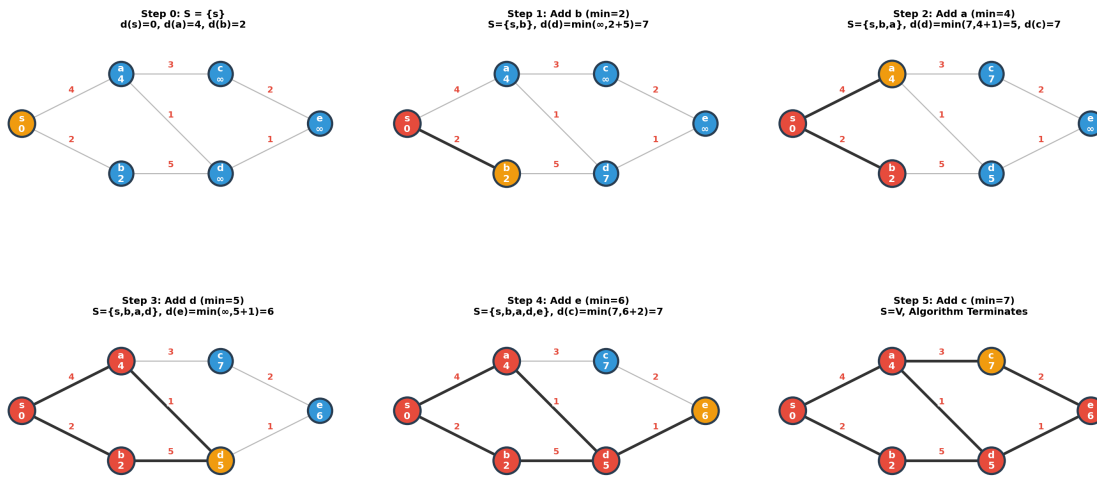


Figure 6: Dijkstra's algorithm execution. Red nodes are in S (distances known). Blue nodes are candidates with distance estimates shown. Orange node is the one being added in each step. Edge weights are shown in red. The algorithm greedily adds the node with minimum $d'(v)$ at each step.

Table 7 Dijkstra Execution Trace

Step	S	Candidates	$d'(v)$ Calculation	Min	Added
0	{s}	a, b	$d'(a) = 0 + 4 = 4, d'(b) = 0 + 2 = 2$	b (2)	b, $d(b) = 2$
1	{s, b}	a, d	$d'(a) = 4, d'(d) = 2 + 5 = 7$	a (4)	a, $d(a) = 4$
2	{s, b, a}	c, d	$d'(c) = 4 + 3 = 7, d'(d) = \min(7, 4 + 1) = 5$	d (5)	d, $d(d) = 5$
3	{s, b, a, d}	c, e	$d'(c) = 7, d'(e) = 5 + 1 = 6$	e (6)	e, $d(e) = 6$
4	{s, b, a, d, e}	c	$d'(c) = \min(7, 6 + 2) = 7$	c (7)	c, $d(c) = 7$
5	All nodes	—	—	—	Done

Resulting Shortest Path Tree:

- $s \rightarrow b$ (length 2)
- $s \rightarrow a$ (length 4)
- $s \rightarrow a \rightarrow d$ (length 5)
- $s \rightarrow a \rightarrow c$ (length 7)
- $s \rightarrow a \rightarrow d \rightarrow e$ (length 6)

8.4 Full Complexity Derivation

Theorem 4.15: Dijkstra Time Complexity

Using a priority queue, Dijkstra's Algorithm can be implemented to run in $O(m)$ time plus the time for n ExtractMin and m ChangeKey (DecreaseKey) operations.

Step-by-Step Derivation

Step 1: Node Extractions

Each node is added to S exactly once. Each addition requires one **ExtractMin** from the priority queue. Total ExtractMin operations: n .

Step 2: Key Updates

For each edge (u, v) , when u is added to S , we may need to update the key (distance estimate) of v . This happens at most **once per edge**, giving m **ChangeKey** (DecreaseKey) operations total.

Step 3: All Other Operations

Computing d' values, updating S , and initializing data structures are all $O(1)$ per edge/node. Total: $O(m + n)$.

Step 4: Total Abstract Running Time

$$T_{\text{total}} = O(m) + n \cdot T_{\text{ExtractMin}} + m \cdot T_{\text{ChangeKey}}$$

Step 5: Concrete Implementations

Priority Queue	ExtractMin	ChangeKey	Total Time
Binary Heap	$O(\log n)$	$O(\log n)$	$O(m \log n)$
Fibonacci Heap	$O(\log n)$ amortized	$O(1)$ amortized	$O(m + n \log n)$
Array (linear scan)	$O(n)$	$O(1)$	$O(n^2)$

For dense graphs where $m \approx n^2$, the array implementation achieves $O(n^2)$, matching the binary heap. For sparse graphs, the binary heap at $O(m \log n)$ is preferred. ■

8.5 Proof of Correctness

Theorem 4.14: Dijkstra's Distances Are Optimal

Consider the set S at any point in the algorithm's execution. For each $u \in S$, the path P_u is a shortest s - u path.

Proof (by Induction on $|S|$)

Base Case: $|S| = 1$, so $S = \{s\}$. The path P_s has length 0, which is optimal.

Inductive Step: Assume the theorem holds for $|S| = k$. Let v be the next node added to S , with:

$$d(v) = d'(v) = d(u) + l(u, v)$$

for some $u \in S$.

Suppose for contradiction that there exists a **shorter** s - v path P . Let y be the **first** node on P that is not in S , and let x be the node just before y on P (so $x \in S$).

The subpath from s to x has length at least $d(x)$ (by induction, $d(x)$ is the shortest distance). The edge (x, y) has length $l(x, y) \geq 0$. So the length of P up to y is at least:

$$d(x) + l(x, y) = d'(y)$$

Since all edge lengths are non-negative, the length of P from y to v is ≥ 0 . Therefore:

$$\text{length}(P) \geq d'(y) \geq d'(v)$$

where the last inequality follows because the algorithm chose v as the node with **minimum** d' value among all candidates. But $d'(v) = d(v)$, so:

$$\text{length}(P) \geq d(v)$$

This contradicts P being shorter than $d(v)$. Therefore P_v is a shortest s - v path. ■

Chapter 9: Minimum Spanning Trees

9.1 The MST Problem

Problem: Minimum Spanning Tree (MST)

Given a connected, undirected graph $G = (V, E)$ with edge costs c_e , find a subset $T \subseteq E$ such that (V, T) is connected and the total cost $\sum_{e \in T} c_e$ is minimized.

Theorem 4.16: The MST is a Tree

Let T be a minimum-cost solution to the network design problem. Then (V, T) is a tree.

Proof

Suppose (V, T) is connected but not a tree. Then T contains a cycle C . Remove any edge e from C . The graph remains connected (removing an edge from a cycle does not disconnect).

The new set $T' = T \setminus \{e\}$ has strictly lower cost since $c_e > 0$ (or even if $c_e = 0$, we can remove it without increasing cost). This contradicts the minimality of T .

Therefore (V, T) is acyclic and connected, hence a tree. ■

9.2 The Cut Property

Definition: Cut

A **cut** in a graph is a partition of V into two non-empty sets S and $V \setminus S$. An edge **crosses** the cut if it has one end in S and one end in $V \setminus S$.

Theorem 4.17: The Cut Property

Assume all edge costs are distinct. Let S be any non-empty proper subset of V , and let $e = (v, w)$ be the minimum-cost edge with one end in S and the other in $V \setminus S$. Then every minimum spanning tree contains e .

Proof

Suppose for contradiction that there exists an MST T that does not contain $e = (v, w)$, where $v \in S$ and $w \in V \setminus S$.

Since T is a spanning tree, there is a **unique path** P in T from v to w . This path must cross the cut $(S, V \setminus S)$ at some edge $e' = (x, y)$ where $x \in S$ and $y \in V \setminus S$.

Consider $T' = T \cup \{e\} \setminus \{e'\}$. Adding e to T creates a cycle (the path P plus e). Removing e' breaks the cycle and restores a tree.

Since e is the **minimum-cost** edge crossing the cut and all costs are distinct:

$$c_e < c_{e'}$$

Therefore:

$$\text{cost}(T') = \text{cost}(T) + c_e - c_{e'} < \text{cost}(T)$$

This contradicts T being minimum. Therefore **every** MST contains e . ■

9.3 The Cycle Property

Theorem 4.20: The Cycle Property

Assume all edge costs are distinct. Let C be any cycle in G , and let e be the most expensive edge in C . Then e does not belong to any minimum spanning tree.

Proof

Suppose for contradiction that an MST T contains $e = (v, w)$, the most expensive edge in cycle C .

Removing e from T disconnects T into two components (since T is a tree). The cycle C must contain another edge e' that reconnects these two components (since C is a cycle, there is another path from v to w).

Consider $T' = T \setminus \{e\} \cup \{e'\}$. Since e is the most expensive edge in C :

$$c_{e'} < c_e$$

Therefore:

$$\text{cost}(T') = \text{cost}(T) - c_e + c_{e'} < \text{cost}(T)$$

This contradicts T being minimum. Therefore e does not belong to any MST. ■

9.4 Prim's Algorithm

Prim's algorithm grows an MST from a starting node, adding the cheapest edge that connects the growing tree to a node outside it. It is analogous to Dijkstra's algorithm but with edge costs instead of path lengths.

Algorithm: Prim's Algorithm

```

Prim(G):
  Initialize S to be an empty set
  Initialize Q to be a priority queue
  For each node v in V
    Set a[v] = infinity
    Set edgeTo[v] = null
    Insert v into Q with key a[v]
  Endfor
  Choose an arbitrary starting node s
  Set a[s] = 0
  ChangeKey(Q, s, a[s])
  While Q is not empty
    Take a node u from Q using ExtractMin(Q)
    Add u to S
    For each edge (u, v) incident to u
      If v is not in S then
        If cost(u, v) < a[v] then
          Set a[v] = cost(u, v)
          Set edgeTo[v] = u
          ChangeKey(Q, v, a[v])
        Endif
      Endif
    Endfor
  Endwhile
  Return edgeTo

```

Working: Each node v has a key $a[v]$ representing the minimum cost to connect v to the growing tree S . When we extract v with minimum $a[v]$, we add the cheapest possible connection from S to $V \setminus S$.

Prim's Algorithm Visual Walkthrough: MST Growth

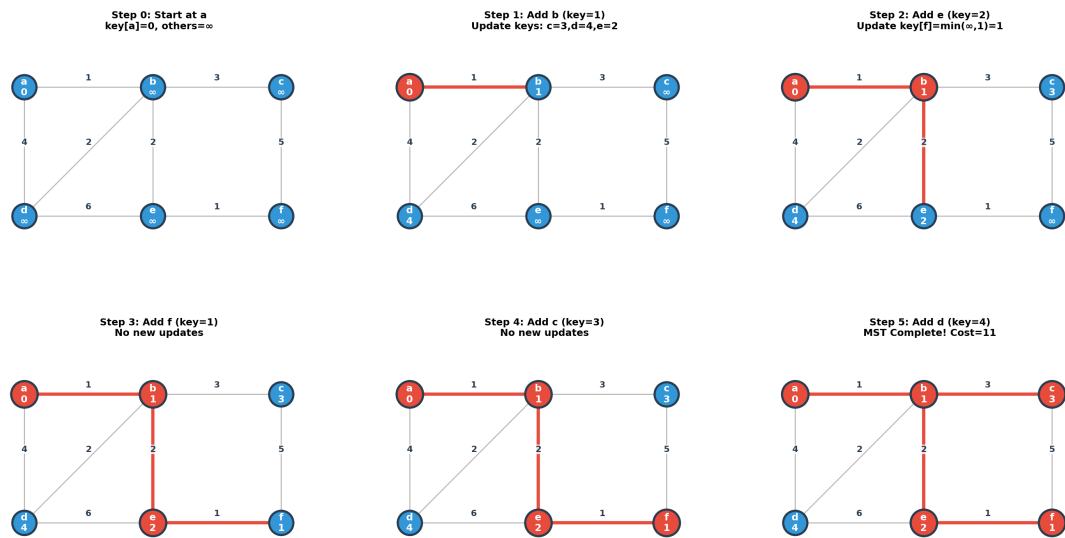


Figure 7: Prim's algorithm starting from node *a*. Red nodes are in *S*; keys show minimum connection cost. At each step, the node with minimum key is extracted and its neighbors' keys are updated. Total MST cost = 11.

Theorem 4.19: Prim's Algorithm Produces an MST

*Prim's Algorithm produces a minimum spanning tree of *G*.*

Proof

Prim's algorithm maintains a set $S \subseteq V$ on which a partial spanning tree has been constructed. In each iteration, it adds a node v and edge e that minimize:

$$\min_{e=(u,v):u \in S} c_e$$

By definition, e is the **cheapest edge** with one end in S and the other in $V \setminus S$. By the Cut Property (Theorem 4.17), e belongs to **every** minimum spanning tree.

Since Prim's only adds edges that belong to every MST, and it produces a spanning tree (connects all nodes, acyclic), the result is a minimum spanning tree. ■

Theorem 4.22: Prim's Complexity

Using a priority queue, Prim's Algorithm runs in $O(m)$ time plus the time for n ExtractMin and m ChangeKey operations.

Step-by-Step Derivation**Step 1: Initialization**

Setting up keys and inserting into queue: $O(n)$.

Step 2: Main Loop

n iterations. Each performs one ExtractMin: total n **ExtractMin** operations.

Step 3: Edge Processing

Each edge (u, v) is examined at most twice (once from each endpoint). When examining (u, v) , if $v \notin S$, we may perform a ChangeKey: at most m **ChangeKey** operations total.

Step 4: Total

$$T_{\text{total}} = O(m) + n \cdot T_{\text{ExtractMin}} + m \cdot T_{\text{ChangeKey}}$$

With a binary heap: $O(m \log n)$. With a Fibonacci heap: $O(m + n \log n)$. ■

9.5 Reverse-Delete Algorithm

Algorithm: Reverse-Delete

```
Reverse-Delete(G):
    Sort all edges in decreasing order of cost
    T = E (all edges)
    For each edge e in decreasing order
        If removing e does not disconnect (V, T) then
            Remove e from T
        Endif
    Endfor
    Return T
```

Working: The algorithm processes edges from most expensive to least expensive. For each edge, if removing it does **not** disconnect the graph, it is removed. Otherwise, it is kept because it is a bridge.

Theorem 4.21: Reverse-Delete Produces an MST

The Reverse-Delete Algorithm produces a minimum spanning tree of G .

Proof

Consider any edge $e = (v, w)$ removed by Reverse-Delete. At the time e is removed, it lies on a cycle C (otherwise removing it would disconnect the graph).

Since edges are processed in **decreasing** order, e is the **most expensive** edge on C .

By the Cycle Property (Theorem 4.20), the most expensive edge on any cycle is **not** in any MST. Therefore e does not belong to any minimum spanning tree.

Since Reverse-Delete only removes edges that are not in any MST, and the remaining edges form a spanning tree, the result is a minimum spanning tree. ■

Summary: Key Facts and Complexity Table

Table 8 Algorithm Complexities Summary

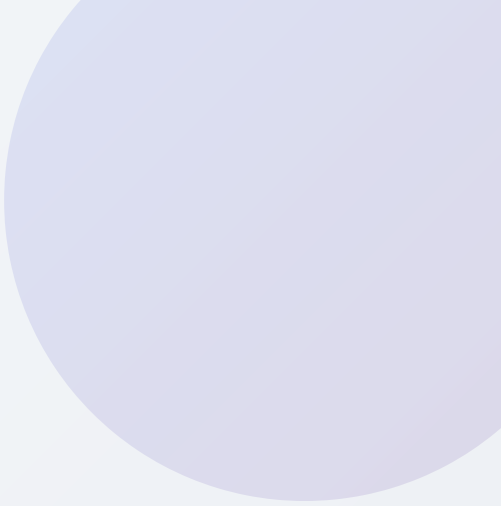
Algorithm	Time Complexity (Adj. List)	Space	Key Data Structure
BFS	$O(m + n)$	$O(m + n)$	Queue (FIFO)
DFS	$O(m + n)$	$O(m + n)$	Stack (LIFO / recursion)
Dijkstra (binary heap)	$O(m \log n)$	$O(m + n)$	Priority Queue
Dijkstra (Fibonacci heap)	$O(m + n \log n)$	$O(m + n)$	Priority Queue
Prim (binary heap)	$O(m \log n)$	$O(m + n)$	Priority Queue
Prim (Fibonacci heap)	$O(m + n \log n)$	$O(m + n)$	Priority Queue
Topological Sort	$O(m + n)$	$O(m + n)$	Queue (in-degree tracking)
Bipartite Test	$O(m + n)$	$O(m + n)$	BFS layers + coloring
Strong Connectivity Test	$O(m + n)$	$O(m + n)$	Two BFS/DFS passes

Table 9 Key Theorems Summary

Theorem	Statement	Significance
3.1	n -node tree has $n - 1$ edges	Characterizes trees
3.2	Any two of {connected, acyclic, $n - 1$ edges} implies the third	Equivalent tree definitions
3.3	BFS layers contain nodes at exact distance j from s	BFS correctness for shortest paths
3.4	BFS tree edges connect nodes at most 1 layer apart	Structure of BFS trees
3.5	Component exploration finds exact connected component	Correctness of connectivity algorithms
3.6	DFS marks only descendants during a recursive call	DFS tree structure
3.7	Non-tree edges in DFS connect ancestor-descendant	Fundamental DFS property
3.8	Connected components are identical or disjoint	Partition property
3.9	$\sum \deg(v) = 2m$	Handshaking lemma
3.10	Adjacency matrix: $O(n^2)$; Adj. list: $O(m + n)$	Space trade-offs
3.11	BFS runs in $O(m + n)$ time	Linear-time traversal
3.13	DFS runs in $O(m + n)$ time	Linear-time traversal
3.14	Bipartite graphs have no odd cycles	Necessary condition
3.15	Same-layer edge $\Leftrightarrow$ odd cycle $\Leftrightarrow$ non-bipartite	BFS bipartite test is correct
3.16	Mutual reachability is transitive	SCCs form equivalence classes
3.17	Strong components are identical or disjoint	Partition into SCCs
3.18	Topological ordering $\Rightarrow$ DAG	Necessary condition
3.19	Every DAG has a node with in-degree 0	Existence for topological sort

3.20	DAG $\Rightarrow$ has topological ordering	Sufficient condition
4.14	Dijkstra's distances are optimal when added to S	Dijkstra correctness
4.15	Dijkstra with priority queue: $O(m) + n \text{ ExtractMin} + m \text{ ChangeKey}$	Complexity framework
4.16	MST solution is always a tree	Justifies "tree" terminology
4.17	Cut Property: minimum crossing edge is in every MST	Foundation of MST algorithms
4.19	Prim's algorithm produces an MST	Prim's correctness
4.21	Reverse-Delete produces an MST	Reverse-Delete correctness

Greedy & Dynamic Programming Algorithms



Complete University Study Notes with Proofs, Complexity Analysis, and Visual Walkthroughs

Based on Kleinberg & Tardos, CLRS, and Standard Course Curriculum

Comprehensive Reference for University Examinations



Table of Contents

Part I: Greedy Algorithms

1. Interval Scheduling: The Greedy Algorithm Stays Ahead
 - 1.1 Algorithm and Working
 - 1.2 Time Complexity Analysis
 - 1.3 Visual Walkthrough
 - 1.4 Theorem 4.1: Compatible Set
 - 1.5 Theorem 4.2: Stays Ahead Property
 - 1.6 Theorem 4.3: Optimality Proof
2. Interval Partitioning (Depth/Classroom Scheduling)
3. Fractional Knapsack Problem
4. 0-1 Knapsack: Why Greedy Fails
5. Elements of the Greedy Strategy (CLRS)

Part II: Dynamic Programming

6. Weighted Interval Scheduling
 - 6.1 Problem Definition and $p(j)$
 - 6.2 The Recurrence Relation
 - 6.3 Naive Recursive Algorithm
 - 6.4 Memoization (M-Compute-Opt)
 - 6.5 Iterative Bottom-Up
 - 6.6 Finding the Solution Set
7. Subset Sum Problem
8. 0-1 Knapsack (Dynamic Programming)
9. Principles of Dynamic Programming

Summary: Algorithm Comparison

PART I

Greedy Algorithms

1. Interval Scheduling: The Greedy Algorithm Stays Ahead

1.1 Problem Definition

Given a set of n requests, where each request i corresponds to a time interval with start time $s(i)$ and finish time $f(i)$, the **Interval Scheduling Problem** asks us to select a **maximum-size compatible subset** of intervals. Two intervals are **compatible** if they do not overlap in time.

Key Difference from Weighted Version: In the standard interval scheduling problem, all intervals have equal value (implicitly 1), so we maximize the *count* of intervals. In the weighted version (covered in Part II), each interval has a value v_i and we maximize total value.

1.2 The Greedy Algorithm

Algorithm: Greedy Interval Scheduling

```
Sort intervals by finish time in nondecreasing order
Let  $I_1, I_2, \dots, I_n$  be the sorted intervals (so  $f(i) \leq f(j)$  when  $i < j$ )
 $A$  = empty set
For  $j = 1, 2, \dots, n$ 
    If  $I_j$  is compatible with all intervals in  $A$  then
        Add  $I_j$  to  $A$ 
    Endif
Endfor
Return  $A$ 
```

Compatibility Check: Since intervals in A are sorted by finish time and all mutually compatible, we only need to check if the start time of I_j is at least the finish time of the *last* interval added to A :

$$s(j) \geq f(\text{last}(A)) \quad (1.1)$$

1.3 Working of the Algorithm

The algorithm operates in two phases:

1. **Sorting Phase:** All n intervals are sorted by finish time in ascending order. This takes $O(n \log n)$ time.
2. **Selection Phase:** We scan through the sorted intervals exactly once. We always select the first interval (it has the earliest finish time). For each subsequent interval, we check if it starts at or after the finish time of the most recently selected interval. If yes, we select it; otherwise, we skip it.

Why this greedy choice works: By always selecting the interval with the *earliest finish time* among all compatible options, we leave the maximum possible remaining time for future intervals. This greedy choice property is what makes the algorithm optimal.

1.4 Time Complexity Analysis (Full Derivation)

Theorem: The greedy interval scheduling algorithm runs in $O(n \log n)$ time.

Proof of Time Complexity:

Phase 1 — Sorting: Using an efficient comparison-based sorting algorithm (Merge Sort or Heap Sort), sorting n intervals by finish time requires $O(n \log n)$ comparisons in the worst case. Each comparison of finish times takes $O(1)$ time. Therefore:

$$T_{\text{sort}}(n) = O(n \log n)$$

Phase 2 — Selection: After sorting, the selection phase makes a single pass through the n intervals. For each interval I_j , the compatibility check compares $s(j)$ with the finish time of the last selected interval:

$$T_{\text{check}}(j) = O(1)$$

Since this is done for all n intervals:

$$T_{\text{selection}}(n) = \sum_{j=1}^n O(1) = O(n)$$

Total Time Complexity:

$$T(n) = T_{\text{sort}}(n) + T_{\text{selection}}(n) = O(n \log n) + O(n) = O(n \log n)$$

The $O(n \log n)$ term dominates, giving the overall complexity.

Space Complexity: $O(n)$ to store the sorted intervals and the selected set A.

1.5 Visual Walkthrough with Example

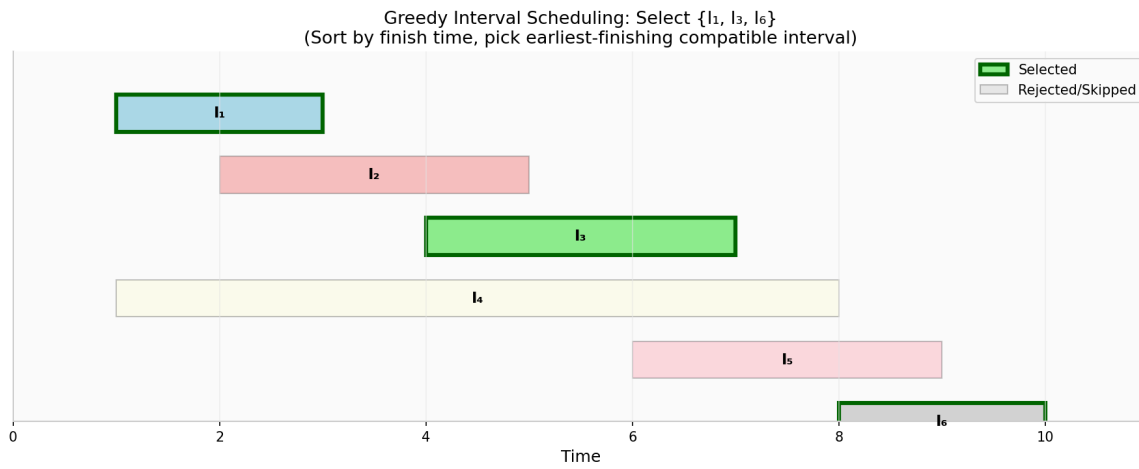


Figure 1.1 Greedy Interval Scheduling execution. Top row shows all intervals; green-bordered intervals are selected by the greedy algorithm (earliest finish time first).

Detailed Step-by-Step Trace:

Table 1.1 Greedy Selection Trace for Example Instance

Step	Consider	Start	Finish	Last in A	Compatible?	Action	A after
1	I_1	1	3	—	Yes (A empty)	Add I_1	$\{I_1\}$
2	I_2	2	5	I_1 (finishes 3)	No ($2 < 3$)	Skip	$\{I_1\}$
3	I_3	4	7	I_1 (finishes 3)	Yes ($4 \geq 3$)	Add I_3	$\{I_1, I_3\}$
4	I_5	6	9	I_3 (finishes 7)	No ($6 < 7$)	Skip	$\{I_1, I_3\}$
5	I_6	8	10	I_3 (finishes 7)	Yes ($8 \geq 7$)	Add I_6	$\{I_1, I_3, I_6\}$
6	I_4	1	8	I_6 (finishes 10)	No ($1 < 10$)	Skip	$\{I_1, I_3, I_6\}$

Result: $A = \{I_1, I_3, I_6\}$ with $|A| = 3$ intervals. **Verification of optimality:** Intervals I_1, I_2, I_4 all overlap at time 2.5-3, so at most one can be chosen. Similarly I_3 and I_5 overlap. Thus no solution can contain more than 3 intervals.

1.6 Theorem 4.1: The Greedy Set is Compatible

Theorem 4.1

The set A produced by the greedy algorithm is a compatible set of requests (no two intervals in A overlap).

Proof:

By construction, when the algorithm considers interval I_j , it adds I_j to A only if $s(j) \geq f(\text{last}(A))$, where $\text{last}(A)$ is the interval most recently added to A . Since intervals are processed in order of nondecreasing finish time, all intervals already in A finish no later than $\text{last}(A)$. Therefore, I_j starts at or after all intervals in A finish, meaning I_j is compatible with *every* interval in A . By induction on the number of intervals added, all pairs in A are compatible. ■

1.7 Theorem 4.2: The "Stays Ahead" Property

Theorem 4.2

Let $A = \{i_1, i_2, \dots, i_k\}$ be the intervals selected by the greedy algorithm in order. Let $O = \{j_1, j_2, \dots, j_m\}$ be any optimal solution, also ordered by finish time. Then for all indices $r \leq k$:

$$f(i_r) \leq f(j_r) \quad (1.2)$$

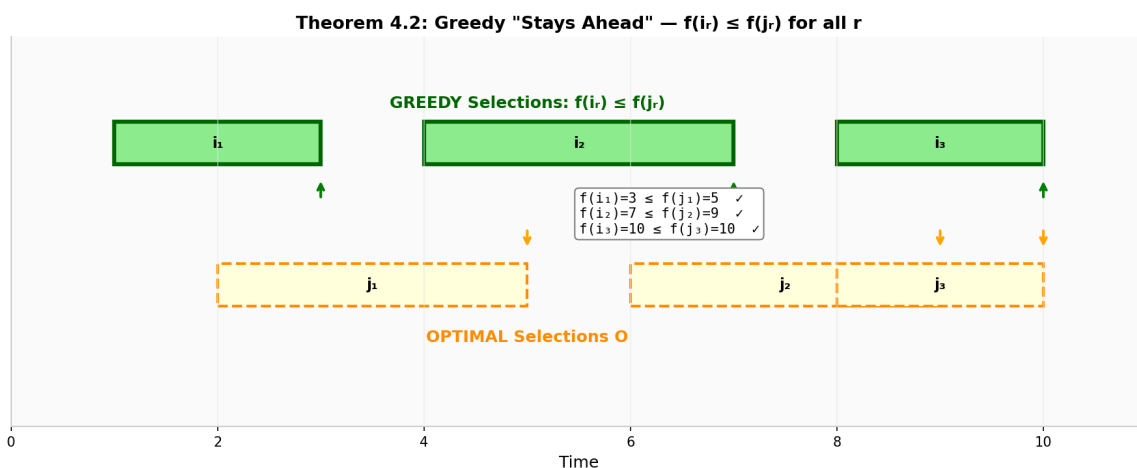


Figure 1.2 The "Stays Ahead" property: each greedy interval finishes no later than the corresponding optimal interval.

Proof (by induction on r):

Base Case ($r = 1$): The greedy algorithm starts by selecting the interval with the *minimum finish time* among all intervals. Therefore, $f(i_1) \leq f(j_1)$ for any interval j_1 in any optimal solution. The base case holds. ✓

Inductive Hypothesis: Assume the statement is true for $r - 1$, i.e., $f(i_{r-1}) \leq f(j_{r-1})$.

Inductive Step (prove for r):

Since O consists of compatible intervals, we know that:

$$f(j_{r-1}) \leq s(j_r)$$

This is because j_r must start after j_{r-1} finishes (they are compatible).

From the inductive hypothesis:

$$f(i_{r-1}) \leq f(j_{r-1})$$

Combining these two inequalities by transitivity:

$$f(i_{r-1}) \leq f(j_{r-1}) \leq s(j_r)$$

Therefore:

$$f(i_{r-1}) \leq s(j_r)$$

This means that interval j_r starts after interval i_{r-1} finishes. So when the greedy algorithm was selecting its r -th interval, j_r was still available in the set R of intervals that don't overlap with any previously selected interval.

The greedy algorithm always selects the available interval with the **smallest finish time**. Since j_r was available at that time, the greedy algorithm's choice i_r must have finish time no later than j_r :

$$f(i_r) \leq f(j_r)$$

This completes the induction step. ■

1.8 Theorem 4.3: The Greedy Algorithm is Optimal

Theorem 4.3

The greedy algorithm returns an optimal set A .

Proof (by contradiction):

Assume for contradiction that A is not optimal. Then there exists an optimal set O with strictly more intervals than A , i.e., $m > k$ where $|A| = k$ and $|O| = m$.

Apply Theorem 4.2 with $r = k$:

$$f(i_k) \leq f(j_k)$$

Since $m > k$, there exists an interval j_{k+1} in O . Because O is a set of compatible intervals:

$$f(j_k) \leq s(j_{k+1})$$

Combining with the above by transitivity:

$$f(i_k) \leq f(j_k) \leq s(j_{k+1})$$

This means j_{k+1} starts after i_k finishes, so j_{k+1} is compatible with all intervals in $A = \{i_1, i_2, \dots, i_k\}$. Therefore, j_{k+1} should still be in the set R of available intervals when the greedy algorithm finished selecting i_k .

But the greedy algorithm stopped after selecting k intervals, which only happens when R is empty (no more compatible intervals available). This is a **contradiction**.

Therefore, our assumption that A is not optimal must be false. The greedy algorithm returns an optimal set. ■

Summary: Interval Scheduling Greedy Algorithm

- **Algorithm:** Sort by finish time, then greedily pick earliest-finishing compatible interval.
- **Time Complexity:** $O(n \log n)$ — dominated by sorting.
- **Space Complexity:** $O(n)$.
- **Proof Technique:** "Stays Ahead" — greedy always finishes no later than optimal.
- **Key Theorems:** Compatible set (4.1), Stays Ahead (4.2), Optimality (4.3).

2. Interval Partitioning (Depth / Classroom Scheduling)

2.1 Problem Definition

The **Interval Partitioning Problem** (also called Interval Coloring or Classroom Scheduling) asks: given a set of intervals, assign a label (color/resource) to each interval such that no two overlapping intervals share the same label. The goal is to minimize the number of labels used.

Definition: Depth of an Interval Set

The **depth** of a set of intervals is the maximum number of intervals that pass over any single point on the time-line. This is a lower bound on the number of labels needed.

2.2 The Greedy Algorithm

Algorithm: Interval Partitioning (Greedy by Start Time)

```
Sort the intervals by their start times, breaking ties arbitrarily
Let  $I_1, I_2, \dots, I_n$  denote the intervals in this order
For  $j = 1, 2, \dots, n$ 
    For each interval  $I_i$  that precedes  $I_j$  in sorted order and overlaps
    it
        Exclude the label of  $I_i$  from consideration for  $I_j$ 
    Endfor
    If there is any label from  $\{1, 2, \dots, d\}$  that has not been excluded
    then
        Assign a non-excluded label to  $I_j$  (smallest available)
    Else
        Leave  $I_j$  unlabeled (need a new label)
    Endif
Endfor
```

2.3 Working of the Algorithm

By processing intervals in order of increasing start time, when we reach interval I_j , we have complete information about all intervals that could possibly conflict with it (any interval that starts later cannot overlap with I_j 's start). The algorithm always tries to reuse an existing label before introducing a new one.

2.4 Time Complexity Analysis (Full Derivation)

Theorem: The interval partitioning algorithm runs in $O(n^2)$ time (naive) or $O(n \log n)$ with a priority queue.

Naive Analysis:

Phase 1 — Sorting: $O(n \log n)$ by start time.

Phase 2 — Label Assignment: For each of the n intervals, in the worst case we may need to check all $O(n)$ preceding intervals to find overlaps and excluded labels. Checking compatibility of two intervals takes $O(1)$ time. Therefore:

$$T_{\text{assign}}(n) = \sum_{j=1}^n O(j) = O(n^2)$$

Total (naive):

$$T(n) = O(n \log n) + O(n^2) = O(n^2)$$

Optimized with Priority Queue: Maintain a min-heap of labels with their "release times" (when the interval using that label finishes). For each new interval, extract all labels whose release time $\leq$ current start time (they're now free). Assign the smallest available label. Heap operations take $O(\log d)$ where d is the number of labels. Since $d \leq n$:

$$T_{\text{optimized}}(n) = O(n \log n) + O(n \log n) = O(n \log n)$$

2.5 Visual Walkthrough

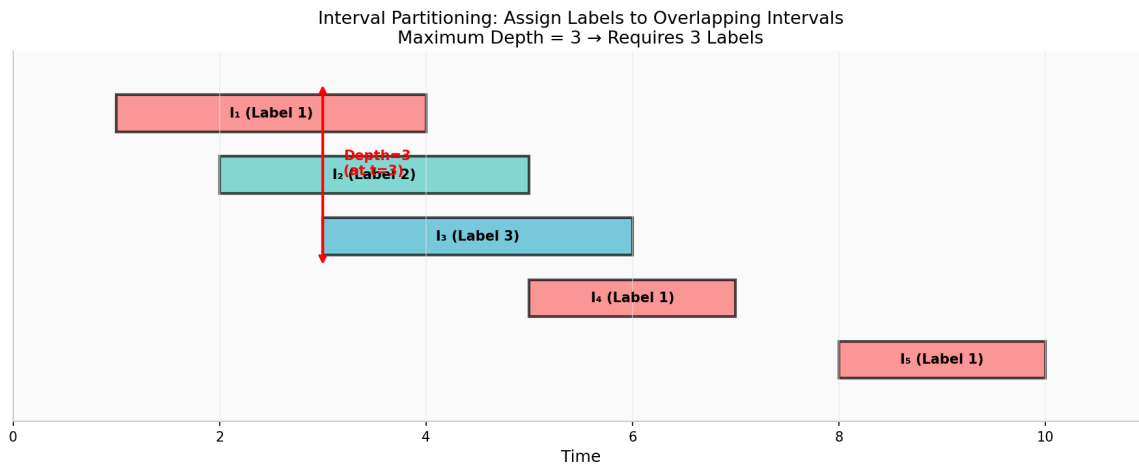


Figure 2.1 Interval partitioning: maximum depth = 3 requires 3 labels. The greedy algorithm processes by start time and assigns the smallest available label.

2.6 Proof of Correctness

Theorem

The greedy interval partitioning algorithm uses exactly d labels, where d is the depth of the interval set. This is optimal.

Proof:

Upper bound: We show the algorithm never needs more than d labels. When processing interval I_j , at most $d - 1$ intervals can overlap with it at any point (by definition of depth). These are the only intervals that could exclude labels. Since there are d labels total, at least one label is always available.

Lower bound: At the point where depth d is achieved, d intervals simultaneously overlap. Each needs a distinct label. Thus any valid labeling needs at least d labels.

Since the algorithm uses at most d labels and at least d labels are necessary, the algorithm is optimal. ■

3. Fractional Knapsack Problem

3.1 Problem Definition

Given n items, where item i has weight w_i and value v_i , and a knapsack with capacity W , the **Fractional Knapsack Problem** asks us to select fractions of items to maximize total value without exceeding weight capacity. Unlike 0-1 knapsack, we can take any fraction $x_i \in [0, 1]$ of each item.

3.2 The Greedy Algorithm

Algorithm: Fractional Knapsack (Greedy by Density)

```

Sort the items by value per pound ( $v_i / w_i$ ) in decreasing order
Let  $a_1, a_2, \dots, a_n$  denote the items in this order
Let  $W$  be the knapsack capacity
Set  $total\_value = 0$ 
Set  $remaining\_capacity = W$ 
For  $j = 1, 2, \dots, n$ 
    If  $remaining\_capacity = 0$  then Break
    If  $w_j \leq remaining\_capacity$  then
        Take all of item  $a_j$ 
         $total\_value = total\_value + v_j$ 
         $remaining\_capacity = remaining\_capacity - w_j$ 
    Else
        Take fraction =  $remaining\_capacity / w_j$  of item  $a_j$ 
         $total\_value = total\_value + fraction * v_j$ 
         $remaining\_capacity = 0$ 
    Endif
Endfor
Return  $total\_value$ 

```

3.3 Working of the Algorithm

1. **Sorting Phase:** Compute the value-to-weight ratio (density) $r_i = v_i / w_i$ for each item. Sort items in decreasing order of this ratio. This takes $O(n \log n)$ time.
2. **Greedy Selection:** Process items in sorted order. For each item, if it fits entirely, take all of it. If it doesn't fit, take exactly enough to fill the remaining capacity.

Why this works: The greedy strategy ensures every unit of capacity is filled with the maximum possible value density. Since we can take fractions, we never "waste" capacity.

3.4 Time Complexity Analysis (Full Derivation)

Theorem: The fractional knapsack algorithm runs in $O(n \log n)$ time.

Phase 1 — Computing Ratios: For each of n items, compute v_i/w_i . Division of two numbers takes $O(1)$ time.

$$T_{\text{ratios}}(n) = \sum_{i=1}^n O(1) = O(n)$$

Phase 2 — Sorting by Ratio: Sort n items by their density values. Using Merge Sort or Heap Sort:

$$T_{\text{sort}}(n) = O(n \log n)$$

Phase 3 — Selection: Single pass through n items. Each item requires $O(1)$ work (comparison, arithmetic, update):

$$T_{\text{selection}}(n) = \sum_{j=1}^n O(1) = O(n)$$

Total Time Complexity:

$$T(n) = O(n) + O(n \log n) + O(n) = O(n \log n)$$

Space Complexity: $O(n)$ to store items, their ratios, and the sorted order.

3.5 Visual Walkthrough

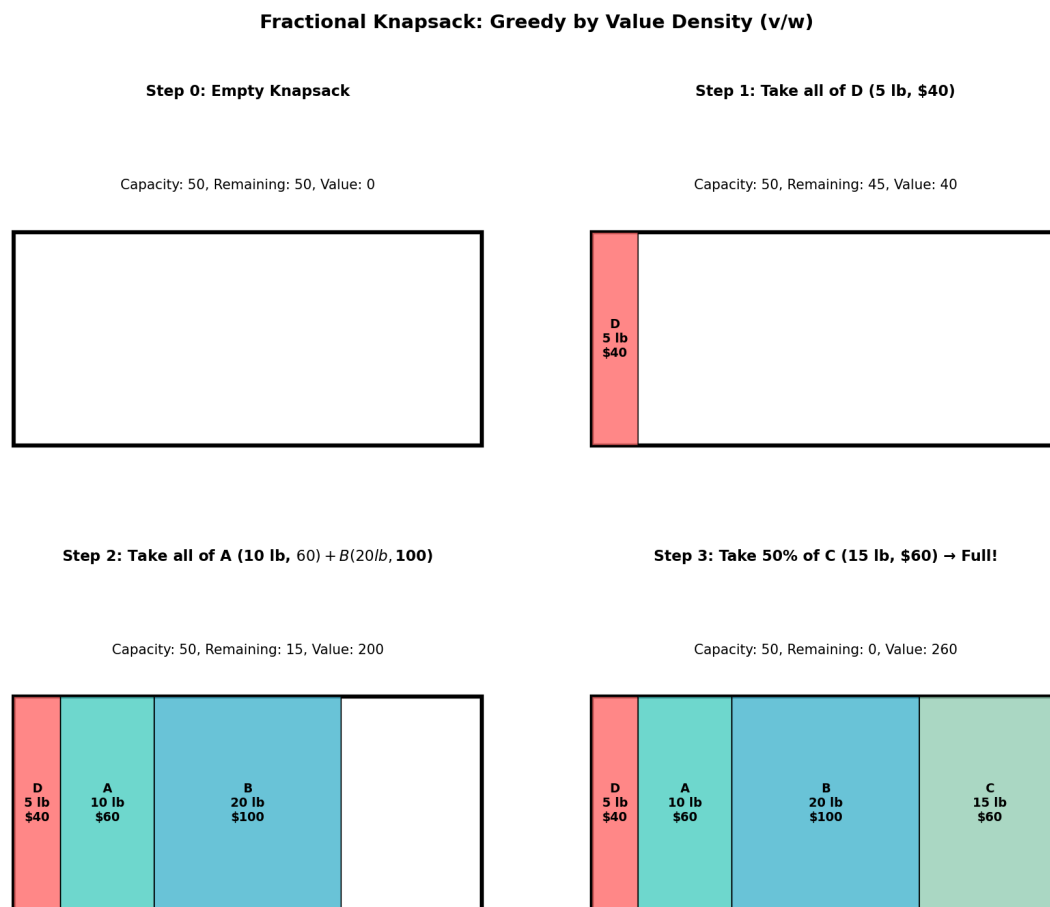


Figure 3.1 Fractional knapsack execution: items sorted by value density (D: $8/lb$, A: $6/lb$, B: $5/lb$, C: $4/lb$). The knapsack is filled greedily, taking fractions of the last item if needed.

Detailed Trace:

Table 3.1 Fractional Knapsack Execution Trace ($W = 50$)

Step	Item	Ratio	Action	Weight Taken	Value Added	Remaining	Total Value
1	D	8.0	Take all	5	\$40	45	\$40
2	A	6.0	Take all	10	\$60	35	\$100
3	B	5.0	Take all	20	\$100	15	\$200
4	C	4.0	Take $15/30 = 0.5$	15	\$60	0	\$260

Proof (Exchange Argument):

Let the items be sorted by value density: $v_1/w_1 \geq v_2/w_2 \geq \dots \geq v_n/w_n$.

Let $X = (x_1, x_2, \dots, x_n)$ be the solution produced by the greedy algorithm, where $x_i \in [0, 1]$ is the fraction of item i taken.

Let $Y = (y_1, y_2, \dots, y_n)$ be any optimal solution. Suppose for contradiction that Y is strictly better than X .

Then there must exist some item i where $y_i > x_i$ and some item j where $y_j < x_j$, with the density of item i less than the density of item j (i.e., $i > j$ in sorted order, meaning $v_i/w_i < v_j/w_j$).

Consider transferring a small amount δ of weight from item i to item j in solution Y :

- Remove δ weight of item i : lose value $\delta \cdot (v_i/w_i)$
- Add δ weight of item j : gain value $\delta \cdot (v_j/w_j)$

Since $v_j/w_j > v_i/w_i$, the net change is:

$$\delta \cdot \left(\frac{v_j}{w_j} - \frac{v_i}{w_i} \right) > 0$$

This means Y was not optimal — we can improve it by shifting weight from lower-density to higher-density items. Repeating this argument, any optimal solution must allocate weight in order of decreasing density, which is exactly what the greedy algorithm does. ■

4. 0-1 Knapsack: Why Greedy Fails

4.1 Problem Definition

In the **0-1 Knapsack Problem**, each item must be taken whole or not at all. The thief cannot take fractional amounts. This seemingly small change makes the greedy strategy fail.

4.2 The Counterexample

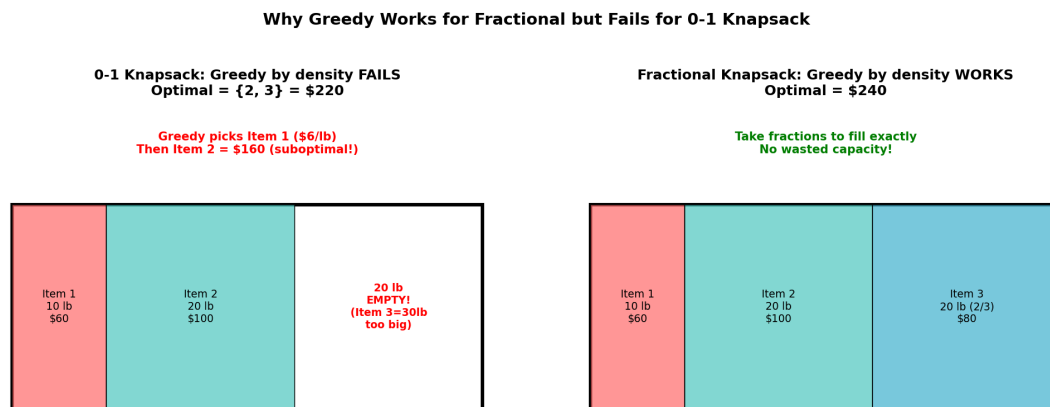


Figure 4.1 0-1 knapsack (left) vs fractional knapsack (right) with the same instance. Greedy by density fails for 0-1 because taking Item 1 first leaves empty space that cannot be efficiently filled.

Table 4.1 0-1 Knapsack Counterexample: Capacity $W = 50$

Item	Weight	Value	Density (\$/lb)
1	10	\$60	\$6.00
2	20	\$100	\$5.00
3	30	\$120	\$4.00

Greedy by density: Take Item 1 (6/lb), then Item 2 (5/lb), then cannot fit Item 3 (30 lb > 20 lb remaining). Total value = **\$160**.

Optimal 0-1 solution: Take Items 2 and 3 (20 + 30 = 50 lb). Total value = **\$220**.

Key Insight: Taking the highest-density item first may leave the knapsack with unused capacity that cannot be efficiently filled by remaining items. The "empty space" lowers the effective value per pound of the load.

4.3 Why Greedy Works for Fractional but Not 0-1

Table 4.2 Fractional vs 0-1 Knapsack

Aspect	Fractional Knapsack	0-1 Knapsack
Decision	Take any fraction $x_i \in [0, 1]$	Binary: take or leave
Greedy works?	Yes — always fill with highest density	No — may leave gaps
Empty space	Never — fill exactly to W with fractions	Can occur — items don't fit perfectly
Solution approach	Greedy $O(n \log n)$	Dynamic Programming $O(nW)$
Why empty space matters	Not applicable	Taking Item 1 (10 lb) leaves 20 lb gap. Item 3 (30 lb) doesn't fit, so space is wasted.

Mathematical Explanation: In 0-1 knapsack, when deciding to include item i , we must *compare* the solution to the subproblem *including* the item with the solution to the subproblem *excluding* the item *before* making the choice. This gives rise to **many overlapping subproblems** — a hallmark of dynamic programming. The greedy algorithm makes a local choice without this global comparison, which can be suboptimal.

5. Elements of the Greedy Strategy (CLRS Framework)

5.1 Core Definition

A **greedy algorithm** obtains an optimal solution to a problem by making a **sequence of choices**. At each decision point, the algorithm makes the choice that **seems best at the moment** (locally optimal). This heuristic does not always produce an optimal solution, but sometimes it does.

5.2 Two Methodologies for Developing Greedy Algorithms

Method 1: The Dynamic-Programming-First Approach

Six-step process:

1. Determine the **optimal substructure** of the problem.
2. Develop a **recursive solution**.
3. Show that if you make the **greedy choice**, then only **one subproblem** remains.
4. **Prove** that it is always safe to make the greedy choice.
5. Develop a **recursive algorithm** that implements the greedy strategy.
6. Convert the recursive algorithm to an **iterative algorithm**.

Used in Section 15.1 for the activity-selection problem.

Method 2: The Direct/Greedy-First Approach

Three-step process:

1. **Cast the optimization problem** as one in which you make a choice and are left with **one subproblem** to solve.
2. **Prove** that there is always an **optimal solution** to the original problem that makes the greedy choice, so that the greedy choice is **always safe**.
3. **Demonstrate optimal substructure** by showing that, having made the greedy choice, what remains is a subproblem with the property that if you combine an **optimal solution to the subproblem** with the **greedy choice**, you arrive at an **optimal solution to the original problem**.

This is the more typical approach.

Important Caveat

Beneath every greedy algorithm, there is **almost always** a more cumbersome dynamic-programming solution.

5.3 The Two Key Ingredients

No universal method works all the time. The two key ingredients for greedy algorithms are:

1. **Greedy-choice property:** You can assemble a globally optimal solution by making locally optimal (greedy) choices. The choice depends on choices so far, but **cannot depend on future choices or solutions to subproblems**.
2. **Optimal substructure:** An optimal solution to the problem contains within it optimal solutions to subproblems.

5.4 Greedy vs Dynamic Programming

Table 5.1 Greedy Algorithms vs Dynamic Programming

Aspect	Dynamic Programming	Greedy Algorithm
Choice dependency	Choice usually depends on solutions to subproblems	Choice depends on choices so far , not future subproblems
Order of solving	Typically bottom-up	Typically top-down
Memoization	Can solve top-down with memoization	Makes choice before solving any subproblems
Number of subproblems	May need to solve many subproblems	Only one subproblem remains after greedy choice
Proof requirement	Optimal substructure + overlapping subproblems	Greedy-choice property + optimal substructure
Efficiency	Generally slower (polynomial but higher degree)	Often faster (can be linear or linearithmic)

PART II

Dynamic Programming

6. Weighted Interval Scheduling

6.1 Problem Definition

The **Weighted Interval Scheduling Problem** is a strictly more general version of interval scheduling where each interval i has a **value/weight** v_i , and we want to select a compatible subset with **maximum total value** (not just maximum count). The standard problem is the special case where all $v_i = 1$.

Why Greedy Fails for Weighted Version

Most greedy algorithms that work for standard interval scheduling **do not** solve the weighted version optimally. Even the previously optimal greedy approach (repeatedly choosing the interval that ends earliest) **fails** in the weighted setting. Consider: three intervals with values 1, 3, and 1. Greedy by earliest finish picks intervals 1 and 3 (total value 2), but the optimal is interval 2 alone (value 3).

6.2 Notation and Setup

- n requests labeled $1, 2, \dots, n$
- Each request i : start time s_i , finish time f_i , value v_i
- Requests sorted by **nondecreasing finish time**: $f_1 \leq f_2 \leq \dots \leq f_n$
- **Compatibility**: Two intervals are compatible if they do not overlap
- **Goal**: Select subset $S \subseteq \{1, \dots, n\}$ of mutually compatible intervals to maximize $\sum_{i \in S} v_i$

6.3 The $p(j)$ Function

Definition: $p(j)$

For interval j , $p(j)$ is the **largest index** $i < j$ such that intervals i and j are **disjoint** (non-overlapping). If no such interval exists, $p(j) = 0$.

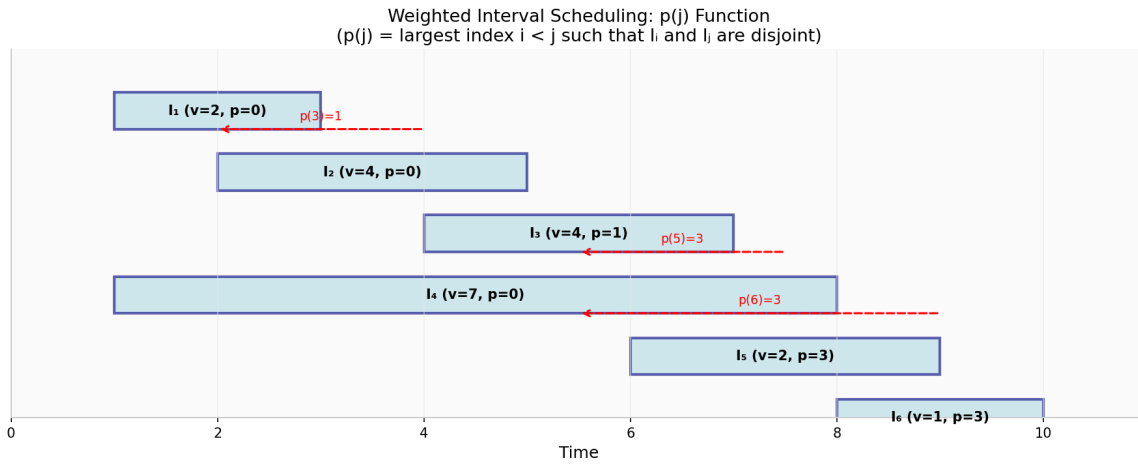


Figure 6.1 The $p(j)$ function: dashed red arrows show the last compatible interval before each j . For example, $p(5) = 3$ because I_3 is the rightmost interval that finishes before I_5 starts.

Computing $p(j)$ efficiently: Since intervals are sorted by finish time, we can find $p(j)$ for all j in $O(n \log n)$ using binary search, or $O(n^2)$ naively by checking all previous intervals.

6.4 The Recurrence Relation

Theorem 6.1 (Recurrence)

For each $j = 1, 2, \dots, n$:

$$OPT(j) = \max(v_j + OPT(p(j)), OPT(j-1)) \quad (6.1)$$

Proof (by case analysis):

Consider an optimal solution O_j for intervals $\{1, \dots, j\}$. There are exactly two cases:

Case 1: $j \in O_j$ (interval j is selected).

By definition of $p(j)$, intervals $p(j) + 1, p(j) + 2, \dots, j - 1$ all overlap interval j . Therefore, **none** of these can be in O_j . The remaining selected intervals must come from $\{1, \dots, p(j)\}$. Moreover, O_j must include an **optimal solution** to $\{1, \dots, p(j)\}$ — otherwise we could replace it with a better one without conflicting with j .

$$\text{Value} = v_j + \text{OPT}(p(j))$$

Case 2: $j \notin O_j$ (interval j is not selected).

Then O_j is simply the optimal solution to $\{1, \dots, j - 1\}$.

$$\text{Value} = \text{OPT}(j - 1)$$

Since these are the only two possibilities, the optimal value is the maximum of these two cases. ■

Theorem 6.2 (Decision Rule)

Request j belongs to an optimal solution on $\{1, 2, \dots, j\}$ if and only if:

$$v_j + \text{OPT}(p(j)) \geq \text{OPT}(j - 1) \quad (6.2)$$

6.5 Naive Recursive Algorithm: Compute-Opt

Algorithm: Compute-Opt (Naive Recursive)

```

Compute-Opt(j)
  If j = 0 then
    Return 0
  Else
    Return max(v_j + Compute-Opt(p(j)), Compute-Opt(j - 1))
  Endif
End

Invoke Compute-Opt(n) to obtain OPT(n)

```

Theorem 6.3 (Correctness of Compute-Opt)

Compute-Opt(j) correctly computes $\text{OPT}(j)$ for each $j = 1, 2, \dots, n$.

Proof (by strong induction on j):

Base Case ($j = 0$): Compute-Opt(0) returns 0, and by definition $OPT(0) = 0$. Correct. ✓

Inductive Hypothesis: Assume Compute-Opt(i) correctly computes $OPT(i)$ for all $i < j$.

Inductive Step (prove for j):

Compute-Opt(j) returns:

$$\max(v_j + \text{Compute-Opt}(p(j)), \text{Compute-Opt}(j - 1))$$

By the inductive hypothesis:

- $\text{Compute-Opt}(p(j)) = OPT(p(j))$ since $p(j) < j$
- $\text{Compute-Opt}(j - 1) = OPT(j - 1)$ since $j - 1 < j$

Therefore:

$$\text{Compute-Opt}(j) = \max(v_j + OPT(p(j)), OPT(j - 1)) = OPT(j)$$

The last equality follows from Theorem 6.1. Thus Compute-Opt(j) correctly computes $OPT(j)$. ■

6.6 Why the Naive Algorithm is Exponential

Theorem: The running time of naive Compute-Opt is $O(2^n)$.

Full Derivation:

Let $T(j)$ be the number of recursive calls made by Compute-Opt(j). From the recurrence:

$$T(j) = T(p(j)) + T(j - 1) + 1$$

Worst-case instance: Consider the case where $p(j) = j - 2$ for all $j = 2, 3, \dots, n$. This creates the recurrence:

$$T(j) = T(j - 2) + T(j - 1) + 1$$

This is **identical to the Fibonacci recurrence**. The Fibonacci numbers grow as $F_j \approx \phi^j / \sqrt{5}$ where $\phi = (1 + \sqrt{5})/2 \approx 1.618$.

Therefore:

$$T(n) = \Theta(\phi^n) = \Theta(1.618^n) \subseteq O(2^n)$$

Key insight: The exponential runtime is purely due to **redundant calls** to the same subproblems. Compute-Opt only solves $n + 1$ **distinct** subproblems: Compute-Opt(0), Compute-Opt(1), ..., Compute-Opt(n). But many subproblems are solved repeatedly.

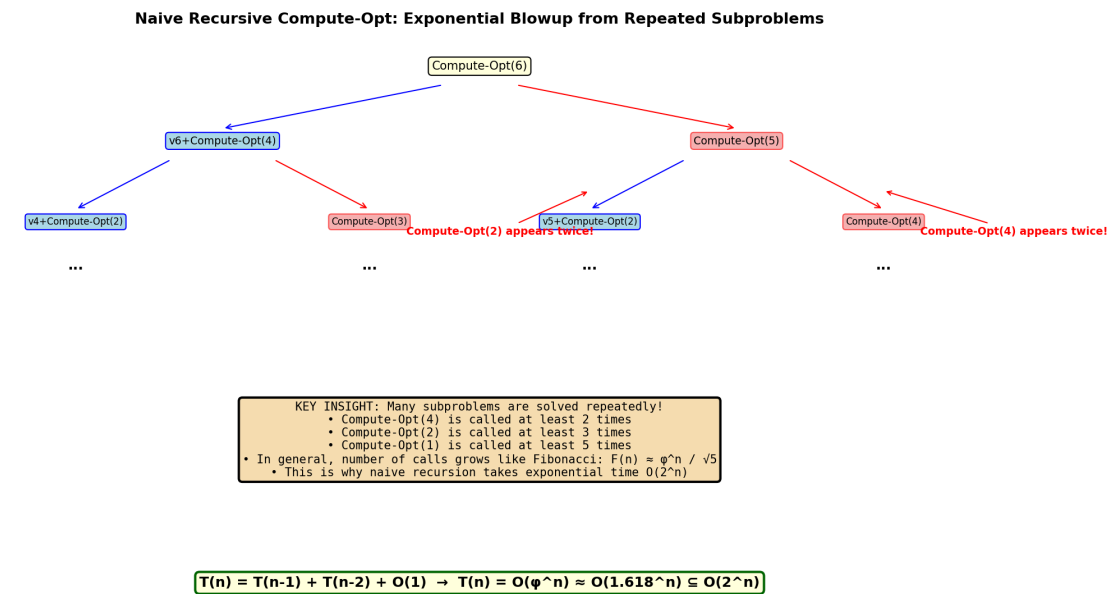


Figure 6.2 Recursion tree for Compute-Opt showing exponential blowup. Compute-Opt(4) and Compute-Opt(2) appear multiple times — this redundancy is what makes the algorithm exponential.

6.7 Memoization: M-Compute-Opt

The key insight is to **store computed values** in a globally accessible array $M[0 \dots n]$.

Algorithm: M-Compute-Opt (Memoized)

Initialize array $M[0 \dots n]$ with all entries set to "empty"
 Set $M[0] = 0$

M-Compute-Opt(j)

 If $j = 0$ then

 Return 0

 Else if $M[j]$ is not empty then

 Return $M[j]$

 Else

 Define $M[j] = \max(v_j + \text{M-Compute-Opt}(p(j)), \text{M-Compute-Opt}(j - 1))$

 Return $M[j]$

 Endif

End

Invoke M-Compute-Opt(n) to obtain OPT(n)

Theorem 6.4 (Time Complexity of Memoized Version)

The running time of $M\text{-Compute-Opt}(n)$ is $O(n)$ (assuming intervals are sorted by finish times and $p(j)$ is precomputed).

Proof (Progress Measure Argument):

Time per call: Each call to $M\text{-Compute-Opt}(j)$ (excluding recursive calls it generates) performs $O(1)$ work:

- Check if $j = 0$: $O(1)$
- Check if $M[j]$ is empty: $O(1)$
- Compute max and store in $M[j]$: $O(1)$

Number of calls: We use a **progress measure** — the number of entries in M that are not "empty."

- Initially: 0 filled entries (only $M[0]$ is set, which is given).
- Each time $M\text{-Compute-Opt}$ is called with $M[j]$ empty, it fills $M[j]$ with a new value.
- The key insight: $M[j]$ **can transition from "empty" to "filled" at most once**.
- Since M has only $n + 1$ entries, there are at most $n + 1$ "productive" calls that actually compute a new value.
- Calls on already-filled entries return immediately without generating new recursive calls.

Therefore, the total number of calls to $M\text{-Compute-Opt}$ is bounded by $O(n)$, and since each call does $O(1)$ work:

$$T(n) = O(n) \cdot O(1) = O(n)$$

■

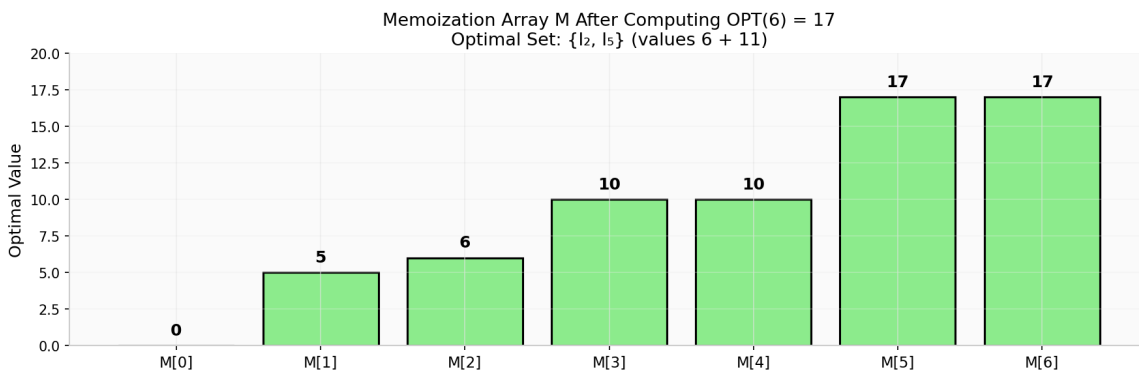


Figure 6.3 Memoization array M after computing $\text{OPT}(6)$. Each entry is filled exactly once, with $M[j]$ storing the optimal value for the first j intervals.

6.8 Iterative Bottom-Up Algorithm

Instead of memoized recursion, we can fill the array M **iteratively** from smallest to largest.

Algorithm: Iterative-Compute-Opt

```

Iterative-Compute-Opt
  M[0] = 0
  For j = 1, 2, ..., n
    M[j] = max(v_j + M[p(j)], M[j - 1])
  Endfor
  Return M[n]

```

Theorem (Correctness of Iterative Version)

Iterative-Compute-Opt correctly computes $OPT(n)$ in $O(n)$ time.

Proof (by induction on j):

Base Case: $M[0] = 0 = OPT(0)$. ✓

Inductive Step: Assume $M[i] = OPT(i)$ for all $i < j$. When computing $M[j]$, both $M[p(j)]$ and $M[j - 1]$ have already been computed (since $p(j) < j$ and $j - 1 < j$). By the inductive hypothesis, these equal $OPT(p(j))$ and $OPT(j - 1)$. Therefore:

$$M[j] = \max(v_j + OPT(p(j)), OPT(j - 1)) = OPT(j)$$

by Theorem 6.1. ✓

Running Time: The loop runs for n iterations, each doing $O(1)$ work:

$$T(n) = \sum_{j=1}^n O(1) = O(n)$$

■

Comparison: Memoization vs Iteration

Table 6.1 Memoized Recursion vs Bottom-Up Iteration

Aspect	Memoized (M-Compute-Opt)	Iterative (Iterative-Compute-Opt)
Direction	Top-down (starts from n)	Bottom-up (starts from 0)
Array filling	On-demand, only when needed	Systematic, all entries in order
Recursion depth	$O(n)$	None — purely iterative
Time complexity	$O(n)$	$O(n)$
Space complexity	$O(n)$ for M + recursion stack	$O(n)$ for M only
Code complexity	Slightly more complex	Simpler, more explicit

6.9 Finding the Actual Solution (Find-Solution)

So far we've only computed $OPT(n)$ — the **value** of the optimal solution. We want the **actual set of intervals**.

Algorithm: Find-Solution (Backtracking)

```

Find-Solution(j)
  If j = 0 then
    Output nothing
  Else
    If  $v_j + M[p(j)] \geq M[j - 1]$  then
      Output j together with the result of Find-Solution(p(j))
    Else
      Output the result of Find-Solution(j - 1)
    Endif
  Endif
End

Invoke Find-Solution(n) to obtain the optimal set

```

Theorem 6.5 (Correctness and Complexity of Find-Solution)

Given the array M of optimal values, Find-Solution returns an optimal solution in $O(n)$ time.

Proof:

Correctness (by induction on j):

Base case $j = 0$: Returns empty set, optimal with value 0. ✓

At step j , there are two cases:

Case 1: $v_j + M[p(j)] \geq M[j - 1]$

By Theorem 6.2, I_j is in some optimal solution. The optimal value $M[j] = v_j + M[p(j)]$. Find-Solution outputs j and recursively calls Find-Solution($p(j)$). By inductive hypothesis, this returns an optimal subset for $\{I_1, \dots, I_{p(j)}\}$ with value $M[p(j)]$. Adding I_j gives value $v_j + M[p(j)] = M[j]$. ✓

Case 2: $v_j + M[p(j)] < M[j - 1]$

Then $M[j] = M[j - 1]$, so the optimal solution doesn't include I_j . Find-Solution recursively calls Find-Solution($j - 1$). By inductive hypothesis, this returns an optimal subset for $\{I_1, \dots, I_{j-1}\}$ with value $M[j - 1] = M[j]$. ✓

Time Complexity:

Each call to Find-Solution(j) does $O(1)$ work and makes exactly one recursive call to either Find-Solution($p(j)$) or Find-Solution($j - 1$). Since $p(j) < j$ and $j - 1 < j$, the argument strictly decreases. Starting from n , we can make at most n recursive calls before reaching 0. Therefore:

$$T(n) = O(n)$$

■

6.10 Complete Example Trace

Table 6.2 Full Trace of Iterative-Compute-Opt and Find-Solution

j	I_j	$[s_j, f_j]$	v_j	$p(j)$	$v_j + M[p(j)]$	$M[j - 1]$	$M[j]$	Include?
1	I_1	[1, 3]	5	0	$5 + 0 = 5$	0	5	Yes
2	I_2	[2, 5]	6	0	$6 + 0 = 6$	5	6	Yes
3	I_3	[4, 7]	5	1	$5 + 5 = 10$	6	10	Yes
4	I_4	[6, 8]	4	2	$4 + 6 = 10$	10	10	Either
5	I_5	[5, 9]	11	2	$11 + 6 = 17$	10	17	Yes
6	I_6	[8, 10]	2	4	$2 + 10 = 12$	17	17	No

Find-Solution(6):

- At $j = 6$: $v_6 + M[p(6)] = 2 + 10 = 12 < M[5] = 17$. Skip I_6 . $\rightarrow$ Find-Solution(5)
- At $j = 5$: $v_5 + M[p(5)] = 11 + 6 = 17 \geq M[4] = 10$. Include I_5 . $\rightarrow$ Find-Solution(2)
- At $j = 2$: $v_2 + M[p(2)] = 6 + 0 = 6 \geq M[1] = 5$. Include I_2 . $\rightarrow$ Find-Solution(0)
- At $j = 0$: Done.

Optimal Set: $\{I_2, I_5\}$ with total value $6 + 11 = 17$. ✓

Summary: Weighted Interval Scheduling

Algorithm	Time	Space	Type
Naive Compute-Opt	$O(2^n)$	$O(n)$	Recursive
M-Compute-Opt	$O(n)$	$O(n)$	DP (Memoization)
Iterative-Compute-Opt	$O(n)$	$O(n)$	DP (Bottom-up)
Find-Solution	$O(n)$	$O(n)$	Backtracking

7. Subset Sum Problem

7.1 Problem Definition

Given n items with weights $w_1, w_2, \dots, w_n$ and a target weight W , find a subset of items whose total weight is **as large as possible without exceeding W** .

Formal Definition

Find subset $S \subseteq \{1, \dots, n\}$ maximizing $\sum_{i \in S} w_i$ subject to $\sum_{i \in S} w_i \leq W$.

7.2 Why the "Obvious" Subproblems Fail

The natural first attempt is to define $OPT(i)$ as the best solution using only the first i items. But this fails:

- **Case 1 ($n \notin O$):** $OPT(n) = OPT(n - 1)$ — ignore item n .
- **Case 2 ($n \in O$):** Item n uses weight w_n . The remaining capacity is $W - w_n$. But $OPT(n - 1)$ doesn't tell us the best solution with capacity $W - w_n$ — it only tells us the best solution with the *original* capacity using $n - 1$ items.

Key Insight: We need to know not just which items, but **how much weight capacity remains**. This requires adding a new variable.

7.3 Adding a Variable: The 2D Subproblem

Definition: $OPT(i, w)$

For each $i = 0, 1, \dots, n$ and each integer $0 \leq w \leq W$:

$$OPT(i, w) = \max_{S \subseteq \{1, \dots, i\}, \sum_{j \in S} w_j \leq w} \sum_{j \in S} w_j \quad (7.1)$$

This is the maximum total weight achievable using a subset of items $\{1, \dots, i\}$ with maximum allowed weight w .

7.4 The Recurrence Relation

Theorem 6.8 (Subset Sum Recurrence)

For all $i = 1, 2, \dots, n$ and all $w = 0, 1, \dots, W$:

If $w < w_i$:

$$OPT(i, w) = OPT(i - 1, w) \quad (6.8a)$$

Otherwise:

$$OPT(i, w) = \max (OPT(i - 1, w), w_i + OPT(i - 1, w - w_i)) \quad (6.8b)$$

Proof (by case analysis):

Consider the optimal solution for the first i items with weight limit w .

Case 1: $w < w_i$

Item i cannot be included (it's too heavy). The optimal solution uses only items 1 through $i - 1$:

$$OPT(i, w) = OPT(i - 1, w)$$

Case 2: $w \geq w_i$

We have two choices for item i :

- **Exclude item i :** Best we can do is $OPT(i - 1, w)$.
- **Include item i :** We gain w_i weight, and have remaining capacity $w - w_i$ for items 1 through $i - 1$. Best value: $w_i + OPT(i - 1, w - w_i)$.

We take the maximum of these two options. ■

7.5 The Algorithm

Algorithm: Subset-Sum

```
Subset-Sum(n, W)
  Array M[0 ... n, 0 ... W]
  Initialize M[0, w] = 0 for each w = 0, 1, ..., W
  For i = 1, 2, ..., n
    For w = 0, 1, ..., W
      If w < w_i then
        M[i, w] = M[i - 1, w]
      Else
        M[i, w] = max(M[i - 1, w], w_i + M[i - 1, w - w_i])
      Endif
    Endfor
  Endfor
  Return M[n, W]
```

7.6 Time Complexity Analysis (Full Derivation)

Theorem 6.9: The Subset-Sum algorithm runs in $O(nW)$ time and uses $O(nW)$ space.

Proof of Time Complexity:

Table size: The table M has $(n + 1) \times (W + 1)$ entries.

Time per entry: Computing each entry $M[i, w]$ requires:

- One comparison $w < w_i$: $O(1)$
- At most one addition $w_i + M[i - 1, w - w_i]$: $O(1)$
- One max operation: $O(1)$

Therefore, each entry takes $O(1)$ time.

Total time:

$$T(n, W) = \sum_{i=1}^n \sum_{w=0}^W O(1) = (n + 1)(W + 1) \cdot O(1) = O(nW)$$

Space Complexity: The table M has $(n + 1)(W + 1) = O(nW)$ entries, each storing an integer. Thus:

$$S(n, W) = O(nW)$$

Note: This is a **pseudo-polynomial** algorithm — polynomial in the numeric value W , but exponential in the number of bits needed to represent W (which is $\log W$). If W is encoded in binary, the input size is $O(n \log W)$, making $O(nW) = O(n \cdot 2^{\log W})$ exponential in the input size.

7.7 Visual Walkthrough

Subset Sum DP Table: Items weights {2, 2, 3}, W=6
Green=Include chosen, Yellow=Can't fit (inherited), White=Exclude chosen

	w=0	w=1	w=2	w=3	w=4	w=5	w=6
i=0	0	0	0	0	0	0	0
i=1	0	0	2	2	2	2	2
i=2	0	0	2	2	4	4	4
i=3	0	0	2	3	4	5	5

Figure 7.1 Subset Sum DP table for weights {2, 2, 3} with $W = 6$. Green cells indicate "include" was chosen; yellow indicates the item couldn't fit (value inherited from above).

Detailed Table Trace:

Table 7.1 Subset Sum Execution Trace ($W = 6$, weights {2, 2, 3})

Phase	Item	Weight	Explanation
Initial	—	—	Row 0: all zeros (no items selected)
$i = 1$	$w_1 = 2$	2	For $w \geq 2$: $\max(0, 2 + 0) = 2$. Row 1: [0, 0, 2, 2, 2, 2, 2]
$i = 2$	$w_2 = 2$	2	At $w = 4$: $\max(2, 2 + M[1,2]) = \max(2, 4) = 4$. Row 2: [0, 0, 2, 2, 4, 4, 4]
$i = 3$	$w_3 = 3$	3	At $w = 5$: $\max(4, 3 + M[2,2]) = \max(4, 5) = 5$. Row 3: [0, 0, 2, 3, 4, 5, 5]

Final Answer: $M[3, 6] = 5$ — achieved by items 2 and 3 (weights $2 + 3 = 5$).

7.8 Finding the Solution Set

Theorem 6.10

Given a table M of optimal values, the optimal set S can be found in $O(n)$ time.

Algorithm and Proof:

Trace back from $M[n, W]$:

- If $M[i, w] = M[i - 1, w]$, item i is **not** in the set. Move to $M[i - 1, w]$.
- If $M[i, w] \neq M[i - 1, w]$, item i is in the set. Move to $M[i - 1, w - w_i]$.

Each step decreases i by 1, so at most n steps. Time: $O(n)$. ■

8. 0-1 Knapsack (Dynamic Programming)

8.1 Problem Definition

Given n items with weights $w_1, \dots, w_n$ and values $v_1, \dots, v_n$, and a knapsack capacity W , select a subset of items to **maximize total value** without exceeding weight capacity. Unlike fractional knapsack, items cannot be divided.

8.2 The Recurrence Relation

Theorem 6.11 (0-1 Knapsack Recurrence)

For all $i = 1, 2, \dots, n$ and all $w = 0, 1, \dots, W$:

If $w < w_i$:

$$OPT(i, w) = OPT(i - 1, w) \quad (6.11a)$$

Otherwise:

$$OPT(i, w) = \max (OPT(i - 1, w), v_i + OPT(i - 1, w - w_i)) \quad (6.11b)$$

Proof:

Identical structure to Theorem 6.8 (Subset Sum), but maximizing **value** instead of weight.

Case 1: $w < w_i$

Item i doesn't fit. Must exclude it. $OPT(i, w) = OPT(i - 1, w)$.

Case 2: $w \geq w_i$

- **Exclude:** Best value without item i is $OPT(i - 1, w)$.
- **Include:** Value v_i plus best value from remaining capacity $w - w_i$ using items 1 through $i - 1$: $v_i + OPT(i - 1, w - w_i)$.

Take the maximum. ■

8.3 The Algorithm

Algorithm: 0-1 Knapsack (Dynamic Programming)

```
Initialize array M[0 ... n, 0 ... W]
For w = 0, 1, ..., W
    M[0, w] = 0
Endfor

For i = 1, 2, ..., n
    For w = 0, 1, ..., W
        If w < wi then
            M[i, w] = M[i - 1, w]
        Else
            include_value = vi + M[i - 1, w - wi]
            exclude_value = M[i - 1, w]
            M[i, w] = max(exclude_value, include_value)
        Endif
    Endfor
Endfor

Return M[n, W]
```

8.4 Time Complexity Analysis (Full Derivation)

Theorem 6.12: The 0-1 Knapsack algorithm runs in $O(nW)$ time and uses $O(nW)$ space.

Proof of Time Complexity:

Initialization: Setting $M[0, w] = 0$ for all $w = 0, \dots, W$ takes $O(W)$ time.

Main loops:

- Outer loop: i runs from 1 to $n \rightarrow n$ iterations
- Inner loop: w runs from 0 to $W \rightarrow W + 1$ iterations
- Body operations: one comparison ($w < w_i$), at most one addition, one max $\rightarrow$ all $O(1)$

Therefore:

$$T(n, W) = O(W) + \sum_{i=1}^n \sum_{w=0}^W O(1) = O(W) + n(W + 1) \cdot O(1) = O(nW)$$

Space Complexity: The 2D table has $(n + 1)(W + 1) = O(nW)$ entries.

Space Optimization: Using a **rolling array**, we only need two rows (current and previous) at any time:

$$S_{\text{opt}}(n, W) = O(W)$$

Pseudo-polynomial nature: As with Subset Sum, this is pseudo-polynomial. If W requires $b = \log_2 W$ bits to represent, then the input size is $\Theta(n \cdot b)$, and $O(nW) = O(n \cdot 2^b)$, which is exponential in the input size.

8.5 Visual Walkthrough

0-1 Knapsack DP Table: Items (5,60), (4,50), (6,70), (3,30), W=10
 Green=Include chosen, Yellow=Can't fit (inherited), White=Exclude chosen

	w=0	w=1	w=2	w=3	w=4	w=5	w=6	w=7	w=8	w=9	w=10
i=0	0	0	0	0	0	0	0	0	0	0	0
i=1	0	0	0	0	0	60	60	60	60	60	60
i=2	0	0	0	0	50	60	60	60	60	110	110
i=3	0	0	0	0	50	60	70	70	70	110	120
i=4	0	0	0	30	50	60	70	80	90	110	120

Figure 8.1 0-1 Knapsack DP table for items {(5,60), (4,50), (6,70), (3,30)} with W=10. Green cells show where "include" gave a better value.

Detailed Trace:

Table 8.1 0-1 Knapsack Execution Trace

Item	Weight	Value	Key Computations
1	5	60	For $w \geq 5$: $\max(0, 60+0) = 60$. Row 1: [0,0,0,0,0,60,60,60,60,60,60]
2	4	50	At $w=4$: $\max(0, 50+0) = 50$. At $w=9$: $\max(60, 50+60) = 110$. Row 2: [0,0,0,0,50,60,60,60,60,110,110]
3	6	70	At $w=6$: $\max(60, 70+0) = 70$. At $w=10$: $\max(110, 70+50) = 120$. Row 3: [0,0,0,0,50,60,70,70,70,110,120]
4	3	30	At $w=3$: $\max(0, 30+0) = 30$. At $w=7$: $\max(70, 30+60) = 90$. Row 4: [0,0,0,30,50,60,70,90,90,110,120]

Final Answer: $M[4, 10] = 120$.

8.6 Tracing the Solution

Algorithm: Trace-Solution

```

Trace-Solution(i, w)
  If i = 0 then
    Output nothing
  Else
    If w < w_i then
      Output the result of Trace-Solution(i - 1, w)
    Else if v_i + M[i - 1, w - w_i] > M[i - 1, w] then
      Output i together with Trace-Solution(i - 1, w - w_i)
    Else
      Output the result of Trace-Solution(i - 1, w)
    Endif
  Endif
End

```

Trace from $M[4, 10] = 120$:

- At (4, 10): $w_4 = 3 \leq 10$. Check: $v_4 + M[3, 7] = 30 + 70 = 100 \not> M[3, 10] = 120$. Skip item 4. $\rightarrow$ Trace-Solution(3, 10)
- At (3, 10): $w_3 = 6 \leq 10$. Check: $v_3 + M[2, 4] = 70 + 50 = 120 > M[2, 10] = 110$. Include item 3. $\rightarrow$ Trace-Solution(2, 4)
- At (2, 4): $w_2 = 4 \leq 4$. Check: $v_2 + M[1, 0] = 50 + 0 = 50 > M[1, 4] = 0$. Include item 2. $\rightarrow$ Trace-Solution(1, 0)
- At (1, 0): $w_1 = 5 > 0$. Skip. $\rightarrow$ Trace-Solution(0, 0)
- Done.

Optimal Set: $\{2, 3\}$ with total weight $4 + 6 = 10$ and total value $50 + 70 = 120$. ✓

9. Principles of Dynamic Programming

9.1 The Three Essential Properties

To develop a dynamic programming algorithm, one needs a collection of subproblems satisfying three properties:

Property (i): Polynomial Number of Subproblems

There are only a polynomial number of distinct subproblems.

- Weighted Interval Scheduling: $n + 1$ subproblems ($OPT(0)$ through $OPT(n)$)
- Subset Sum: $(n + 1)(W + 1) = O(nW)$ subproblems
- 0-1 Knapsack: $(n + 1)(W + 1) = O(nW)$ subproblems

Property (ii): Original Problem Computable from Subproblems

The solution to the original problem can be easily computed from the solutions to the subproblems. Often, the original problem **is** one of the subproblems.

- Weighted Interval Scheduling: $M[n]$ directly gives $OPT(n)$
- Subset Sum: $M[n, W]$ directly gives the optimal value

Property (iii): Natural Ordering + Recurrence

There is a natural ordering on subproblems from "smallest" to "largest," and an easy-to-compute recurrence that determines the solution to a subproblem from solutions to strictly smaller subproblems.

- Weighted Interval Scheduling: Order $0, 1, 2, \dots, n$; recurrence uses $p(j) < j$ and $j - 1 < j$
- Subset Sum: Order by increasing i and w ; each entry depends only on previous row ($i - 1$)

9.2 Memoization vs Iteration

Table 9.1 Memoization vs Iteration over Subproblems

Aspect	Memoization	Iteration
Definition	Saving computed values to avoid recomputation	Systematically computing all subproblems in order
Role in DP	Technique to optimize recursive algorithms	The fundamental DP paradigm
When to use	When recursion structure is natural	When explicit ordering makes reasoning clearer
Overhead	Recursion stack overhead	No recursion overhead
Subproblems solved	Only those needed (on-demand)	All subproblems systematically

9.3 The Design Process: Chicken-and-Egg Problem

Finding the right subproblems and recurrence is often a back-and-forth process. Two strategies exist:

Strategy A: Start with subproblems, find recurrence.

- Formulate a set of subproblems that looks natural.
- Then figure out a recurrence linking them together.

Strategy B: Start with recurrence, determine subproblems.

- Define a recurrence by reasoning about the structure of an optimal solution.
- Then determine which subproblems are necessary to unwind the recurrence.

Weighted Interval Scheduling followed Strategy B

First reasoned about optimal solution structure (include/exclude j), derived recurrence (6.1), then the subproblems ($OPT(0), \dots, OPT(n)$) naturally emerged.

9.4 When to Use DP vs Greedy

Table 9.2 Choosing Between Greedy and Dynamic Programming

Characteristic	Use Greedy	Use Dynamic Programming
Problem structure	Greedy-choice property holds	Optimal substructure + overlapping subproblems
Subproblems after choice	Only one subproblem remains	Multiple overlapping subproblems
Need to compare	No need to compare include vs exclude globally	Must compare solutions to subproblems
Examples	Interval scheduling, fractional knapsack	Weighted interval scheduling, 0-1 knapsack, subset sum
Time complexity	Faster: $O(n \log n)$ or $O(n)$	Slower: $O(nW)$, $O(n^2)$, etc.

Summary: Algorithm Comparison

Table S.1 Complete Algorithm Comparison

Algorithm	Time Complexity	Space Complexity	Type	Key Technique
Interval Scheduling (Greedy)	$O(n \log n)$	$O(n)$	Greedy	Earliest finish time first
Interval Partitioning	$O(n \log n)$	$O(n)$	Greedy	Assign smallest available label
Fractional Knapsack	$O(n \log n)$	$O(n)$	Greedy	Highest value density first
Recursive Weighted Interval	$O(2^n)$	$O(n)$	Recursive	Include/exclude recurrence
Memoized Weighted Interval	$O(n)$	$O(n)$	DP (Memo)	Store OPT(j) in array M
Iterative Weighted Interval	$O(n)$	$O(n)$	DP (Iter)	Bottom-up fill of M
Find-Solution (WIS)	$O(n)$	$O(n)$	Backtrack	Trace through M array
Subset Sum	$O(nW)$	$O(nW)$ or $O(W)$	DP (2D)	Add weight variable
0-1 Knapsack	$O(nW)$	$O(nW)$ or $O(W)$	DP (2D)	Maximize value with weight constraint
Trace-Solution (Knapsack)	$O(n)$	$O(n)$	Backtrack	Trace through 2D table

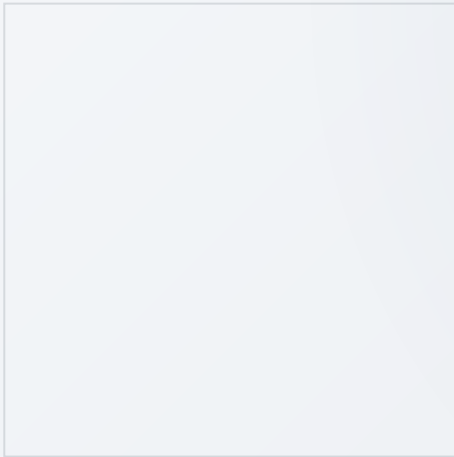
Key Theorems Reference

Table S.2 Key Theorems and Recurrences

Theorem	Statement
4.1	The greedy set A is compatible
4.2	Stays Ahead: $f(i_r) \leq f(j_r)$ for all $r \leq k$
4.3	Greedy algorithm returns an optimal set
6.1	$OPT(j) = \max(v_j + OPT(p(j)), OPT(j - 1))$
6.2	Item j is in optimal solution iff $v_j + OPT(p(j)) \geq OPT(j - 1)$
6.3	Compute-Opt correctly computes $OPT(j)$
6.4	M-Compute-Opt runs in $O(n)$ time
6.5	Find-Solution returns optimal solution in $O(n)$ time
6.8	Subset Sum recurrence with 2D subproblems
6.9	Subset-Sum algorithm is correct and runs in $O(nW)$
6.10	Optimal set can be found in $O(n)$ time from table
6.11	0-1 Knapsack recurrence (maximize value)
6.12	0-1 Knapsack can be solved in $O(nW)$ time

Final Summary: What Makes DP Work

1. **Optimal Substructure:** The optimal solution to the main problem contains optimal solutions to subproblems.
2. **Overlapping Subproblems:** The same subproblems are solved repeatedly in a naive recursive approach.
3. **Memoization or Iteration:** Store solutions to subproblems to avoid redundant computation.
4. **Polynomial Subproblems:** The number of distinct subproblems must be polynomially bounded.
5. **Natural Ordering:** There must be an ordering that allows building solutions from smaller to larger.



Searching & Hashing

Complete Algorithm Reference Notes

Linear Search • Binary Search • Hash Tables

Hash Functions • Collision Resolution • Open Addressing

Comprehensive Analysis with Proofs, Complexity Derivations & Visual Walkthroughs



Table of Contents

Part 1: Linear Search

- 1.1 Algorithm Overview
- 1.2 Complete Complexity Analysis
- 1.3 Properties
- 1.4 Visual Walkthrough
- 1.5 Loop Invariant & Correctness Proof

Part 2: Binary Search

- 2.1 Algorithm Overview
- 2.2 Complete Complexity Analysis
- 2.3 Properties
- 2.4 Visual Walkthrough
- 2.5 Loop Invariant & Correctness Proof

Part 3: Hash Tables

- 3.1 Introduction & Direct Addressing Problem
- 3.2 Hash Table Core Concept
- 3.3 The Collision Problem
- 3.4 Independent Uniform Hashing
- 3.5 Collision Resolution by Chaining
- 3.6 Hash Table Operations & Analysis

Part 4: Hash Functions

- 4.1 What Makes a Good Hash Function?
- 4.2 Static vs. Random Hashing
- 4.3 The Division Method
- 4.4 The Multiplication Method
- 4.5 The Multiply-Shift Method

Part 5: Open Addressing

5.1 Concept & Overview

5.2 Linear Probing

5.3 Quadratic Probing

5.4 Double Hashing

5.5 Deletion in Open Addressing

5.6 Comparison of Probing Methods

Part 6: Summary & Comparison Tables

Part 1: Linear Search

1.1 Algorithm Overview

Type: Sequential / Brute-force search algorithm.

How it works: Linear Search examines each element of the array sequentially from the first index to the last, comparing every element with the target key until either a match is found or the entire array has been traversed.

Algorithm: Linear-Search($A[1\dots n]$, key)

```
1  i ← 1
2  while i ≤ n do
3      if A[i] = key then
4          return i
5      end
6      i ← i + 1
7  end
8  return 0          // key not found
```

Best cases to use this algorithm:

- When the array is **unsorted** or **unorganized**.
- When the dataset is **small** (small n).
- When searching is performed **infrequently**.
- When the **target is expected to be near the beginning** of the array.
- When memory access patterns are sequential and cache-friendly for small arrays.

1.2 Complete Complexity Analysis

1.2.1 RAM Model & Line-by-Line Cost

RAM Model Assumptions:

- Each primitive operation (assignment, comparison, arithmetic) takes constant time.
- Memory access is uniform and constant time.

Table 1: Line-by-line cost analysis for Linear Search

Line	Operation	Cost	Times (Worst Case)
1	$i \leftarrow 1$	c_1	1
2	$i \leq n$ (loop test)	c_2	$n + 1$
3	$A[i] = \text{key}$ (compare)	c_3	n
4	$\text{return } i$	c_4	0 (worst case)
6	$i \leftarrow i + 1$	c_5	n
8	$\text{return } 0$	c_6	1

1.2.2 Recurrence Relation & Full Derivation

Let $T(n)$ be the total number of comparisons (or time) for an array of size n .

Recurrence for Worst Case (unsuccessful search):

$$T(0) = 0 \quad (\text{empty array, no comparisons})$$

$$T(n) = T(n - 1) + 1 \quad \text{for } n > 0$$

Step-by-step solving by substitution (expansion method):

We expand the recurrence by repeatedly substituting the definition of T :

$$\begin{aligned}
 T(n) &= T(n - 1) + 1 \\
 &= [T(n - 2) + 1] + 1 = T(n - 2) + 2 \\
 &= [T(n - 3) + 1] + 2 = T(n - 3) + 3 \\
 &\vdots \\
 &= T(n - k) + k
 \end{aligned}$$

We reach the base case when $n - k = 0$, i.e., $k = n$:

$$T(n) = T(0) + n = 0 + n = n$$

Result:

$$T(n) = n \Rightarrow T(n) = \Theta(n)$$

1.2.3 Alternative Derivation: Summing the Loop

The total worst-case cost can also be obtained by direct summation:

$$\begin{aligned} T(n) &= c_1 + (n + 1) \cdot c_2 + n \cdot c_3 + n \cdot c_5 + c_6 \\ &= (c_2 + c_3 + c_5) \cdot n + (c_1 + c_2 + c_6) \\ &= a \cdot n + b \quad (\text{a linear function of } n) \\ &= \Theta(n) \end{aligned}$$

1.2.4 All Cases Summary**Table 2: Linear Search: complexity cases**

Case	Scenario	# Comparisons	Complexity
Best Case	Key at $A[1]$ (first position)	1	$\Theta(1)$
Average Case	Key at random position or not present	$\frac{n+1}{2}$	$\Theta(n)$
Worst Case	Key at $A[n]$ or not present	n	$\Theta(n)$

Average case detailed analysis: If the key is equally likely to be at any position, the expected number of comparisons is:

$$E[\text{comparisons}] = \frac{1}{n} \sum_{i=1}^n i = \frac{1}{n} \cdot \frac{n(n+1)}{2} = \frac{n+1}{2}$$

If the key is not present with probability p , then $E = p \cdot n + (1 - p) \cdot \frac{n+1}{2} = \Theta(n)$.

1.3 Properties**Table 3: Linear Search properties**

Property	Linear Search	Reason
----------	---------------	--------

Stable	N/A	Search algorithms do not reorder elements; stability (preservation of relative order of equal keys) is a property of sorting algorithms.
In-Place	Yes	Requires only $O(1)$ extra space: one index variable i . No additional data structures needed.
Adaptive	Partially	Terminates early if target is near the beginning. However, the algorithm always starts from index 1 regardless of input patterns.

1.4 Visual Walkthrough

Array A: [7, 12, 5, 22, 13, 32] (indices 1--6) Target key: 13

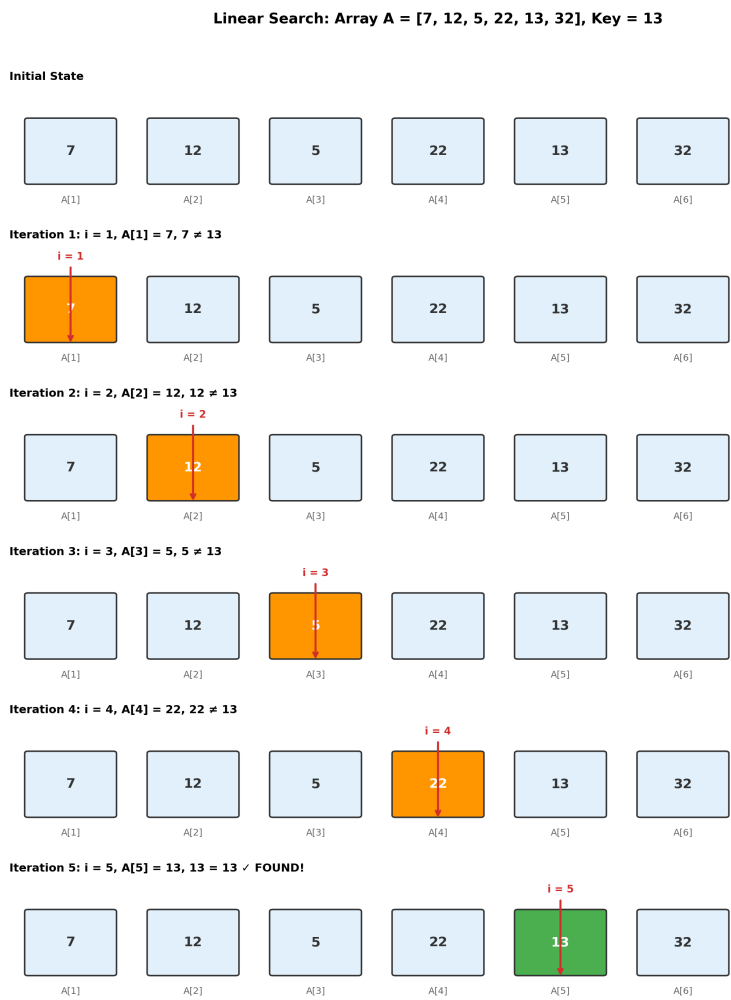


Figure 1: Linear Search iteration steps: checking each element sequentially until key 13 is found at index 5.

Table 4: Linear Search step-by-step trace

4th	4	22	$22 \neq 13$	$i \leftarrow 5$
5th	5	13	$13 = 13$	return 5

1.5 Loop Invariant & Correctness Proof

1.5.1 Loop Invariant

Hypothesis $H(r)$:

When the control reaches the "while" statement for the r -th time:

1. $i = r$
2. $key \notin A[1 \dots r - 1]$ (key is not present in the already-examined subarray)

Range: $\forall 1 \leq r \leq n + 1$

1.5.2 Proof by Mathematical Induction

Base Case ($r = 1$):

- Before the first iteration: $i = 1$ ✓
- $A[1 \dots 0]$ is empty, so $key \notin A[1 \dots 0]$ holds **vacuously** ✓

Induction Hypothesis: Assume $H(m)$ holds for some m where $1 \leq m \leq n$:

- $i = m$
- $key \notin A[1 \dots m - 1]$

Inductive Step (prove $H(m + 1)$):

At the start of iteration m , we check $A[m]$:

- **Case A:** If $A[m] = key$, the algorithm returns m (correct index). Loop terminates.
- **Case B:** If $A[m] \neq key$, then $key \notin A[1 \dots m]$ (since it was not in $A[1 \dots m - 1]$ by I.H., and $A[m] \neq key$). Then i increments to $m + 1$.

Therefore, at the start of the next iteration ($r = m + 1$): $i = m + 1$ ✓ and $key \notin A[1 \dots m]$ ✓

Thus $H(m) \Rightarrow H(m + 1)$. By induction, $H(r)$ holds for all valid r .

1.5.3 Using the Invariant to Prove Correctness

Termination Analysis: The loop terminates when either:

1. $A[i] = \text{key}$ is found $\rightarrow$ returns correct index i .
2. $i > n \rightarrow$ loop exits, returns 0.

Case 1 (Key found at position $k \leq n$):

By $H(k)$: $i = k$ and $\text{key} \notin A[1 \dots k - 1]$. When $A[k] = \text{key}$, return k -- this is the **first occurrence** ✓

Case 2 (Key not in array):

Loop runs until $i = n + 1$. By $H(n + 1)$: $i = n + 1$ and $\text{key} \notin A[1 \dots n]$. Loop condition $i \leq n$ fails, return 0 -- correct "not found" signal ✓

Conclusion:

The Linear Search algorithm is correct.

blacksquare

Part 2: Binary Search

2.1 Algorithm Overview

Type: Divide-and-Conquer search algorithm.

Precondition: The input array must be **sorted** (monotonically increasing or decreasing).

How it works:

1. Compare the target key with the middle element of the current interval.
2. If key equals middle element, return the middle index.
3. If key is less than middle element, search the left half.
4. If key is greater than middle element, search the right half.
5. Repeat until found or interval is empty.

Algorithm: Binary-Search-Recursive(A, first, last, key)

```
1  if first ≤ last then
2      mid ← ⌊(first + last) / 2⌋
3      if A[mid] = key then
4          return mid
5      end
6      if A[mid] < key then
7          return Binary-Search-Recursive(A, mid+1, last, key)
8      end
9      else
10         return Binary-Search-Recursive(A, first, mid-1, key)
11     end
12 end
13 else
14     return -1
15 end
```

Best cases to use this algorithm:

- When the array is **sorted** (precondition).
- When the dataset is **large** (large n).
- When **frequent searches** are performed on the same dataset (amortize sorting cost).
- When **random access** is available (arrays, not linked lists).

- When $O(\log n)$ time complexity is required.

2.2 Complete Complexity Analysis

2.2.1 RAM Model & Line-by-Line Cost

Table 5: Line-by-line cost analysis for Binary Search

Line	Operation	Cost
1	$first \leq last$ (comparison)	c_1
2	$mid \leftarrow \lfloor (first + last)/2 \rfloor$	c_2
3	$A[mid] = key$ (compare)	c_3
4	$return\ mid$	c_4
6	$A[mid] < key$ (compare)	c_5
7, 10	Recursive call	$T(n/2)$
14	$return\ -1$	c_8

2.2.2 Recurrence Relation & Full Derivation

Let $W(n)$ be the number of comparisons in the worst case for an array of size n .

Recurrence for Worst Case:

$$W(0) = 1 \quad (\text{one comparison: } first \leq last \text{ fails})$$

$$W(1) = 2 \quad (first \leq last \text{ check} + A[mid] = key \text{ check})$$

$$W(n) = W(\lfloor n/2 \rfloor) + 2 \quad \text{for } n > 1$$

Method 1: Substitution (Expansion)

$$\begin{aligned}
W(n) &= W(n/2) + 2 \\
&= [W(n/4) + 2] + 2 = W(n/4) + 2 \cdot 2 \\
&= [W(n/8) + 2] + 4 = W(n/8) + 3 \cdot 2 \\
&\vdots \\
&= W(n/2^k) + 2k
\end{aligned}$$

Base case reached when $n/2^k = 1$, i.e., $2^k = n$, so $k = \log_2 n$:

$$\begin{aligned}
W(n) &= W(1) + 2 \log_2 n \\
&= 2 + 2 \log_2 n \\
&= 2 \log_2 n + 2 \\
&= \Theta(\log n)
\end{aligned}$$

Method 2: Master Theorem

The recurrence $T(n) = T(n/2) + O(1)$ fits the Master Theorem form $T(n) = aT(n/b) + f(n)$ with $a = 1, b = 2, f(n) = O(1) = O(n^0)$.

Compare $n^{\log_b a} = n^{\log_2 1} = n^0 = 1$ with $f(n) = 1$.

Since $f(n) = \Theta(n^{\log_b a})$, we are in **Case 2** of the Master Theorem:

$$T(n) = \Theta(n^{\log_b a} \log n) = \Theta(\log n)$$

Method 3: Alternative (counting only key comparisons)

If we count only the comparison $A[mid] \stackrel{?}{=} key$:

$$T(n) = T(n/2) + 1, \quad T(1) = 1$$

Expanding:

$$\begin{aligned}
T(n) &= T(n/2) + 1 \\
&= T(n/4) + 1 + 1 = T(n/4) + 2 \\
&= T(n/8) + 3 \\
&\vdots \\
&= T(n/2^k) + k
\end{aligned}$$

Set $n/2^k = 1 \Rightarrow k = \log_2 n$:

$$T(n) = T(1) + \log_2 n = 1 + \log_2 n = \Theta(\log n)$$

2.2.3 All Cases Summary

Table 6: Binary Search: complexity cases

Case	Scenario	# Comparisons	Complexity
Best Case	Key is at the middle (first check)	2	$\Theta(1)$
Average Case	Key at random position or not present	$\sim \log_2 n$	$\Theta(\log n)$
Worst Case	Key not present, or at first/last position	$2 \log_2 n + 2$	$\Theta(\log n)$

2.3 Properties

Table 7: Binary Search properties

Property	Binary Search	Reason
Stable	N/A	Binary search is read-only; it never reorders or modifies the array.
In-Place	Yes	Requires only $O(1)$ extra space: variables <i>first</i> , <i>last</i> , <i>mid</i> . Even the recursive version uses $O(\log n)$ stack space, which is considered in-place for search algorithms.
Adaptive	No	Always divides the array in half regardless of input distribution. The split point is always the middle.

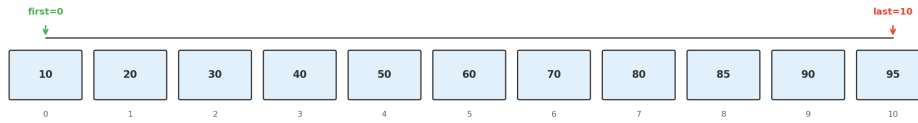
2.4 Visual Walkthrough

Array *A* (sorted): [10, 20, 30, 40, 50, 60, 70, 80, 85, 90, 95] (indices 0–10) Target key: 65

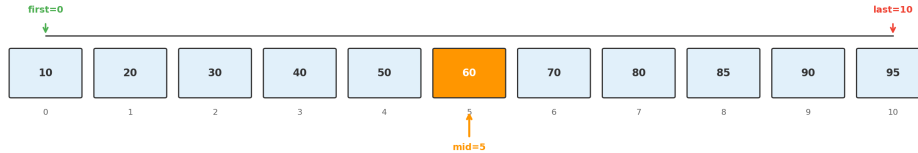
Part 2: Binary Search

Binary Search: Array A = [10, 20, 30, 40, 50, 60, 70, 80, 85, 90, 95], Key = 65

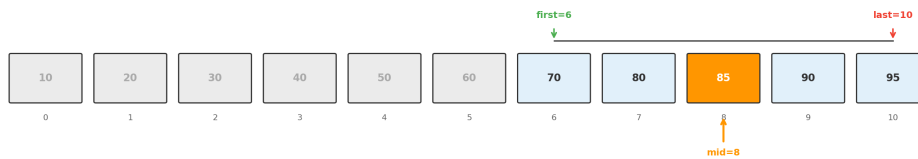
Initial: first = 0, last = 10, Search entire array



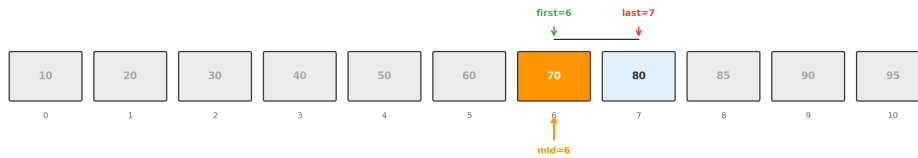
Iteration 1: mid = 5, A[5] = 60 < 65 → Search RIGHT: first = 6



Iteration 2: mid = 8, A[8] = 85 > 65 → Search LEFT: last = 7



Iteration 3: mid = 6, A[6] = 70 > 65 → Search LEFT: last = 5



Iteration 4: first = 6 > last = 5 → STOP, return -1 (Not Found)

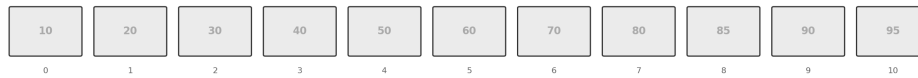


Figure 2: Binary Search iteration steps searching for key 65 (not found). Each iteration halves the search interval.

Table 8: Binary Search trace: key = 65 (unsuccessful)

Iteration	<i>first</i>	<i>last</i>	<i>mid</i>	$A[mid]$	Comparison	Action
Initial	0	10	--	--	--	--
1st	0	10	5	60	$60 < 65$	$first \leftarrow 6$
2nd	6	10	8	85	$85 > 65$	$last \leftarrow 7$
3rd	6	7	6	70	$70 > 65$	$last \leftarrow 5$
4th	6	5	--	--	$6 > 5$	return -1

Example where key IS found (key = 70):

Table 9: Binary Search trace: key = 70 (successful)

2.5 Loop Invariant & Correctness Proof

2.5.1 Loop Invariant

Invariant I :

At the start of each recursive call `Binary-Search-Recursive( $A$ ,  $first$ ,  $last$ ,  $key$ )`:

If key is present in array A at any index p , then $first \leq p \leq last$.

Equivalently: The subarray $A[first \dots last]$ contains all possible locations where key could be found.

2.5.2 Proof by Mathematical Induction

Initialization (Base Case):

- Initial call: $first = 0$, $last = n - 1$ (entire array).
- If key is anywhere in A , it is certainly in $A[0 \dots n - 1]$ ✓
- **Invariant holds.**

Maintenance (Inductive Step):

Assume I holds at the start of a call with parameters $(first, last)$. Let $mid = \lfloor (first + last) / 2 \rfloor$.

- **Case 1:** $A[mid] = key \rightarrow$ return mid . Correct! ✓
- **Case 2:** $A[mid] < key$ -- Since the array is sorted, key cannot be in $A[first \dots mid]$ (all elements $\leq A[mid] < key$). If key exists, it must be in $A[mid + 1 \dots last]$. Recursive call: $(mid + 1, last)$ -- invariant maintained ✓
- **Case 3:** $A[mid] > key$ -- Since the array is sorted, key cannot be in $A[mid \dots last]$ (all elements $\geq A[mid] > key$). If key exists, it must be in $A[first \dots mid - 1]$. Recursive call: $(first, mid - 1)$ -- invariant maintained ✓

In all cases, if key exists, the invariant is preserved.

Termination:

- The interval size strictly decreases: $last - first + 1$ becomes at most $\lceil n/2 \rceil$.
- Eventually $first > last$ (empty interval).
- At this point, by the contrapositive of the invariant: if the interval is empty and we have not found key , then key is **not present** in the original array.

- Return -1 is correct ✓

2.5.3 Correctness Proof Using the Invariant

Theorem: Binary Search returns the index mid where $A[mid] = key$ if key exists in A , and -1 if key does not exist in A .

Soundness (if algorithm returns index, it is correct):

The algorithm only returns mid after checking $A[mid] = key$ (line 3). Therefore the returned index is always correct ✓

Completeness (if key exists, algorithm finds it):

By the invariant, if key is in A , it is always within the current search interval. The interval size decreases by at least half each step: $n \rightarrow n/2 \rightarrow n/4 \rightarrow \dots \rightarrow 1$. After at most $\lceil \log_2(n + 1) \rceil$ steps, interval size becomes 1. At that point, $mid = first = last$, and we check if $A[mid] = key$. If yes, return it. If no, the subsequent call will have $first > last$, and by the invariant contrapositive, key was never in the array ✓

Conclusion:

Binary Search is correct. ■

Part 3: Hash Tables

3.1 Introduction & The Problem with Direct Addressing

A **direct-address table** is an array $T[0 \dots m - 1]$ where an element with key k is stored directly in slot $T[k]$. This provides $O(1)$ worst-case time for all dictionary operations (INSERT, SEARCH, DELETE).

The Downside of Direct Addressing:

If the universe U of possible keys is **large** or **infinite**, storing a table T of size $|U|$ becomes **impractical** or **impossible** given typical memory constraints. The set K of keys actually stored may be very small relative to U , causing most of the space allocated for T to be wasted.

Key Insight: When $|K| \ll |U|$ (the set of stored keys is much smaller than the universe of all possible keys), a hash table provides a much more storage-efficient solution.

3.2 Hash Table Core Concept

A **hash table** reduces the storage requirement to $\Theta(|K|)$ while maintaining $O(1)$ *average-case* search time.

Table 10: Direct Addressing vs. Hash Tables

Aspect	Direct Addressing	Hash Tables
Time Bound	Worst-case $O(1)$	Average-case $O(1)$
Storage	$\Theta(U)$	$\Theta(K)$
Slot Determination	Element with key k stored in slot k	Element with key k stored in slot $h(k)$

Hash Function Definition

$$h : U \rightarrow \{0, 1, \dots, m - 1\}$$

Where:

- U = universe of all possible keys.
- m = size of hash table (typically $m \ll |U|$).
- $h(k)$ = hash value of key k (the slot number where key k is stored).

Terminology:

- An element with key k **hashes to** slot $h(k)$.
- $h(k)$ is the **hash value** of key k .

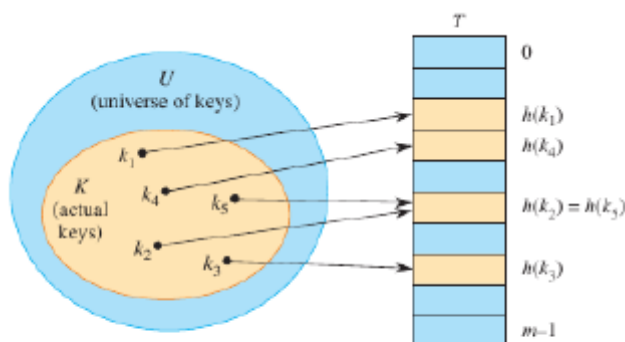


Figure 11.2 Using a hash function h to map keys to hash-table slots. Because keys k_2 and k_3 map to the same slot, they collide.

Figure 3: Using a hash function h to map keys to hash-table slots. Because keys k_2 and k_3 map to the same slot, they collide.

3.3 The Collision Problem

Definition: Collision

A **collision** occurs when two distinct keys $k_1 \neq k_2$ hash to the same slot: $h(k_1) = h(k_2)$.

Fundamental Constraint: Since $|U| > m$, collisions are **unavoidable** by the pigeonhole principle -- there must exist at least two keys with the same hash value.

Ideal vs. Reality:

- **Ideal:** Avoid collisions altogether.
- **Reality:** Impossible to avoid completely when $|U| > m$.
- **Practical Goal:** Use a well-designed hash function to **minimize** collisions, plus a **collision resolution technique**.

Important Note:

A hash function h must be **deterministic** -- a given input k must always produce the same output $h(k)$.

3.4 Independent Uniform Hashing

Definition: Independent Uniform Hashing (SUHA)

Each key is equally likely to hash to any of the m slots, independently of where any other keys have hashed to. Formally:

1. **Uniformity:** $\Pr[h(k) = i] = \frac{1}{m}$ for all $i \in \{0, 1, \dots, m - 1\}$.
2. **Independence:** The hash values of different keys are independent random variables.

Important caveats: In practice, truly uniform and independent hash functions do not exist for arbitrary key sets. SUHA is a theoretical model used to analyze expected performance. **Universal hashing** provides a practical way to approximate these assumptions by randomly selecting a hash function from a carefully designed family.

3.5 Collision Resolution by Chaining

Hashing with chaining is a **nonrecursive form of divide-and-conquer**:

- The input set of n elements is divided randomly into m subsets.
- Each subset has approximate size n/m .
- The hash function determines which subset an element belongs to.
- Each subset is managed independently as a **linked list**.

Structure

- Each nonempty slot points to a **linked list**.
- All elements that hash to the same slot go into that slot's linked list.
- Slot j contains a pointer to the **head** of the list of all stored elements with hash value j .
- If no elements hash to slot j , then slot j contains **NIL**.

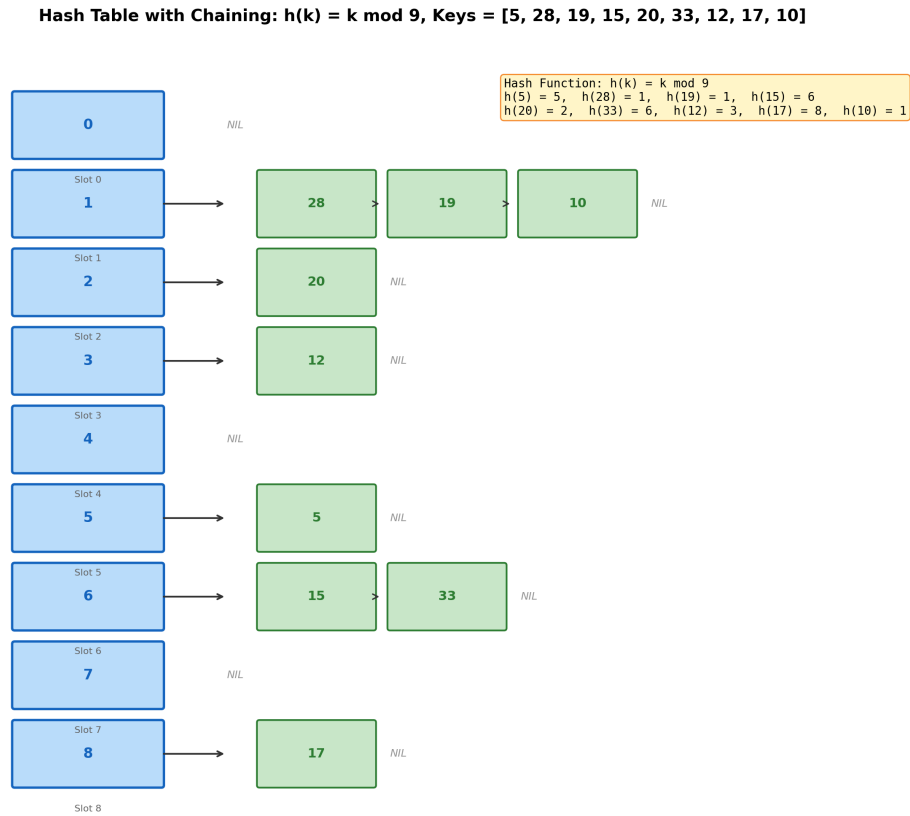


Figure 4: Hash table with chaining. Each slot points to a linked list of all keys that hash to it. Keys that collide are stored in the same linked list.

Dictionary Operations

Table 11: Chaining: dictionary operations

Operation	Pseudocode	Worst-Case Time	Expected Time (SUHA)
INSERT	LIST-PREPEND($T[h(x.key)], x$)	$O(1)$	$O(1)$
SEARCH	LIST-SEARCH($T[h(k)], k$)	$O(n)$	$O(1 + \alpha)$
DELETE	LIST-DELETE($T[h(x.key)], x$)	$O(1)$	$O(1)$

Deletion details:

- **Doubly linked lists:** $O(1)$ time (input is element x , not key k , so no search needed).
- **Singly linked lists:** Time proportional to list length.
- **Recommendation:** If the hash table supports deletion, linked lists should be **doubly linked**.

Expected Time Analysis Under SUHA

traverse the entire list at $h(k)$:

$$E[\text{unsuccessful search time}] = \Theta(1 + \alpha)$$

Analysis of successful search: Consider the i -th inserted element x_i . At the time of insertion, there are $i - 1$ elements already in the table. The expected length of the list where x_i is inserted is $(i - 1)/m$. The expected number of elements examined in a successful search for x_i is $1 + (i - 1)/m$ (1 for x_i itself, plus the expected number of elements inserted after x_i that hash to the same slot... actually, we need to average over all n elements):

$$\begin{aligned} E[\text{successful search time}] &= \frac{1}{n} \sum_{i=1}^n \left(1 + \frac{i-1}{m} \right) \\ &= 1 + \frac{1}{nm} \sum_{i=1}^n (i-1) \\ &= 1 + \frac{1}{nm} \cdot \frac{(n-1)n}{2} \\ &= 1 + \frac{n-1}{2m} \\ &= 1 + \frac{\alpha}{2} - \frac{1}{2m} \\ &= \Theta(1 + \alpha) \end{aligned}$$

Result:

Under SUHA, chaining achieves $\Theta(1 + \alpha)$ expected time for both successful and unsuccessful search. When $\alpha = O(1)$ (i.e., $n = O(m)$), all operations take $O(1)$ **expected time**.

3.6 Hash Table Operations & Pseudocode

CHAINED-HASH-INSERT(T, x)

```
1 LIST-PREPEND(T[h(x.key)], x)
```

CHAINED-HASH-SEARCH(T, k)

```
1 return LIST-SEARCH(T[h(k)], k)
```

CHAINED-HASH-DELETE(T, x)

```
1  LIST-DELETE( $T[h(x.key)], x$ )
```

===== PART 4: HASH FUNCTIONS ===== -->

Part 4: Hash Functions

4.1 What Makes a Good Hash Function?

A good hash function must satisfy two key requirements:

1. **Efficiently computable:** The hash value should be computed quickly, ideally in $O(1)$ time.
2. **Satisfies (approximately) independent uniform hashing:** Each key is equally likely to hash to any of the m slots, independently of where any other keys have hashed to.

The Problem with "Equally Likely": If the hash function is fixed, probabilities depend on the **probability distribution of input keys**. Typically, you have no way to verify this condition unless you know the key distribution. Keys might not be drawn independently.

Known distribution example: If keys are random real numbers k , independently and uniformly distributed in range $0 \leq k < 1$, then $h(k) = \lfloor km \rfloor$ satisfies independent uniform hashing.

4.2 Static vs. Random Hashing

Table 12: Static Hashing vs. Random Hashing

Aspect	Static Hashing	Random Hashing
Definition	Hash function is fixed and does not change with the input.	Hash function is chosen randomly from a family of hash functions at runtime.
Universality	Not universal; may not evenly distribute keys.	Universal hash families guarantee low collision probability $\leq 1/m$.
Performance	Deterministic; may perform poorly on certain key sets.	Probabilistic guarantees; good average performance on any data.
Use Case	Simple implementations, known key distributions.	Robust against adversarial inputs, theoretical analysis.
Examples	Division method, Multiplication method.	Universal hash families, Multiply-Shift method.

Key Insight:

Random hashing provides provably good average-case performance for **any data** without knowing the input distribution. The randomization comes from the **known** process of picking the hash function, not the **unknown** process creating the input keys. **Universal hashing** is a particular kind of random hashing that works well.

Key Types in Practice

Hash functions handle keys of two types:

- **Type 1:** Short nonnegative integers -- fits in a w -bit machine word (typical $w = 32$ or 64).
- **Type 2:** Short vector of nonnegative integers (e.g., byte strings of variable length).

4.3 The Division Method**Formula:**

$$h(k) = k \bmod m$$

How it works: Computes the remainder when key k is divided by table size m .

Example: If $m = 12$ and $k = 100$, then $h(k) = 100 \bmod 12 = 4$.

Table 13: Division Method characteristics

Feature	Description
Speed	Very fast -- requires only a single division operation.
Constraint	Works best when m is a prime number not too close to an exact power of 2 .
Limitation	No guarantee of good average-case performance; constrains hash table size to be prime.

Critical Choice: Avoid powers of 2!

If $m = 2^p$, then $h(k) = k \bmod 2^p$ depends **only on the lowest p bits** of k , ignoring all higher bits. This causes many collisions when keys share the same low bits. **Best practice:** Choose m to be a prime number not too close to a power of 2.

4.4 The Multiplication Method

General Formula (Two Steps):

1. Multiply key k by constant A (where $0 < A < 1$) and extract the **fractional part** of kA .
2. Multiply this value by m and take the **floor** of the result.

$$h(k) = \lfloor m \cdot (kA \bmod 1) \rfloor$$

where $kA \bmod 1$ means the fractional part of kA , i.e., $kA - \lfloor kA \rfloor$.

Advantages:

- Value of m is **not critical** -- can choose m independently of how you choose A .
- Spreads keys more uniformly across all slots.
- Provides good foundation for **random hashing** (nonstatic hashing).

Good choice of A : Knuth suggests $A \approx (\sqrt{5} - 1)/2 \approx 0.6180339887...$ (the golden ratio conjugate). This irrational number tends to spread keys uniformly.

Example: For $m = 8$, $A = 0.618$, $k = 31$:

$$kA = 31 \times 0.618 = 19.158$$

$$kA \bmod 1 = 0.158$$

$$h(31) = \lfloor 8 \times 0.158 \rfloor = \lfloor 1.264 \rfloor = 1$$

4.5 The Multiply-Shift Method

This is a highly efficient hash function based on bitwise operations.

Formula:

Treat key k and a random odd constant a as w -bit integers. Compute:

$$h(k) = (a \cdot k) \gg (w - p)$$

where $m = 2^p$ (table size is a power of 2), and $\gg$ denotes right bit shift.

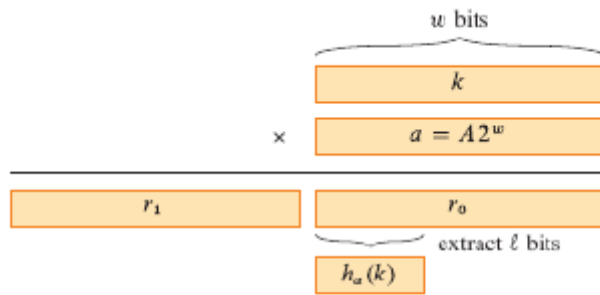


Figure 11.4 The multiply-shift method to compute a hash function. The w -bit representation of the key k is multiplied by the w -bit value $a = A \cdot 2^w$. The ℓ highest-order bits of the lower w -bit half of the product form the desired hash value $h_a(k)$.

Figure 5: The multiply-shift method: the w -bit representation of key k is multiplied by the w -bit value $a = A \cdot 2^w$. The ℓ highest-order bits of the lower w -bit half form the hash value $h_a(k)$.

How it works visually (from Figure 5):

1. Key k is represented as w bits.
2. Multiply k by w -bit value $a = A \cdot 2^w$.
3. The product produces two w -bit halves: r_1 (high) and r_0 (low).
4. Extract ℓ highest-order bits of the lower w -bit half (r_0).
5. These ℓ bits form the desired hash value $h_a(k)$.

Table 14: Multiply-Shift Method characteristics

Feature	Description
Speed	Extremely fast: single multiply + shift (2–3 CPU cycles on modern hardware).
Table size	Must be a power of 2 ($m = 2^p$).
Constant a	Must be odd (ensures multiplication is a bijection in w -bit space).
Theoretical guarantee	Part of 2-independent hash families.

Why must a be odd?

If a were even, then $a \cdot k$ would always be even, meaning the least significant bit would always be 0, severely limiting the hash distribution. An odd a ensures the multiplication is a bijection (permutation) in the w -bit space, preserving the randomness of key bits in the product.

Comparison of Hash Function Methods

Table 15: Comparison of hash function methods

Feature	Division	Multiplication	Multiply-Shift
Formula	$k \bmod m$	$\lfloor m(kA \bmod 1) \rfloor$	$(a \cdot k) \gg (w - p)$
Table Size	Best: prime	Any value	Must be power of 2
Speed	Fast	Moderate	Very fast
Key Dependency	All bits of k	All bits (via multiplication)	All bits (via multiplication)
Independence Level	None (static)	None (static)	2-independent

Part 5: Open Addressing

5.1 Concept & Overview

In **open addressing**, all elements are stored **directly in the hash table itself**. When a collision occurs, alternative slots within the table are probed until an empty slot is found.

Key difference from chaining:

- **Chaining:** Colliding keys are stored outside the table (in linked lists).
- **Open Addressing:** All keys are stored within the table; no external storage is used.

Probe sequence: For a key k , the hash function generates a sequence of slots to try:

$$(h(k, 0), h(k, 1), \dots, h(k, m - 1))$$

This sequence should be a **permutation** of $\{0, 1, \dots, m - 1\}$ to ensure all slots are eventually checked.

Table 16: Chaining vs. Open Addressing

Feature	Chaining	Open Addressing
Storage	Colliding keys stored in linked lists outside the table.	All keys stored within the table; alternative slots probed.
Extra Space	$O(n)$ for pointers and nodes.	No extra space beyond the table.
Load Factor	$\alpha = n/m$ can exceed 1 (unbounded).	Must have $\alpha \leq 1$ (cannot store more keys than slots).
Cache Performance	Poor: linked lists scatter memory accesses.	Better: all data in contiguous array.
Deletion	Simple: remove from linked list.	Complex: requires special handling (tombstones).
Clustering	No clustering within the table.	Primary and secondary clustering possible.
Expected Search	$O(1 + \alpha)$	$O\left(\frac{1}{1-\alpha}\right)$ for uniform probing

Open Addressing: Core Operations

HASH-INSERT(T, k)

```

1   $i \leftarrow 0$ 
2  repeat
3       $q \leftarrow h(k, i)$ 
4      if  $T[q] == \text{NIL}$  then
5           $T[q] \leftarrow k$ 
6          return  $q$ 
7      else
8           $i \leftarrow i + 1$ 
9  until  $i == m$ 
10 error "hash table overflow"
```

HASH-SEARCH(T, k)

```

1   $i \leftarrow 0$ 
2  repeat
3       $q \leftarrow h(k, i)$ 
4      if  $T[q] == k$  then
5          return  $q$ 
6       $i \leftarrow i + 1$ 
7  until  $T[q] == \text{NIL}$  or  $i == m$ 
8  return NIL
```

5.2 Linear Probing

Linear probing is the **simplest open-addressing approach** to resolving collisions. It is a **special case of double hashing** where the step size is fixed at 1.

Probe Function:

$$h(k, i) = (h'(k) + i) \bmod m$$

where $h'(k)$ is an ordinary hash function and $i = 0, 1, 2, \dots$ is the probe number.

How Linear Probing Works

1. An auxiliary hash function h' determines the first probe position $h'(k)$.
2. If slot $T[h'(k)]$ is occupied, probe the next position $T[h'(k) + 1]$.

3. Keep going as necessary, up to slot $T[m - 1]$.
4. **Wrap around** to slots $T[0]$, $T[1]$, and so on.
5. Never go past slot $T[h'(k) - 1]$ (ensures complete cycle without repetition).

Characteristics

Table 17: Linear Probing characteristics

Feature	Description
Probe sequence	$h'(k)$ alone determines the entire probe sequence.
Distinct probe sequences	Only m distinct probe sequences (one for each value of $h'(k)$).
Step size	Fixed at 1.

Primary Clustering

Primary Clustering Problem:

Linear probing suffers from **primary clustering** -- long runs of occupied slots build up, increasing average search time. Since all keys that hash to the same initial position follow the same probe sequence, collisions tend to cluster together.

Performance Analysis

For an unsuccessful search at load factor $\alpha = n/m$:

$$\text{Expected probes} \approx \frac{1}{2} \left(1 + \frac{1}{(1 - \alpha)^2} \right)$$

For a successful search:

$$\text{Expected probes} \approx \frac{1}{2} \left(1 + \frac{1}{1 - \alpha} \right)$$

Performance at different load factors:

At $\alpha = 0.5$: ~2.5 probes for unsuccessful search.

At $\alpha = 0.75$: ~8.5 probes.

At $\alpha = 0.9$: ~50 probes -- performance degrades rapidly as $\alpha \rightarrow 1$.

5.3 Quadratic Probing

Quadratic probing uses a quadratic function of the probe number to determine the next slot to examine, eliminating primary clustering.

Probe Function:

$$h(k, i) = (h'(k) + c_1 i + c_2 i^2) \bmod m$$

where c_1 and c_2 are constants (commonly $c_1 = 0$, $c_2 = 1$ or using triangular numbers).

Common simplification: Using $c_1 = c_2 = 1/2$ gives triangular number offsets:

$$h(k, i) = \left(h'(k) + \frac{i(i+1)}{2} \right) \bmod m$$

How Quadratic Probing Works

Quadratic Probing: $h(k,i) = (h'(k) + c_1i + c_2i^2) \bmod m$, with $c_1=0$, $c_2=1$

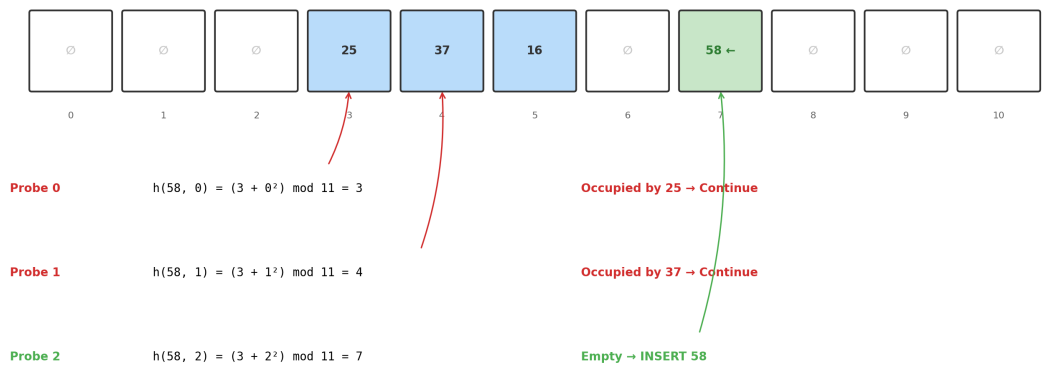


Figure 7: Quadratic Probing: inserting key 58 with $h'(58) = 3$. Probe 0: slot 3 (occupied); Probe 1: $(3 + 1) = 4$ (occupied); Probe 2: $(3 + 4) = 7$ (empty) -- insert 58. Quadratic gaps avoid primary clustering.

Characteristics

Table 18: Quadratic Probing characteristics

Feature	Description
Clustering	Avoids primary clustering because keys with the same initial hash follow different probe sequences.
Secondary Clustering	Still suffers from secondary clustering: if two keys have the same initial hash value, they follow the same probe sequence.
Slot coverage	If m is not chosen carefully, the probe sequence may not cover all slots. Must choose m to be prime and $c_2 < m$.

5.4 Double Hashing

Double hashing uses **two hash functions** to determine the probe sequence. The first gives the initial position; the second gives the step size.

Probe Function:

$$h(k, i) = (h_1(k) + i \cdot h_2(k)) \bmod m$$

where $h_1(k)$ and $h_2(k)$ are two independent hash functions.

Requirements for Full Coverage

For the entire hash table to be searched, $h_2(k)$ **must be relatively prime to m** (i.e., $\gcd(h_2(k), m) = 1$).

Two convenient methods to ensure this:

Table 19: Ensuring full coverage in double hashing

Method	Description
Method 1	Let m be an exact power of 2, and design h_2 so that it always produces an odd number .
Method 2	Let m be prime , and design h_2 so that it always returns a positive integer less than m .

Practical Example

Given: $k = 123456$, $m = 701$ (prime), $m' = 700$.

Auxiliary hash functions:

$$h_1(k) = k \bmod m \quad h_2(k) = 1 + (k \bmod m')$$

Calculation:

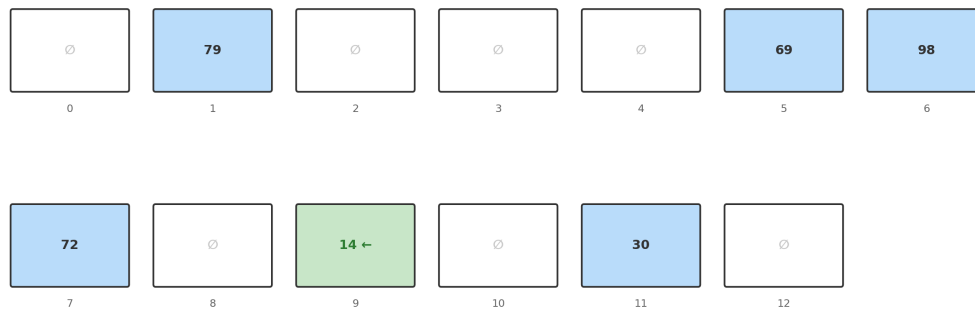
$$h_1(123456) = 123456 \bmod 701 = \mathbf{80}$$

$$h_2(123456) = 1 + (123456 \bmod 700) = 1 + 256 = \mathbf{257}$$

Probe sequence:

- First probe: position **80**
- Successive probes: examine every **257th slot** (modulo 701)
- Sequence: 80, $(80 + 257) \bmod 701 = 337$, $(337 + 257) \bmod 701 = 594, \dots$

Double Hashing: Inserting key 14, $h_1(k) = k \bmod 13$, $h_2(k) = 1 + (k \bmod 11)$



Probe 0 $h(14, 0) = (h_1(14) + 0 \cdot h_2(14)) \bmod 13 = (1 + 0) \bmod 13 = 1$

Slot 1: Occupied by 79 → Continue

Probe 1 $h(14, 1) = (1 + 1 \cdot 4) \bmod 13 = 5$

Slot 5: Occupied by 69 → Continue

Probe 2 $h(14, 2) = (1 + 2 \cdot 4) \bmod 13 = 9$

Slot 9: Empty → INSERT 14

Parameters:
 $h_1(k) = k \bmod 13$
 $h_2(k) = 1 + (k \bmod 11)$
 $k = 14: h_1(14) = 1, h_2(14) = 4$
 Probe sequence: $(1 + i \cdot 4) \bmod 13$
 $= \{1, 5, 9, 0, 4, 8, 12, 3, 7, 11, 2, 6, 10\}$

Figure 8: Double Hashing: inserting key 14 with $h_1(14) = 1$, $h_2(14) = 4$ into a table of size 13. Probe 0: slot 1 (occupied by 79); Probe 1: slot 5 (occupied by 69); Probe 2: slot 9 (empty) – insert 14.

Number of Probe Sequences

Theorem:

When m is **prime** or an **exact power of 2**, double hashing produces $\Theta(m^2)$ distinct probe sequences.

Reason: Each possible $(h_1(k), h_2(k))$ pair yields a **distinct probe sequence**. For such values of m , double hashing appears to perform close to the "ideal" scheme of independent uniform permutation hashing.

Proof: Double Hashing Covers All Slots

Theorem: If $\gcd(h_2(k), m) = 1$, then the probe sequence generates a permutation of $\{0, 1, \dots, m - 1\}$.

Proof: Suppose two probe numbers i and j (with $0 \leq i < j < m$) yield the same slot:

$$(h_1(k) + i \cdot h_2(k)) \equiv (h_1(k) + j \cdot h_2(k)) \pmod{m}$$

Subtracting $h_1(k)$ from both sides:

$$i \cdot h_2(k) \equiv j \cdot h_2(k) \pmod{m}$$

$$(j - i) \cdot h_2(k) \equiv 0 \pmod{m}$$

This means m divides $(j - i) \cdot h_2(k)$. But $0 < j - i < m$ and $\gcd(h_2(k), m) = 1$, so m cannot divide $(j - i)$ -- contradiction.

Therefore, all m probe positions generate distinct slots, forming a complete permutation. ■

5.5 Deletion in Open Addressing

Deletion is straightforward in chaining (remove from linked list) but **problematic in open addressing**.

The Problem with Direct Deletion

Consider: Key A was inserted at slot 5 after probing. Later, key B (which hashes to the same initial slot) was inserted at slot 6 after finding slot 5 occupied. If we delete A from slot 5, making it empty, a search for B would stop at slot 5 (which now appears empty) and **incorrectly conclude B is not in the table**.

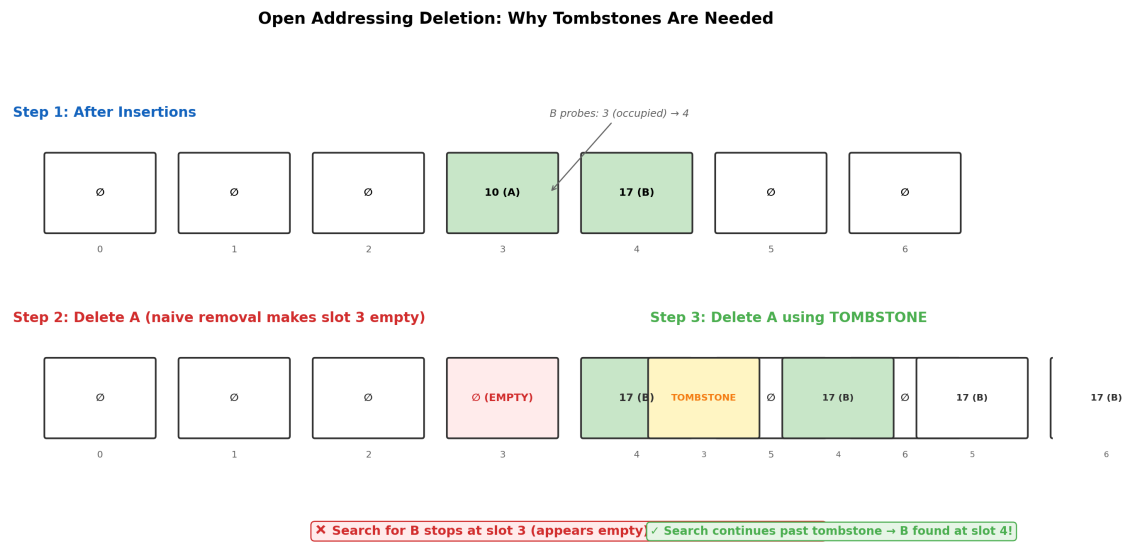


Figure 9: Why tombstones are needed: (top) after insertions; (middle) naive deletion of A makes slot 3 empty, breaking search for B; (bottom) tombstone preserves probe chain while marking slot as reusable for insertion.

Solution 1: Tombstones (Lazy Deletion)

Instead of clearing the slot, mark it with a special **tombstone** (or "deleted") marker.

Table 20: Tombstone behavior

Slot State	Search Behavior	Insertion Behavior
Occupied	Compare key; if match, found; else continue probing.	Continue probing.
Tombstone (Deleted)	Continue probing (key might be further ahead).	Insert here (slot reusable).
Empty (NIL)	Stop: key not present.	Insert here.

Solution 2: Rehashing

When the number of tombstones exceeds a threshold, **rebuild the entire hash table** by re-inserting all remaining keys into a fresh table. This removes all tombstones and restores optimal $O(1)$ expected performance.

Tombstone Accumulation Problem:

Over many deletions, tombstones can accumulate and degrade search performance because searches must traverse through them. Periodic rehashing is necessary when tombstones exceed a threshold (e.g., 1/4 of the table).

5.6 Comparison of Probing Methods

Table 21: Comparison of open addressing probing methods

Feature	Linear Probing	Quadratic Probing	Double Hashing
Formula	$(h'(k) + i) \bmod m$	$(h'(k) + c_1 i + c_2 i^2) \bmod m$	$(h_1(k) + i \cdot h_2(k)) \bmod m$
Step size	1 (fixed)	Quadratic in i	$h_2(k)$ (varies with key)
Probe sequences	m	m	$\Theta(m^2)$
m requirements	Any	Prime, $c_2 < m$	Prime or power of 2
Primary clustering	Yes (severe)	No	No
Secondary clustering	Yes	Yes	No
Cache locality	Excellent	Moderate	Poor
Expected probes (unsuccessful)	$\frac{1}{2}(1 + \frac{1}{(1-\alpha)^2})$	$\frac{1}{1-\alpha}$ (approx.)	$\frac{1}{1-\alpha}$
Expected probes (successful)	$\frac{1}{2}(1 + \frac{1}{1-\alpha})$	$-\frac{1}{\alpha} \ln(1-\alpha)$ (approx.)	$\frac{1}{\alpha} \ln \frac{1}{1-\alpha}$

Part 6: Summary & Comparison Tables

6.1 Search Algorithms Comparison

Table 22: Linear Search vs. Binary Search

Feature	Linear Search	Binary Search
Prerequisite	None	Sorted array
Approach	Sequential	Divide & Conquer
Best Case	$\Theta(1)$	$\Theta(1)$
Average Case	$\Theta(n)$	$\Theta(\log n)$
Worst Case	$\Theta(n)$	$\Theta(\log n)$
Space	$O(1)$	$O(1)$ iterative / $O(\log n)$ recursive
In-Place	Yes	Yes
Adaptive	Partially	No
Data Structure	Any sequential	Random access (arrays)

6.2 Hashing Concepts Summary

Table 23: Key hashing concepts

Concept	Definition
Hash Table	Data structure storing elements in slots computed by hash function; size $m \ll U $.
Hash Function $h(k)$	Maps key k to slot index in $\{0, 1, \dots, m - 1\}$.
Collision	Two keys $k_1 \neq k_2$ where $h(k_1) = h(k_2)$.
Chaining	Collision resolution where each slot points to linked list of colliding elements.

Open Addressing	Collision resolution where all elements store in the table itself; no external storage.
Load Factor α	n/m (average elements per slot).
Independent Uniform Hashing	Each key equally likely to hash to any slot, independent of other keys.
Universal Hashing	Randomly selecting a hash function from a family to approximate SUHA.
Static Hashing	Single fixed hash function; not recommended for adversarial data.
Random Hashing	Hash function chosen randomly from family at runtime; recommended.
Primary Clustering	Long runs of occupied slots forming in linear probing.
Secondary Clustering	Keys with same initial hash following same probe sequence.
Tombstone	Special marker for deleted slots in open addressing to preserve probe chains.
Rehashing	Rebuilding the hash table to remove tombstones and restore performance.

6.3 Collision Resolution Comparison

Table 24: Collision resolution strategies: detailed comparison

Feature	Chaining	Linear Probing	Quadratic Probing	Double Hashing
Storage	External (linked lists)	Internal	Internal	Internal
Extra Space	$O(n)$ pointers	$O(1)$	$O(1)$	$O(1)$
α can exceed 1?	Yes	No	No	No
Deletion	Easy ($O(1)$ with doubly linked)	Tombstones needed	Tombstones needed	Tombstones needed
Clustering	None	Primary	Secondary	None
Expected search	$O(1 + \alpha)$	$O(\frac{1}{1-\alpha})$ (worse)	$O(\frac{1}{1-\alpha})$	$O(\frac{1}{1-\alpha})$

Cache performance	Poor	Excellent	Moderate	Poor
-------------------	------	-----------	----------	------

6.4 Hash Function Methods Comparison

Table 25: Hash function methods: comparison

Feature	Division	Multiplication	Multiply-Shift
Formula	$k \bmod m$	$\lfloor m(kA \bmod 1) \rfloor$	$(a \cdot k) \gg (w - p)$
Speed	Fast (1 division)	Moderate (mult + frac)	Very fast (mult + shift)
Table size m	Best: prime	Any value	Must be 2^p
Distribution quality	Moderate	Good	Good (2-independent)
Best use case	Simple implementations	Flexible table sizes	High-performance systems

Key Takeaways:

- **Linear Search:** Simple, no prerequisites, $O(n)$ -- use for small or unsorted data.
- **Binary Search:** Requires sorted data, $O(\log n)$ -- use for large sorted arrays.
- **Hash Tables:** Average $O(1)$ for all operations -- best for frequent lookups with key-based access.
- **Chaining:** Simple, handles overflow, but uses extra space for pointers.
- **Open Addressing:** No extra space, but needs careful probing and tombstones for deletion.
- **Double Hashing:** Best probing method -- eliminates clustering with $\Theta(m^2)$ probe sequences.